OMIX: Offline Mixing for Scalable Self-Tallying Elections

Sébastien Canard, Liam Medley, Duy Nguyen, and Duong Hieu Phan

Telecom Paris, Institut Polytechnique de Paris, France
{sebastien.canard, liam.medley, dinh.nguyen, hieu.phan}@telecom-paris.fr

Abstract. In electronic voting systems, guaranteeing voter anonymity is essential. One primary method to ensure this is the use of a *mix-net*, in which a set of *mix-servers* sequentially shuffle a set of encrypted votes, and generate proofs that a correct permutation has been applied. Whilst mix-nets offer advantages over alternative approaches, their traditional use during the tallying phase introduces a significant robustness bottleneck: the process is inherently sequential and critically depends on trusted authorities to perform shuffling and decryption. Any disruption can prevent the final result from being revealed.

In this work, we propose offline mixing (OMIX), the first voting framework to support a mix-net-based system in which trustees never handle encrypted votes, while also ensuring that each voter's cost is independent of the total number of voters. In particular, the contributions of permutations by mix-servers and decryption shares by trustees are completed and publicly verified before any vote is cast. This eliminates the need for their participation during tallying and enables the first scalable, mix-net-based, and self-tallying voting protocol in the sense of Kiayias and Yung (PKC'02).

At the core of OMIX is a distributed key-generation mechanism: each voter locally generates a private voting key and registers a constant-size set of basis public keys. These are permuted and partially decrypted in an offline phase, resulting in a final public decryption key that reveals votes in shuffled order. Our construction leverages the homomorphic and structure-preserving properties of function-hiding inner-product functional encryption, combined with standard primitives, to achieve self-tallying, client scalability, ballot privacy and other voting properties. To support the new mixing structure introduced by OMIX, we also develop a compact and verifiable offline mix-net, based on an enhanced linearly-homomorphic signature scheme. This latter primitive may be of independent interest.

Keywords: voting, mix-net, self-tallying, client-scalable, functional encryption, linearly-homomorphic signature

Table of Contents

1 Introduction

Privacy is a fundamental goal of cryptography and a critical requirement for many applications, particularly in secure electronic voting (e-voting). One of the key mechanisms to ensure voter privacy is through a valid *shuffle* protocol, which ensures that ciphertexts (encrypted votes) are shuffled before they are decrypted. In such a protocol, the input is a set of encrypted votes, and the output is the corresponding plaintexts in a permuted order, such that it is impossible to associate any input ciphertext with its corresponding plaintext.

A widely used approach to achieve this shuffle is the *mix-net*. In a mix-net, a group of mix servers sequentially shuffle and re-encrypt (or decrypt) the ciphertexts, ensuring that as long as one server is honest, the initial order of the plaintexts remains private. Since its introduction by Chaum in 1981 [16], the mix-net has proven to be a valuable cryptographic primitive, particularly because it supports elections with complex ballots and arbitrary voting formats.

Consider the standard approach to mix-net-based voting: after ballots are cast, a group of trusted authorities are responsible for mixing and decrypting the ciphertexts in order to compute the final result. Many existing works explore how to verify that these authorities have fulfilled their duties correctly, typically through zero-knowledge proofs of correct shuffling and decryption. However, a critical issue remains insufficiently addressed: how to prevent these authorities from disrupting the election altogether. For instance, an authority may refuse to participate, abort midway, or submit an invalid proof. Any of these actions could obstruct the tallying process and ultimately spoil the outcome. This risk is particularly relevant in practice, as modern elections often release exit polls before official results. This presents an opportunity for a politically influenced authority, upon seeing from an exit poll that the result is unlikely to be favourable, to disrupt the protocol, preventing the result from being returned and damaging public trust in the election system.

To address this, we introduce a new voting framework based on an *offline mix-net*, in which any party can verify that the tasks assigned to mix-servers and decryption trustees are correctly completed during an offline phase, prior to voting. More importantly, the (online) tallying process can then be carried out entirely in public by any casual party, with no authority involved. In other words, our framework eliminates dependence on trusted authorities holding secret keys for computing the result and shifts this capability to the public.

The notion that an election result can be computed publicly from the voters' ballots alone was first introduced under the term *self-tallying* by Kiayias and Yung in [38]. Whilst self-tallying has been achieved in small-scale settings (e.g., boardroom voting), for both homomorphic and mix-net-based systems, extending it to large-scale mix-net-based elections remains a long-standing open problem. The primary challenge lies in the nature of mix-nets: the random permutation used to shuffle ciphertexts acts as a secret key that preserves voter anonymity. The final outcome depends on this key, and traditional mix-net approaches require a mixing authority to generate it through shuffling the ciphertexts. Therefore, achieving self-tallying in mix-net-based systems necessitates a way to separate the permutation generation from shuffling, and move the former entirely into the offline phase. At the same time, for a voting system to be practical at large scale, it must be *client-scalable*, meaning that each voter's cost remains independent of the total number of voters. These issues raise the following question:

> *Is it possible to construct a voting system that is simultaneously client-scalable, mix-net-based, and self-tallying?*

Moreover, reducing the complexity of mix-net verification remains an important goal, with a long line of research devoted to improving verification techniques. This motivates a second question:

> *Is it possible to construct an offline-verifiable mix-net in which the proof size and verification cost are independent of the number of mix-servers?*

In this work, we answer both questions affirmatively through the construction of the OMIX framework.

1.1 Our Results

To address the first question, we propose a novel key-generation mechanism for anonymity, in which a public decryption key is generated during the *offline phase* and ensures that votes are revealed only

in a randomly permuted order. The resulting self-tallying election is illustrated in Figure 1. This is in contrast to the standard approach shown in Figure 2, where the mix-servers and decryption trustees must handle ballots directly, and no self-tallying property is achieved.

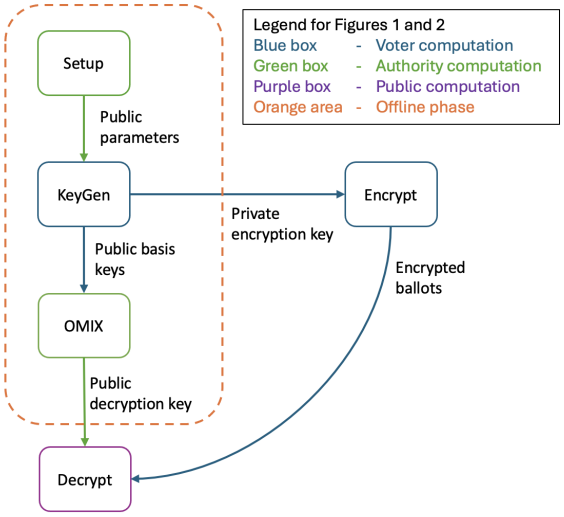

Fig. 1. Our new approach to voting with mix-nets. We separate the permutation generation and shuffling steps, which are executed simultaneously during standard mixing. In the offline phase, mix-servers embed a hidden permutation into a public decryption key. Using this key, any party can perform an oblivious shuffle of votes via decryption, meaning the shuffle is carried out without knowledge of the permutation.

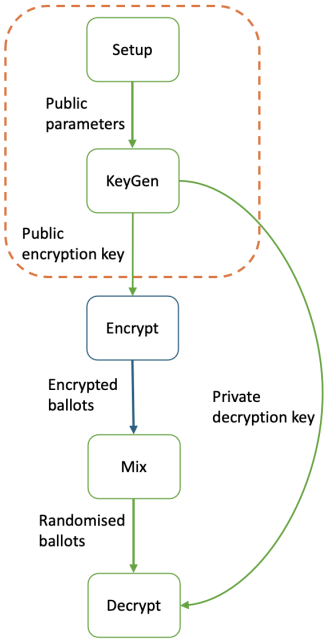

Fig. 2. The standard approach to voting using a mix-net. The mixing and decrypting steps require the respective authorities to be online during the tallying phase, where encrypted votes are mixed and decrypted.

To realise the voting scheme in Figure 1, we leverage the concept of *functional encryption*, a primitive that allows only specific function evaluations of encrypted data to be revealed via functional decryption keys. We demonstrate that it is possible to privately embed a permutation into the voters' registered functional decryption keys. Moreover, this embedding process can be collaboratively performed by multiple authorities during the election setup phase to enhance privacy. During the voting phase, each voter simply encrypts and casts their vote. The tallying phase then reduces to a decryption process, where the embedded functional decryption keys are applied to the voters' ciphertexts to produce the global, shuffled result.

An important advantage of our techniques is that it offers client scalability, a voter-friendly property crucial for large-scale elections. Specifically, the communication and computational costs incurred by each voter remain constant regardless of the total number of voters and parties involved in the protocol. Regarding round complexity, at most three casts from voters are required: one for registering, one for voting, and one for handling dropout voters.

It is important to note that a self-tallying scheme cannot simultaneously guarantee both robustness and privacy (see Proposition 1 in [38]), because if any casual party can tally an arbitrary subset of encrypted votes and obtain the correct result, individual votes will be revealed. Consequently, the tally typically involves all ballots from registered voters. To manage dropout voters who fail to cast their ballots, the tally might require actions from the remaining honest voters, a property known as *corrective fault tolerance* [38]. In our protocol, we achieve this property while still ensuring that such corrective actions can be completed in constant time, thereby confining the involvement of key authorities strictly to the offline phase throughout the entire election. A comparison table with other existing works is provided in Figure 3.

Voting System	Result Function	Client Scalability	Corrective Fault Tolerance	No Mix-servers in Tallying	No Trustees in Tallying
KY1	Aggregate	○	●	N/A	●
KY2	Aggregate	◐	●	N/A	◑
Groth1	Aggregate	○	●	N/A	●
Groth2	Shuffle	○	○	●	●
[5]	Shuffle	○	●	○	●
[56]	Aggregate	○	N/A	N/A	●
Our work	Shuffle	●	●	●	●

Fig. 3. Comparison with a relevant but not exhaustive list of self-tallying voting systems. We highlight [56] as the current state-of-the-art system based on homomorphic aggregation, and [5] as the most recent work addressing mix-nets. The schemes in the table are labeled as KY1 and KY2 for the two schemes from [38] (see Sections 3 and 4.3, respectively), and Groth1 and Groth2 for those from [28] (see Sections 2 and 3, respectively). The symbols ● and ○ indicate that the corresponding property is achieved and not achieved, respectively. The symbols ◐ and ◑ denote that the two properties cannot be achieved simultaneously. "N/A" means the property is not applicable.

Our protocols are described in Section 3 and Section 4. Although this is primarily a conceptual work, our construction relies solely on existing cryptographic primitives that have been shown to be instantiable and interoperable (e.g., within asymmetric pairing groups). We leave the development of an optimised implementation to future work. Furthermore, to facilitate the analysis of our novel approach, we formalise a syntax for self-tallying voting systems, along with precisely defined properties such as corrective fault tolerance, client scalability, ballot privacy, and universal verifiability (see Section 2 for details).

To address the second question, we aim to achieve offline verification that scales with the number of mix-servers. Our initial approach is to adapt the idea of using *linearly-homomorphic signatures* (LHS) with re-randomisable tags, as introduced in [35]. At a high level, LHS acts as a malleable proof system for the language of ciphertext re-randomisation. Each tag is associated with a specific ciphertext, defining a distinct statement. The re-randomisability of the tags ensures unlinkability between the original and the shuffled ciphertexts. Thanks to the malleability property, each mix-server can build its proof incrementally on top of the proof generated by the previous servers, thereby achieving scalable proof size and verification time.

However, several security challenges arise in adapting LHS to our setting, since mix-servers in our framework are required to shuffle rows of a matrix, rather than elements of a vector. To address this, we introduce two key improvements: a stronger unforgeability notion and a new tagging structure. Together, these yield a novel LHS scheme in the generic group model that satisfies the security requirements of our offline mix-net (see Section 5). We believe this contribution may be of independent interest, as it demonstrates that LHS can support a broader class of shuffling languages beyond vector-based settings.

1.2 Technical Overview

Proof of Concept Protocol To illustrate our framework, in Section 3 we introduce a simplified setting with an authority (single trustee) that is responsible for registering voters, generating private encryption keys and a public decryption key that encapsulates a random permutation in the offline phase. Assuming there are n registered voters, our protocol can be briefly described as follows:

– **Setup Phase**: The authority generates, for each voter, a secret FH-IPFE key and a pseudorandom zero-sharing key t_i. The FH-IPFE secret key allows its owner to perform both encryption and functional-key generation. The zero-sharing key t_i allows its owner to non-interactively compute

a random share that, when summed with the shares of other voters, results in zero. Both keys are known only to the authority and the corresponding voter, while a public version $[\![t_i]\!]_2$ is published as voter i's public key.

- *Permutation Generation*: The authority randomly generates a permutation Π over the index set $[n]$. Let $\boldsymbol{P}$ represent the corresponding permutation matrix for Π. In this matrix, each column $\boldsymbol{p}^i$ has an entry $p^i_j = 1$ if $j = \Pi(i)$, and $p^i_j = 0$ otherwise. This structure means that the vote of voter i will appear in position j in the result.
- *Public-Key Aggregation*: The authority computes an aggregate of the public keys of all other voters, defined as $\mathsf{pz}_i = \sum_{k\in[n]\setminus\{i\}}(-1)^{k<i}[\![t_k]\!]_2$ for each voter i. This step is fully transparent, as pz_i is publicly published and can be independently verified.
- *Functional-Key Generation*: The authority generates a one-time pad vector $\boldsymbol{z}$, where each entry z_j corresponds to a row index j of the matrix $\boldsymbol{P}$. This one-time pad is used to re-randomise the zero shares in ciphertexts during FH-IPFE decryption. For each entry p^i_j, the authority uses voter i's secret key to generate a corresponding functional key for the vector $([\![p^i_j]\!]_1, [\![z_j]\!]_1)$. The authority then publishes the permutation Π in the form of these functional keys, along with a public encoding $[\![\boldsymbol{z}]\!]_1$ of the one-time pad, as part of an updated master public key. These are made available for use during the tallying phase.

– **Voting Phase**: Upon registration, each voter is assigned a secret key for FH-IPFE and a secret-key for zero-sharing. The voter encrypts their vote of the form $([\![v_i]\!]_2, \mathsf{zero}_i := t_i \cdot \mathsf{pz}_i)$ during the online voting phase. Here, $t_i \cdot \mathsf{pz}_i$ serves as voter i's additive share of $[\![0]\!]_2$. The encrypted ballots are then submitted to a public ballot box.

– **Tallying Phase**: After the voting phase concludes (with no additional ballots allowed), let $\mathsf{V} \subseteq [n]$ be the set of voters whose ballots are in the box. There are two cases:

- If $\mathsf{V} = [n]$, i.e., all registered voters cast their ballots, the tally proceeds via public decryption using the published ciphertexts and functional keys. The result is computed as:

$$[\![\boldsymbol{v'}]\!]_T = (\sum_{i\in[n]} [\![v_i \cdot p^i_j]\!]_T + \sum_{i\in[n]} \mathsf{zero}_i \cdot z_j)_{j\in[n]} = (\sum_{i\in[n]} [\![v_i \cdot p^i_j]\!]_T)_{j\in[n]} = [\![\boldsymbol{P} \cdot \boldsymbol{v}]\!]_T$$

  This result is thus a permuted version of the vote vector.
- If $\mathsf{V} \subsetneq [n]$, i.e., some registered voters failed to submit their ballots, corrective action is required from the remaining honest voters. Each voter $i \in \mathsf{V}$ publishes a correcting key: $\mathsf{ck}_i = (\sum_{k\in[n]\setminus\mathsf{V}}(-1)^{k<i}[\![t_k]\!]_2) \cdot t_i$, which corresponds to the portion of randomess that was exchanged with dropout voters in zero_i. These corrections are used to cancel the residual terms introduced by the missing shares:

$$[\![\boldsymbol{v'}]\!]_T = (\sum_{i\in\mathsf{V}} [\![v_i \cdot p^i_j]\!]_T + \sum_{i\in\mathsf{V}} \mathsf{zero}_i \cdot z_j - \sum_{i\in\mathsf{V}} e([\![z_j]\!]_1, \mathsf{ck}_i))_{j\in[n]}.$$

  The final result equals $[\![\boldsymbol{P} \cdot \boldsymbol{v''}]\!]_T$, with $v''_i = v_i$ for all $i \in \mathsf{V}$ and $v''_i = 0$ for all $i \in [n] \setminus \mathsf{V}$.

Our ballot privacy for this construction, under selective and symmetric-key setting[1], holds against an unbounded number of corrupted voters. At a high level, the function-hiding property of FH-IPFE ensures that not only each vote v_i, but also each permutation image $\Pi(i)$, represented by the vector $\boldsymbol{p}^i$, remains private. The decryption computes the inner product of each row j in $\boldsymbol{P}$ with the vote vector $\boldsymbol{v}$, i.e., $\sum_i p^i_j \cdot v_i$. At the same time, each share $\mathsf{zero}_{i,j}$ (or, equivalently, the re-randomised value $\mathsf{zero}_i \cdot z_j$) masks the partial term $p^i_j \cdot v_i$.

The proof-of-concept scheme requires a trusted authority who knows the secret keys of voters, along with the permutation Π which maps input ballots to output votes. We next distribute the trust away from a single authority in two steps: First, we allow each voter to locally generate their functional encryption (FH-IPFE) keys, rather than receiving them from the authority. Secondly, in order to generate a random permutation in a distributed manner, we construct an offline mix-net that processes the voters' functional keys, embedding a permutation in them.

[1] These settings are variants of our ballot privacy definition, given in Section 2.

Local Key Generation for Voter As a step forward in reducing the trust, it is desirable to let each voter generate its FH-IPFE secret key locally. However, in the proof-of-concept scheme, the number of functional keys corresponding to each voter grows with the number of voters, which results in a complexity of $O(n)$, posing scalability challenges if we allow the local key generation. To overcome this, we combine functional encryption with key homomorphism and homomorphic public-key encryption, enabling each voter to delegate functional key generation to the mix process, thus reducing their computational cost to a constant. Specifically, voters generate a secret key, along with three *basis* keys: $(1, 0)$, used to decrypt the vote; $(0, 1)$, used to add a one-time pad; and $(0, 0)$, used to re-randomise the keys. It is important that we introduce a public-key encryption scheme (generated by the authority), so that each voter can encrypt their decryption key $(1, 0)$ before it is used in the matrix to ensure that it cannot be used directly to decrypt the votes. Upon this encryption, the basis keys are then passed to the mix-servers for processing, which we shall describe next. This technique removes the dependence on the number of voters, making this step scalable for a large number of voters.

OMIX: Offline Mix-Net for Multi-Server Setting We begin by constructing a n by n matrix which represents the identity permutation, before each mix-server permutes the rows, hence generating a permutation. In order to construct an offline mix-net such that the verification costs are independent of the number of mix-servers, we avoid general-purpose non-interactive zero-knowledge (NIZK) proofs, instead using linearly-homomorphic signatures (LHS); which sign a vector of elements under a given tag. This provides a malleable proof which ensures the scalable verifiability.

The construction proceeds as follows:

- **Initialisation**: For a set of n voters, the authority parses an initial n by n matrix as the public-key encryption of the key $(1, 0)$ of each voter i in position i, i of the matrix, and a public-key encryption of the key $(0, 0)$ for each element j, i where $j \neq i$. This matrix represents the identity permutation. Then, the authority uses a LHS to generate a tag for each column, and a re-randomisable tag for each row, before signing each element under its respective column and row tag.
- **Shuffling**: The encrypted and signed matrix is then passed to a set of mix-servers, which process the matrix sequentially. Each mix-server does the following two tasks to break any link between the input and output rows of ciphertexts:
  First, they re-randomise the public-key encryption layer of all functional keys on the matrix, and adapt the signatures accordingly. Second, they apply a random permutation to the rows of the matrix and re-randomise the row tags of the signatures.
  Additionally, each mix server provides a proof to ensure that the set of output rows contains all of the input rows up to randomisation, ensuring that no input row is re-randomised in multiple ways and inserted into rows of the output matrix. Along with the unforgeability of the LHS, this proof ensures the correctness of the shuffle, i.e., that input ciphertexts correspond to output ciphertexts up to row permutation and re-randomisation.

This mix-net acts upon the functional decryption keys and can be computed offline some time before the election. However, the existing security notion of LHS is not sufficient to guarantee soundness in the context of a matrix, and hence for us to prove this scheme secure we must first extend the security guarantees provided by LHS.

Enhancing Randomisable-Tag LHS In the context of our mix-net, we are working with multiple signatures of an LHS over both columns and rows, each with a different tag. Additionally, each of the row tags must be re-randomised in order to ensure that the shuffle remains hidden, meaning that after the action of each mix-server, we will have n re-randomised row tags. If we were to use the existing definition of linearly-homomorphic signatures, applied independently to each column and each row, we would not have a guarantee that each re-randomised row-vector contains a linear combination of elements from the same initial tag: it is possible for an adversary to combine elements of different rows which will nevertheless verify as correct. As such, in Section 5.2 we extend the standard definition of unforgeability to a notion we call *tag-binding* unforgeability, which guarantees that all elements defined under a given output row tag come from a single initial row tag. Whilst it is clear that tag-binding unforgeability implies standard unforgeability, we complete this technical contribution by showing the non-trivial direction, proving that any LHS scheme with a property we refer to as *homomorphic verification* (Definition 24) satisfies tag-binding unforgeability.

As a second contribution to LHS, we incorporate two-dimensional tags into the tag-binding unforgeable LHS, such that the row (sub-)tags are randomisable, in order to bind elements of a matrix to its position. The resulting LHS scheme can then be used modularly in OMIX and enables verifiability that is scalable with the number of mix-servers.

OPAD: Distributed randomness embedding A final step is that in our proof-of-concept scheme, each functional key is generated using a fresh randomness. Furthermore, a one-time pad is generated in each functional key. We require multiple authorities to jointly compute another n by n matrix OPAD, where each entry contains a one-time pad multiplied by the $(0, 1)$ basis key, summed with some functional-key randomness multiplied by the $(0, 0)$ basis key. These values are then encrypted under the public key, and then summed with the matrix output by OMIX.

Final construction The end result is a mix-net-based voting scheme that simultaneously achieves self-tallying, corrective fault tolerance, client scalability, universal verifiability, and ballot privacy, all within a distributed authority setting. As this work introduces a novel framework, we conclude with a discussion of open questions and directions for future research in Section 6.

1.3 Related Work

Electronic voting is a dynamic research domain where numerous innovative cryptographic tools have been applied to strike a balance between usability, privacy, verifiability, and efficiency. We highlight significant technical advances that are related to our work.

Ballot-private Voting Electronic voting systems can generally be categorised based on two key tallying methods: homomorphic aggregation such as [2, 4, 20, 32, 45], which is well-suited for elections with a small number of candidates, and mix-nets such as [9, 17, 32, 36, 47, 50, 54], which are more appropriate for general forms of elections involving complex ballots and a large number of voters. In this work, we focus on the mix-net-based single-pass electronic voting framework, where each voter needs to send only one message to cast their vote. This vote will be published on a public bulletin board, and all votes will be decrypted in a permuted form with the help of multiple trustees.

For ballot privacy, the search of security models began with the work of Benaloh [8, 18] and has since evolved into a well-established game-based definition in [11]. This latter security game effectively addresses privacy concerns, such as protection against replay attacks [21], while still allowing important features like verifiability and capturing a variety of result functions.

In this work, we adopt the definition of Benaloh in [8] to capture ballot privacy for our voting protocols, rather than relying on recent definitions in [11, 15, 19]. This choice is motivated by the nature of the self-tallying setting, where decryption is public and thus cannot be simulated. As a result, an admissibility condition on vote queries, which was originally introduced in [8], is necessary. As a contribution, we propose a slight modification to Benaloh's definition to mitigate its known limitation on the class of allowable result functions, as identified in [11].

Self-tallying Elections The concept of self-tallying voting was first introduced by Kiayias and Yung in [38], allowing any party, including voters and casual third parties, to compute the election outcome once all ballots have been cast. This eliminates the need for trusted talliers and ensures that the result, as evaluated over the ballots of honest voters, is always correctly returned.

This line of research has seen various developments aimed at improving efficiency and security [22, 23, 28, 34, 37, 39, 41] and supporting more expressive ballot formats [5, 28, 43, 55, 56]. Most existing works focus on the so-called maximal ballot secrecy setting, where ballot privacy is decentralised among the voters themselves. This typically requires each voter to audit the list of participants with whom they wish to vote, resulting in per-voter costs that grow with the total number of participants. Consequently, such systems are generally suitable only for small-scale elections. The independent and concurrent work [56], which represents the state of the art in decentralised self-tallying voting with homomorphically aggregated results, is unfortunately not an exception to this limitation.

In contrast, our framework enables full vote recovery, while achieving client scalability where the per-voter costs are constant and shifting the computational requirements of all trusted authorities to a setup phase. This framework can also be transformed into a fully decentralised design where each voter act as one of the authorities. However, as explained above, this comes at the unavoidable cost of losing client scalability.

Verifiable and Offline Mix-nets Mix-nets is a highly active field of study, with the primary research focus being the challenge of developing more efficient protocols for deployment in real-world scenarios such as e-voting and anonymous routing. A vast number of works have aimed to reduce the concrete computational and communication overheads, for example [1, 6, 24, 25, 27, 30, 33, 35, 49, 53]. Despite these advancements, the overall cost of mixing remains linear in the number of mixing rounds.

A promising direction focuses on using linearly-homomorphic signatures (LHS) to replace traditional NIZK-based shuffle proofs. For example, [35] introduces a framework that removes the dependency on the number of mix-servers in both proof size and verification time. This approach has been further refined in a recent concurrent work [1], which achieves improved efficiency within the same paradigm.

A separate line of research [3, 4, 44] explores offline mix-nets, where the mixing process is carried out entirely during a pre-computation phase. This design shifts the overhead of sequential mixing away from the tallying phase, enabling highly parallelisable result computation. However, these protocols do not achieve self-tallying, as they critically rely on a set of decryption trustees to produce the final outcome.

Our voting approach combines the advantages of above approaches. It allows mix-servers and decryption trustees to operate solely during the setup phase, while providing a verifiable offline mix-net that scales efficiently with the number of mix-servers.

Functional Encryption for Hiding Permutations Whilst functional encryption for inner products is well established, extending functional keys to support permutations is recent. The work in [48] first introduced this to construct a non-interactive anonymous shuffler (NIAS), which relies on a trusted setup for keys and random-permutation generation. The electronic voting context, however, requires the privacy to be distributed. Our mechanism for distributing the generation of functional keys and random permutations among multiple untrusted authorities can serve as an enhancement to the privacy of NIAS.

2 Voting Definitions

2.1 Voting Syntax

To formally present our approach, we introduce a voting model that supports private-key voting, fault correction, and self-tallying. While the notion of private voting key is present in earlier works, it typically refers to the secret keys associated with eligibility credentials. In our model, the private voting key will additionally contain a private encryption key to ensure ballot privacy[2]. Furthermore, as shown in Proposition 1 of [38], no self-tallying voting scheme can achieve robustness and privacy simultaneously. Therefore, fault tolerance should be realised through corrective actions taken by either honest voters or authorities.

In practice, voting systems usually involve multiple authorities that are responsible for various tasks. We specify the required entities in Section B.1.

We provide a definition for single-vote systems, in which each voter can generate at most one ballot, in the following.

Definition 1 (Voting System). *A voting scheme $\mathcal{V}ot$ for a universe of voters $\mathcal{U}$, a vote space $\mathbb{V}$ and a result function $F : (\mathcal{U} \times \mathbb{V})^* \to \{0,1\}^*$ consists of the following algorithms.*

- $\mathsf{SetUp}(1^\lambda)$ *takes as input a security parameter 1^λ. The list of eligible voters L and the public ballot box BB are initialised to be empty. It outputs a master public/secret key pair $(\mathsf{mpk}, \mathsf{msk})$. The master public key mpk is an implicit argument to other algorithms.*
- $\mathsf{KeyGen}(\mathsf{id}, \mathsf{msk})$ *is used by an authenticated voter $\mathsf{id} \in \mathcal{U}$ to generate a private encryption key $\mathsf{ek}_{\mathsf{id}}$ (possibly with the trustee when msk is needed as input) and a voter public key $\mathsf{pk}_{\mathsf{id}}$. The correspondence between id and $\mathsf{pk}_{\mathsf{id}}$ is public.*
- $\mathsf{Register}(\mathsf{L}, \mathsf{pk}_{\mathsf{id}})$ *updates L with the public key $\mathsf{pk}_{\mathsf{id}}$. Typically, this consists of adding $\mathsf{pk}_{\mathsf{id}}$ as a new entry to L, depending on the validity of $\mathsf{pk}_{\mathsf{id}}$. It also outputs 1 if $\mathsf{pk}_{\mathsf{id}}$ is successfully appended, and 0 otherwise.*

[2] Throughout this work, we implicitly assume the existence of an authentication mechanism for voter eligibility.

- $\mathsf{PKProcess}(\mathsf{L}, \mathsf{msk})$ *takes as input a list of eligible public keys* L *and the master secret key* msk*. It outputs a helper public keys* hpk*, which possibly contain* mpk*.*
- $\mathsf{Vote}(\mathsf{ek}_{\mathsf{id}}, v, \mathsf{hpk})$ *is run by a voter* id *with the private encryption key* $\mathsf{ek}_{\mathsf{id}}$*, a vote* $v \in \mathbb{V}$*, and the helper public key* hpk*. It outputs a ballot* b_{id}*. We assume that the correspondence between* id *and* b_{id} *is public and that* $v = \perp$ *means no vote is sent*[3]*.*
- $\mathsf{Append}(\mathsf{BB}, b_{\mathsf{id}}, \mathsf{L}, \mathsf{hpk})$ *updates* BB *with the ballot* b_{id} *and the list of eligible voters* L*. Typically, this consists in adding* b_{id} *as a new entry to* BB*, depending on the validity of* b_{id} *and the eligibility of* id*. It also outputs* 1 *if* b_{id} *is successfully appended and* 0 *otherwise (e.g., malformed, re-voted, or non-eligible).*
- $\mathsf{FaultAgg}(\mathsf{BB}, \mathsf{hpk}, \mathsf{id})$ *takes as input the ballot box* BB*, the helper public key* hpk *and the identity* id*. It outputs a fault aggregate* $\mathsf{fa}_{\mathsf{id}}$ *that might contain information of dropout voters for fault correction by voter* id*.*
- $\mathsf{FaultCrt}(\mathsf{fa}_{\mathsf{id}}, \mathsf{ek}_{\mathsf{id}})$ *takes as input the fault aggregate* $\mathsf{fa}_{\mathsf{id}}$ *and the private encryption key* $\mathsf{ek}_{\mathsf{id}}$*. It outputs a correcting key* $\mathsf{ck}_{\mathsf{id}}$ *for fault correction.*
- $\mathsf{SelfTally}(\mathsf{BB}, \{\mathsf{ck}_{\mathsf{id}}\}_{\mathsf{id}}, \mathsf{hpk})$ *takes as input the ballot box* BB*, possibly the correcting keys* $\{\mathsf{ck}_{\mathsf{id}}\}_{\mathsf{id}}$*, and the helper public key* hpk*. It outputs a result* res *(possibly* $\mathsf{res} = \perp$*).*
- $\mathsf{Verify}(\mathsf{BB}, \{\mathsf{ck}_{\mathsf{id}}\}_{\mathsf{id}}, \mathsf{res}, \mathsf{L}, \mathsf{hpk})$ *takes as input the ballot box* BB*, the correcting keys* $\{\mathsf{ck}_{\mathsf{id}}\}_{\mathsf{id}}$*, a result* res*, a list* L *of eligible voters and the helper public key* hpk*. It returns* 1 *if valid, and* 0 *otherwise.*

The *offline* phase consists of four algorithms. SetUp is run by the election administrator and trustee to generate system parameters and a master public key, respectively. KeyGen is run by each authenticated voter, optionally with trustee interaction, to generate a private encryption key and submit a corresponding public key to the registrar. $\mathsf{Register}$, run by the registrar, grants voting eligibility and compiles the list of eligible voters. Finally, $\mathsf{PKProcess}$ is run by the trustee on all registered public keys to produce a helper public key, which serves as a reference string and encapsulates the necessary decryption keys for shuffling.

The *online* phase contains six algorithms. Vote is run by each voter to cast their ballot. Append, run by the ballot-box manager, ensures only valid ballots are recorded. $\mathsf{FaultAgg}$ is a publicly computable procedure, run by the ballot-box manager. $\mathsf{FaultCrt}$ is run by each of the remaining voters after the ballot-casting phase to handle faults. $\mathsf{SelfTally}$ enables anyone to compute the election result without secret information, and Verify ensures full public verifiability of protocol operations.

The correctness ensures that when all parties run the voting protocol honestly, the final result correctly reflects the intended votes and all verifications are valid. We provide a formalisation in Definition 20.

To define the functionality for a mix-net-based voting, we define an equivalence relation $\sim_n$ on a space $\mathbb{V}^n$ for any integer n as follows: given any tuples $\boldsymbol{v} = (v_i)_{i\in[n]}, \boldsymbol{v}' = (v'_i)_{i\in[n]} \in \mathbb{V}^n$, we have $\boldsymbol{v} \sim_n \boldsymbol{v}'$ if there exists a permutation Π over $[n]$ such that $\boldsymbol{v}'$ is a tuple resulted by applying Π on $\boldsymbol{v}$, i.e. $\mathsf{PerVec}_{\Pi}(\boldsymbol{v}) = \boldsymbol{v}'$, and vice versa. The equivalence class $[\boldsymbol{v}]_{\sim_n}$ is then defined as the set of all $\boldsymbol{v}' \in \mathbb{V}^n$ such that $\boldsymbol{v} \sim_n \boldsymbol{v}'$.

Definition 2 (Mix-Net-based Voting). *A voting scheme* $\mathcal{V}ot$ *is said to be a mix-net-based voting scheme if its result function* F *is defined as follows: given any* n *(distinct) eligible voters* $\{\mathsf{id}_1, ..., \mathsf{id}_n\}$ *and any tuple of votes* $\boldsymbol{v} = (v_i)_{i\in[n]} \in \mathbb{V}^n$*,* $F((\mathsf{id}_i, v_i)_{i\in[n]})$ *outputs the class* $[\boldsymbol{v}]_{\sim_n}$*.*

In the following, when the context is clear, we will only use $\sim$.

2.2 Corrective Fault Tolerance

We also assume that the votes of any eligible voters whose ballots are expected but missing from the ballot box, due to either non-submission or invalid verification, are treated as if they were not sent and will be represented by null values that can be identified and removed during the tallying process. In what follows, we refer to these voters as *dropout* voters. To handle these voters, we introduce the correcting keys $\{\mathsf{ck}_{\mathsf{id}}\}_{\mathsf{id}}$, which are generated by remaining voters. With possibly the help of $\{\mathsf{ck}_{\mathsf{id}}\}_{\mathsf{id}}$, the final result can still be publicly computed using the $\mathsf{SelfTally}$ algorithm. A formal definition is given below.

[3] The null value $\perp$ is not contained in the valid vote space $\mathbb{V}$.

Definition 3 (Corrective Fault Tolerance). *A voting scheme $\mathcal{V}ot$ is said to adopt corrective fault tolerance if for all parameters λ and all integers n, the following properties hold with an overwhelming probability in λ: in the offline phase, given* $(\mathsf{mpk}, \mathsf{msk})$ *issued by* $\mathsf{SetUp}(1^\lambda)$, $(\mathsf{ek}_i, \mathsf{pk}_i)$ *issued by* $\mathsf{KeyGen}(\mathsf{id}_i, \mathsf{msk}, \mathsf{mpk})$ *for* $i \in [n]$, $\mathsf{L} := (\mathsf{pk}_1, ..., \mathsf{pk}_n)$, hpk *issued by* $\mathsf{PKProcess}(\mathsf{L}, \mathsf{msk})$. *In the online phase, for any strict subset* $\mathsf{V} \subsetneq [n]$, *let* $(v_i)_{i\in\mathsf{V}} \in \mathbb{V}^{|\mathsf{V}|}$ *be votes from voters in* $(\mathsf{id}_i)_{i\in\mathsf{V}}$, *let* b_i *be ballots issued by* $\mathsf{Vote}(\mathsf{ek}_i, v_i, \mathsf{hpk})$ *and let* $\mathsf{BB} := \{b_i\}_{i\in\mathsf{V}}$, *let* ck_i *be correcting keys issued by* $\mathsf{FaultCrt}(\mathsf{FaultAgg}(\mathsf{BB}, \mathsf{hpk}, \mathsf{id}_i), \mathsf{ek}_i)$ *for all* $i \in \mathsf{V}$. *Then the tally is correct and valid, that is, given* $\mathsf{res} = \mathsf{SelfTally}(\mathsf{BB}, \{\mathsf{ck}_i\}_{i\in\mathsf{V}}, \mathsf{hpk})$ *then*

$$\mathsf{res} = F((\mathsf{id}_i, v_i)_{i\in\mathsf{V}}) \text{ and } \mathsf{Verify}(\mathsf{BB}, \{\mathsf{ck}_i\}_{i\in\mathsf{V}}, \mathsf{res}, \mathsf{L}, \mathsf{hpk}) = 1.$$

The above definition is adapted from the intuition in [38], where corrective actions are taken by honest voters to ensure perfect ballot secrecy, that is, privacy is fully decentralised among the voters. We remark, however, that corrective fault tolerance, achieved through *low-overhead* participation by honest voters, remains meaningful even in large-scale voting systems involving authorities, as it enables the final result to be computed without any involvement from trustees. Consequently, trustee participation can be confined and publicly verified during the setup phase alone.

2.3 Client Scalability

In large-scale settings, it is realistic to assume that voters use casual devices. Consequently, both computational and communication costs on the per-voter side should depend only on the security parameters of the underlying cryptographic primitives, and in particular, remain independent of the total number of voters. To capture this requirement, we introduce the notion of *client scalability* as a fundamental criterion towards large-scale voting.

Definition 4 (Client Scalability). *A voting scheme $\mathcal{V}ot$ is said to be* client-scalable *if all algorithms involving the generation and use of a voter's private encryption key* ek_id, *including* $\mathsf{KeyGen}(\mathsf{id}, \mathsf{msk})$, $\mathsf{Vote}(\mathsf{ek}_\mathsf{id}, v, \mathsf{hpk})$, *and* $\mathsf{FaultCrt}(\mathsf{fa}_\mathsf{id}, \mathsf{ek}_\mathsf{id})$, *execute in time* $O_\lambda(1)$ *and incur a communication cost of* $O_\lambda(1)$, *where* $O_\lambda(\cdot)$ *hides dependencies on the security parameter* λ.

To achieve client scalability in fault correction, it is necessary to assume that the ballot box manager behaves as an honest-but-curious authority. This assumption is justified for two reasons. First, the operations executed by the Append and $\mathsf{FaultAgg}$ algorithms are transparent and publicly verifiable via a log. Second, in the absence of an honest ballot box manager or another authority capable of verifying a potentially malicious one, each voter would be required to individually verify and retain this information before generating their correction keys to protect against false accusations. This setting immediately precludes the possibility of client scalability.

2.4 Ballot Privacy

In this work, our definition of ballot privacy is inspired by the one given by Benaloh in [8]. Intuitively, a voting scheme is private if no adversary is able to distinguish between two sets of challenge votes that are tallied into the same result, given the election parameter, the ballot box, and the selection on the voters to be corrupted. As in the voting syntax, we define ballot privacy for single-vote systems, where each voter is expected to submit at most one ballot.

Definition 5 (Ballot Privacy). *Let $\mathcal{V}ot$ be a voting scheme for a universe of voters $\mathcal{U}$, a vote space $\mathbb{V}$ and a result function $F : (\mathcal{U} \times \mathbb{V})^* \to \{0,1\}^*$, we define the privacy experiment* $\mathsf{Exp}^{\mathsf{ExtPRIV}}_{\mathcal{A}}$ *in Figure 4, where the oracles are defined as:*

- $\mathsf{QNewHonest}(\mathsf{id})$*: If* id *is already registered, i.e.* $\mathsf{id} \in \mathsf{L}$, *the query is ignored. Otherwise, on input an identity* $\mathsf{id} \in \mathcal{U}$, *the challenger generates* $(\mathsf{ek}_\mathsf{id}, \mathsf{pk}_\mathsf{id}) \leftarrow \mathsf{KeyGen}(\mathsf{id}, \mathsf{msk})$. *Once the registration is succeeded, it stores the association* $(\mathsf{ek}_\mathsf{id}, \mathsf{pk}_\mathsf{id})$.
- $\mathsf{QCor}(\mathsf{id})$*: The challenger recovers the private encryption key* ek_id *associated to* pk_id *and returns it to the adversary. If no such association is found, the query is ignored.*
- $\mathsf{QReqCast}(\mathsf{id}, \mathsf{pk}_\mathsf{id})$*: If* $\mathsf{id} \notin \mathsf{L}$, *on input a public key* pk_id, *the challenger runs* $\mathsf{Register}(\mathsf{L}, \mathsf{pk}_\mathsf{id})$. *Otherwise, the query is ignored.*

– $\mathsf{QPKProcess}()$: *This oracle is used for only one time in the game, the challenger outputs* $\mathsf{hpk} \leftarrow \mathsf{PKProcess}(\mathsf{L}, \mathsf{msk})$ *to the adversary. If* $\mathsf{PKProcess}(\mathsf{L}, \mathsf{msk})$ *is a protocol between the adversary (as corrupted trustees) and the challenger (as honest trustees), then a query asks the challenger to use* msk *to join this protocol. Once a query is sent, no* QNewHonest *and* QReqCast *queries are allowed after.*

The following oracles can be queried after a query is sent to $\mathsf{QPKProcess}()$.

– $\mathsf{QVote}_\beta(\mathsf{id}, v^0, v^1)$: *For a voter* id, *if* $v^0, v^1 \in \mathbb{V}$ *and the private encryption key* $\mathsf{ek}_{\mathsf{id}}$ *can be recovered, the challenger generates* $b_{\mathsf{id}} \leftarrow \mathsf{Vote}(\mathsf{ek}_{\mathsf{id}}, v^\beta, \mathsf{hpk})$. *Once the appending to ballot-box is succeeded, the query is recorded. Else, the query is ignored.*
– $\mathsf{QVotCast}(\mathsf{id}, b_{\mathsf{id}})$: *If* id *is corrupted via* QCor *or registered via* QReqCast[4], *this oracle allows the adversary to cast a ballot* b_{id} *on behalf of voter* id; *the challenger then runs* $\mathsf{Append}(\mathsf{BB}, b_{\mathsf{id}}, \mathsf{L}, \mathsf{hpk})$. *Otherwise, the query is ignored.*
– $\mathsf{QFaultCrt}()$: *This oracle is used for only one time in the game, the challenger returns the correcting keys* $\mathsf{ck}_{\mathsf{id}} \leftarrow \mathsf{FaultCrt}(\mathsf{FaultAgg}(\mathsf{BB}, \mathsf{hpk}, \mathsf{id}), \mathsf{ek}_{\mathsf{id}})$ *for all* id *such that* $(\mathsf{ek}_{\mathsf{id}}, \mathsf{pk}_{\mathsf{id}})$ *is stored. No other oracle queries are allowed after.*

$\underline{\mathsf{Exp}^{\mathsf{ExtPRIV}}_{\mathcal{A}}(\lambda, \mathcal{U}, F)}$:
$\beta \xleftarrow{\$} \{0,1\}$, $(\mathsf{mpk}, \mathsf{msk}) \leftarrow \mathsf{SetUp}(1^\lambda)$, $\mathsf{BB}, \mathsf{L} \leftarrow \emptyset$;
$\beta' \leftarrow \mathcal{A}^{\mathsf{QNewHonest}(\cdot),\mathsf{QCor}(\cdot),\mathsf{QReqCast}(\cdot,\cdot),\mathsf{QPKProcess}(),\mathsf{QVote}_\beta(\cdot,\cdot,\cdot),\mathsf{QVotCast}(\cdot,\cdot),\mathsf{QFaultCrt}()}(\mathsf{mpk}, \mathsf{BB}, \mathsf{L})$;
output $\alpha = \beta'$ if condition $(*)$ is satisfied
and $\alpha \xleftarrow{\$} \{0,1\}$ otherwise.

Fig. 4. Experiment for ballot privacy

At the time the QFaultCrt *query is made, let* L *be the list of eligible voters,* $\mathcal{HS}$ *be the subset of honest voters (whose public keys are generated via* QNewHonest *but not corrupted via* QCor*) that are queried and recorded in* QVote

Condition $(*)$, *which prevents trivial wins, holds if : there does not exist any tuple of votes* $(\mathsf{id}, v_{\mathsf{id}})_{\mathsf{id} \in \mathsf{L} \setminus \mathcal{HS}}$ *such that*

$$F((\mathsf{id}, v^0_{\mathsf{id}})_{\mathsf{id} \in \mathcal{HS}}, (\mathsf{id}, v_{\mathsf{id}})_{\mathsf{id} \in \mathsf{L} \setminus \mathcal{HS}}) \neq F((\mathsf{id}, v^1_{\mathsf{id}})_{\mathsf{id} \in \mathcal{HS}}, (\mathsf{id}, v_{\mathsf{id}})_{\mathsf{id} \in \mathsf{L} \setminus \mathcal{HS}}),$$

where for each $\mathsf{id} \in \mathcal{HS}$, *the query* $\mathsf{QVote}(\mathsf{id}, v^0_{\mathsf{id}}, v^1_{\mathsf{id}})$ *was recorded*[5].

The scheme $\mathcal{V}ot$ *is said to have ballot privacy if for any PPT adversary* $\mathcal{A}$, *the advantage of correctly guessing the challenge bit* β, *namely*

$$\left| \Pr[\mathsf{Exp}^{\mathsf{ExtPRIV}}_{\mathcal{A}}(\lambda, \mathcal{U}, F) = \beta] - \frac{1}{2} \right|$$

is a negligible function of λ.

Remark 1 (Weaker Variants). The scheme $\mathcal{V}ot$ might have ballot privacy in a weaker setting if the following additional conditions are added to Condition $(*)$:

– *Symmetric-key* privacy: Let $\mathcal{CS}$ be the set of voters that are corrupted via QCor at the end of the game. If $\mathsf{id} \in \mathcal{CS}$, then for any queries $\mathsf{QVote}(\mathsf{id}, v^0, v^1)$, it must hold that $v^0 = v^1$.
– *Selective* privacy: Once receiving mpk, the adversary is forced to send all QNewHonest queries, upon which it receives the corresponding public keys of honest voters. Then it commits all QCor and QVote_β queries in one shot.

[4] In Definition 1, we assume the public correspondence between an identity id and its ballot b_{id}, therefore the cases an adversary casts ballots on behalf of honest voters can be ruled out.
[5] The adversary can access only a unique ballot for each voter in $\mathcal{HS}$ via BB, due to the no-revote policy.

In comparison to Benaloh's definition, we modify the restriction on adversaries so that the result function, when evaluated on challenge votes of honest users and *any possible* non-challenge votes, must produce an output independent of the challenge bit. Although there are result functions for which the challenger may not be able to verify this condition in polynomial time, common functions such as homomorphic tallying and mix-net based tallying are not affected by this limitation. Furthermore, under this modified restriction, a voting scheme for the $\mathsf{majority}$ function might be considered ballot-private, unlike in Benaloh's definition, as noted in [11]. On the other hand, the simulated tallying proofs that are enabled in the definition in [11], are non-applicable in settings where tallying can be executed publicly.

The remark on symmetric-key privacy means that each private encryption key $\mathsf{ek}_{\mathsf{id}}$ might have the power of decrypting the ciphertext generated by the voter id. Therefore, if an adversary corrupts a honest voter who has already cast its vote, the adversary can learn the encrypted vote through the use of its private encryption key, and win the privacy game. In practice, symmetric-key privacy requires that the encryption key must be stored securely.

2.5 Verifiability

In a voting system, verifiability ensures that a voter can verify, even in the presence of dishonest authorities, that their ballot has been correctly included in the final tally (individual verifiability), that the result reflects the intended votes of voters as published on the ballot box (universal verifiability), and that only ballots from eligible voters are counted (eligibility verifiability).

In this work, eligibility verifiability is ensured through implicit credentials issued by the registrar and a publicly available list of eligible voters. Individual verifiability is provided by the public nature of the ballot box, allowing each voter to confirm the presence of their own ballot. We provide a formalisation of universal verifiability in Definition 21, which captures vote integrity of honest voters in the final results.

For completeness of voting definitions, we discuss the notions of fairness and dispute-freeness in Section B.2.

3 Voting with Functional Encryption

3.1 Functional Encryption

The definition of functional encryption, which is introduced in [14, 42, 46] and specified for function-hiding inner products in [12, 51], is given below.

Definition 6 (FH-IPFE). *Given a group $\mathcal{PG} = (\lambda, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, p)$, a functional encryption for a family of hiding inner-product functions $\{\langle\cdot,\cdot\rangle_\rho : \mathbb{G}_1^\rho \times \mathbb{G}_2^\rho \to \mathbb{G}_T\}_\rho$ consists of four algorithms:*

- $\mathsf{SetUp}(1^\lambda, \rho)$*: On input a security parameter 1^λ and a dimension ρ, it outputs public parameters* pp *and a secret key* sk*. The parameters* pp *is an implicit argument to other algorithms.*
- $\mathsf{Enc}(\mathsf{sk}, [\![\boldsymbol{m}]\!]_2)$*: On input a secret key* sk *and a message value $[\![\boldsymbol{m}]\!]_2 \in \mathbb{G}_2^\rho$ to encrypt, it outputs a ciphertext* ct*.*
- $\mathsf{DKGen}(\mathsf{sk}, [\![\boldsymbol{k}]\!]_1)$*: On input a secret key* sk *and a key value $[\![\boldsymbol{k}]\!]_1 \in \mathbb{G}_1^\rho$, it outputs a decryption key* dk*.*
- $\mathsf{Dec}(\mathsf{dk}, \mathsf{ct})$*: On input a decryption key* dk *and a ciphertext* ct*, it outputs an element in $\mathbb{G}_T$.*

Correctness For all parameters $\lambda, \rho \in \mathbb{N}$, for any secret key sk generated by $\mathsf{SetUp}(1^\lambda, \rho)$, for any message vector $\boldsymbol{m} \in \mathbb{Z}_p^\rho$ and key vector $\boldsymbol{k} \in \mathbb{Z}_p^\rho$, if $\mathsf{ct} = \mathsf{Enc}(\mathsf{ek}, \boldsymbol{m})$ and $\mathsf{dk} = \mathsf{DKGen}(\mathsf{sk}, \boldsymbol{k})$, then the decryption satisfies $\mathsf{Dec}(\mathsf{dk}_{\boldsymbol{k}}, \mathsf{ct}_{\boldsymbol{m}}) = [\![\langle \boldsymbol{m}, \boldsymbol{k} \rangle]\!]_T$ with overwhelming probability, where the probability is taken over the randomness of all algorithms.

Definition 7 (Function-Hiding Privacy). *Let* FE *be a functional encryption scheme for inner products of dimension ρ, we define the experiment* $\mathrm{IND}^{\mathsf{FE}}$ *in Figure 5, where the oracles are defined as:*

- ***Encryption oracle*** $\mathsf{QEnc}_b([\![\boldsymbol{m}^0]\!]_2, [\![\boldsymbol{m}^1]\!]_2)$*: it returns the ciphertext* $\mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{sk}, [\![\boldsymbol{m}^b]\!]_2)$*.*
- ***Decryption-key oracle*** $\mathsf{QDKGen}_b([\![\boldsymbol{k}^0]\!]_1, [\![\boldsymbol{k}^1]\!]_1)$*: it returns the decryption key* $\mathsf{dk} \leftarrow \mathsf{DKGen}(\mathsf{sk}, [\![\boldsymbol{k}^b]\!]_1)$*.*

$\underline{\mathrm{IND}^{\mathsf{FE}}(\lambda, \mathcal{A})}$:
$b \xleftarrow{\$} \{0,1\}$, $(\mathsf{pp}, \mathsf{sk}) \leftarrow \mathsf{SetUp}(1^\lambda, \rho)$
$\alpha \leftarrow \mathcal{A}^{\mathsf{QEnc}_b(\cdot,\cdot), \mathsf{QDKGen}_b(\cdot,\cdot)}(\mathsf{pp})$
Output: $\beta = \alpha$ if Condition $(*)$ is satisfied, $\beta \xleftarrow{\$} \{0,1\}$ otherwise.

Fig. 5. Security experiment for FE

Condition $(*)$ *holds if the following condition hold:*

- *There do not exist an encryption query on* $([\![\boldsymbol{m}^0]\!]_2, [\![\boldsymbol{m}^1]\!]_2)$ *and a decryption-key query on* $([\![\boldsymbol{k}^0]\!]_1, [\![\boldsymbol{k}^1]\!]_1)$*, such that* $[\![\langle \boldsymbol{m}^0, \boldsymbol{k}^0 \rangle]\!]_T \neq [\![\langle \boldsymbol{m}^1, \boldsymbol{k}^1 \rangle]\!]_T$.

The FE *scheme is said to be IND-CPA secure if given any parameters* $\lambda, \rho \in \mathbb{N}$*, for every PPT adversary* $\mathcal{A}$*, the following advantage is negligible in* λ

$$\mathsf{Adv}^{\mathrm{IND}^{\mathsf{FE}}}(\mathcal{A}) := \left| \Pr[\beta = 1] - \frac{1}{2} \right|.$$

This work also necessitates the following property of FH-IPFE, which is satisfied by many existing schemes in the literature, for example [40, 52].

Definition 8 (Homomorphic FE Key). *An* FE *scheme supports homomorphic functional keys if, given any two keys* $\mathsf{dk}_1 = \mathsf{DKGen}(\mathsf{sk}, [\![\boldsymbol{k}_1]\!]_1; \boldsymbol{r}_1)$ *and* $\mathsf{dk}_2 = \mathsf{DKGen}(\mathsf{sk}, [\![\boldsymbol{k}_2]\!]_1; \boldsymbol{r}_2)$ *(in the form of vectors in* $\mathbb{G}$*) under randomness* $\boldsymbol{r}_1$ *and* $\boldsymbol{r}_2$ *(in the form of vectors in* $\mathbb{Z}_p$*) respectively, for any scalars* $\alpha_1, \alpha_2 \in \mathbb{Z}_p$*, the following holds:*

$$\alpha_1 \cdot \mathsf{dk}_1 + \alpha_2 \cdot \mathsf{dk}_2 = \mathsf{DKGen}(\mathsf{sk}, [\![\alpha_1 \cdot \boldsymbol{k}_1 + \alpha_2 \cdot \boldsymbol{k}_2]\!]_1; \alpha_1 \cdot \boldsymbol{r}_1 + \alpha_2 \cdot \boldsymbol{r}_2).$$

3.2 A Proof-of-Concept Protocol

By leveraging functional encryption for function-hiding inner products, we present a mix-net-based voting protocol that incorporates the generation of a permutation and its corresponding decryption key in the offline phase. For simplicity, our conceptual protocol assumes an authority that serves as both election administrators and trustees. We also assume secure, authenticated private channels between the authority and each voter for generating private encryption keys.

In this conceptual protocol, we do not aim for universal verifiability. This simplification allows us to focus on the role of functional encryption in ensuring ballot privacy while enabling the generation of decryption key for shuffling to occur in the offline phase, thereby removing the need of any trusted authority in the tallying phase. Furthermore, the ballot privacy proof for this protocol serves as a foundation for our more advanced construction which incorporates universal verifiability and an offline mix-net with multiple servers.

Assuming an FH-IPFE scheme with $(\mathsf{ip.Setup}, \mathsf{ip.DKGen}, \mathsf{ip.Enc}, \mathsf{ip.Dec})$ algorithms in a prime-order and asymmetric pairing group $\mathcal{PG} = (\lambda, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, e, p)$, where the message vectors are elements in $\mathbb{G}_2$, the key vectors are elements in $\mathbb{G}_1$, and the decrypted inner product are elements in $\mathbb{G}_T$, we present our proof-of-concept protocol for the mixing result function (see Definition 2) in Figure 6. The vote space is defined as $\mathbb{V} = \{1, \ldots, V\} \subset \mathbb{Z}_p$ for some λ-polynomially large integer V. As the universal verifiability will not be considered in this version, we omit the Verify algorithm. The intuition on how an election will proceed is given below.

The authority computes the offline phase itself: it begins by generating an FE key $\mathsf{ip.sk}_{\mathsf{id}}$ and a zero-share key t_{id} for each voter, sending these keys privately to the voters, and registering voters in a list L.

Next, they run the PKProcess algorithm, in which a permutation Π is generated in order to map each voters ballot from i to $\Pi(i)$ during decryption. The authority generates functional decryption keys using this permutation Π, along with a one-time-pad vector $(z_j)_{j \in [n]}$ to re-randomise the zero shares that mask the leakage from partial FH-IPFE decryptions between functional keys of each row j and ciphertexts. To gain constant costs for each voter i, the authority aggregates the public zero-share keys of other ones into pz_i. The component pz_i is public and will later be used for generating a zero share within the ciphertext.

It is in the $\mathsf{PKProcess}$ algorithm that we shall later incorporate an offline mix-net to generate the permutation in a distributed manner, and to distribute the generation of the one-time-pad vectors. As such, we include the extension components aux_i to ensure this scheme is compatible with the later enhancements.

The voting phase consists of each voter using their received secret keys to encrypt a ballot and cast it to the bulletin board. If the voter is registered in list L, and this is the first vote they are casting, its ballot is appended to the box.

After the voting phase has concluded, if all voters successfully cast their ballots into the box, the tallying phase is processed publicly via decryption between the ciphertexts that encrypt votes and the functional keys that encapsulate the permutation.

If some registered voters drop out after the offline phase and are aggregated through the $\mathsf{FaultAgg}$ algorithm, the protocol can still produce a correctly shuffled tally of the remaining votes without requiring interaction from any trusted authority. This is enabled by an agreement of the remaining honest voters, who collaboratively reveal their pseudorandom values that were computed pairwise with the dropout voters, in the form of a correcting key. This revealing process is executed locally by running the $\mathsf{FaultCrt}$ algorithm.

Correctness. Using the contemporary notation from the voting protocol, one can parse columns $(\boldsymbol{p}^i)_{i\in[n]}$ as the identity matrix with rows permuted by Π, or $\mathsf{PerMatRow}_\Pi(\boldsymbol{I})$. Assuming all eligible voters send their ballots, we have

$$\begin{aligned}
[\![\boldsymbol{v}']\!]_T &= (\sum_{i\in[n]} \mathsf{ip.Dec}(\mathsf{idk}_j^i, \mathsf{ict}_i))_{j\in[n]} = (\sum_{i\in[n]} [\![v_i \cdot p_j^i + z_j \sum_{k\in[n]\setminus\{i\}} (-1)^{k<i} t_k \cdot t_i]\!]_T)_{j\in[n]} \\
&= ([\![\sum_{i\in[n]} v_i \cdot p_j^i + \sum_{i,k\in[n],i\neq k} \left((-1)^{k<i} + (-1)^{i<k}\right) \cdot t_k t_i]\!]_T)_{j\in[n]} \\
&= ([\![\sum_{i\in[n]} v_i \cdot p_j^i]\!]_T)_{j\in[n]} = [\![\mathsf{PerMatRow}_\Pi(\boldsymbol{I}) \cdot \boldsymbol{v}]\!]_T = [\![\mathsf{PerVec}_\Pi(\boldsymbol{v})]\!]_T
\end{aligned}$$

As $\boldsymbol{v} \in \mathbb{V}$ and $[\![\boldsymbol{v}']\!]_T$ can be recovered by the computation of discrete logarithms, we have $\boldsymbol{v}^\star = \mathsf{PerVec}_\Pi(\boldsymbol{v})$. Therefore, $[\boldsymbol{v}^\star]_{\sim_n} = [\boldsymbol{v}]_{\sim_n}$.

Corrective Fault Tolerance. Using the contemporary notation from the voting protocol, one can parse columns $(\boldsymbol{p}^i)_{i\in[n]}$ as the identity matrix with rows permuted by Π, or $\mathsf{PerMatRow}_\Pi(\boldsymbol{I})$. Assuming voters in $\mathsf{V} \subset [n]$ send their ballots and correcting keys, we have

$$\begin{aligned}
[\![\boldsymbol{v}']\!]_T &= (\sum_{i\in\mathsf{V}} \mathsf{ip.Dec}(\mathsf{idk}_j^i, \mathsf{ict}_i) - \sum_{i\in\mathsf{V}} e([\![z_j]\!]_1, \mathsf{ck}_i))_{j\in[n]} \\
&= (\sum_{i\in\mathsf{V}} [\![v_i \cdot p_j^i + z_j \sum_{k\in[n]\setminus\{i\}} (-1)^{k<i} t_k \cdot t_i - z_j \sum_{k\in[n]\setminus\mathsf{V}} (-1)^{k<i} t_k \cdot t_i]\!]_T)_{j\in[n]} \\
&= (\sum_{i\in\mathsf{V}} [\![v_i \cdot p_j^i + z_j \sum_{k\in\mathsf{V}\setminus\{i\}} (-1)^{k<i} t_k \cdot t_i]\!]_T)_{j\in[n]} \\
&= ([\![\sum_{i\in\mathsf{V}} v_i \cdot p_j^i + \sum_{i,k\in\mathsf{V},i\neq k} \left((-1)^{k<i} + (-1)^{i<k}\right) \cdot t_k t_i]\!]_T)_{j\in[n]} \\
&= ([\![\sum_{i\in\mathsf{V}} v_i \cdot p_j^i]\!]_T)_{j\in[n]}
\end{aligned}$$

Then we have $v'_j = v_i$ if $j = \Pi(i)$ for $i \in \mathsf{V}$, and $v'_j = 0$ if $j = \Pi(i)$ for $i \notin \mathsf{V}$. When zero entries of $\boldsymbol{v}'$ are removed, we have the $\mathsf{SelfTally}$ output $[\boldsymbol{v}^\star]_{\sim_{|\mathsf{V}|}} = [\boldsymbol{v}']_{\sim_{|\mathsf{V}|}}$.

Client Scalability. The protocol guarantees that both the per-voter communication and computation costs remain constant with respect to the total number of voters. Specifically, since the number of operations in each FH-IPFE algorithm is $O_\lambda(1)$, we have:

- In $\mathsf{KeyGen}(\mathsf{id}, \mathsf{msk}, \mathsf{mpk})$, the private encryption key $\mathsf{ek}_\mathsf{id} = (\mathsf{ip.sk}_\mathsf{id}, t_\mathsf{id})$ and the public key $\mathsf{pk}_\mathsf{id} = [\![t_\mathsf{id}]\!]_2$ are of size $O_\lambda(1)$. The generation of pk_id requires one exponentiation in $\mathbb{G}_2$.

$\underline{\mathsf{SetUp}(1^\lambda)}$: The authority generates and stores random $t_{\mathsf{id}} \xleftarrow{\$} \mathbb{Z}_p$ and private encryption keys $\mathsf{ek}_{\mathsf{id}} = \mathsf{ip.sk}_{\mathsf{id}} \leftarrow \mathsf{ip.Setup}(1^\lambda, \rho(\lambda)+2)$ for $\mathsf{id} \in \mathcal{U}$ in msk. It outputs mpk as the public parameter for FH-IPFE (including a pairing group $\mathcal{PG}$) and the vote space $\mathbb{V} = \{1, \ldots, V\}$. Concretely, $\rho(\lambda) = 1$ for security proof in this work.

$\underline{\mathsf{KeyGen}(\mathsf{id}, \mathsf{msk}, \mathsf{mpk})}$: Via a private and authenticated channel, a voter id receives $\mathsf{ek}_{\mathsf{id}} = (\mathsf{ip.sk}_{\mathsf{id}}, t_{\mathsf{id}})$ from the authority, $\mathsf{pk}_{\mathsf{id}}$ contains $[\![t_{\mathsf{id}}]\!]_2$.

$\underline{\mathsf{Register}(\mathsf{L}, \mathsf{pk}_{\mathsf{id}})}$: The authority adds $\mathsf{pk}_{\mathsf{id}}$ to the list of eligible voters L.

$\underline{\mathsf{PKProcess}(\mathsf{L}, \mathsf{msk})}$: For n voters in $\mathsf{L} = \{\mathsf{id}_i\}_{i\in[n]}$, the authority generates a random permutation $\Pi : [n] \to [n]$, a one-time-pad vector $\boldsymbol{z} = (z_j)_{j\in[n]} \xleftarrow{\$} \mathbb{Z}_p^n$, and for each $i \in [n]$:

1. an aggregate public key for zero sharing: $\mathsf{pz}_i = \sum_{k\in[n]\setminus\{i\}} (-1)^{k<i} \cdot [\![t_k]\!]_2$
2. a functional key for each $j \in [n]$: $\mathsf{ip.dk}^i_j \leftarrow \mathsf{ip.DKGen}(\mathsf{ip.sk}_i, [\![p^i_j, z_j, \mathbf{0}^{\rho(\lambda)}]\!]_1)$ where $\boldsymbol{p}^i$ is a vector that represents $\Pi(i)$ such that $p^i_j = 1$ if $j = \Pi(i)$ and $p^i_j = 0$ otherwise;
3. components for extension purposes[a] $\mathsf{aux}_i = (\mathsf{ip.dk}^i_{(0,1)}, \mathsf{ip.dk}^i_{(0,0)})$ where

$$\mathsf{ip.dk}^i_{(0,1)} = \mathsf{ip.DKGen}(\mathsf{ip.sk}_i, [\![0, 1, \mathbf{0}^{\rho(\lambda)}]\!]_1)$$
$$\mathsf{ip.dk}^i_{(0,0)} = \mathsf{ip.DKGen}(\mathsf{ip.sk}_i, [\![0, 0, \mathbf{0}^{\rho(\lambda)}]\!]_1).$$

It publishes $\mathsf{hpk} = \mathsf{mpk} || \{\mathsf{id}_i, \mathsf{pk}_i, (\mathsf{ip.dk}^i_j)_{j\in[n]}, \mathsf{pz}_i, \mathsf{aux}_i\}_{i\in[n]} || [\![\boldsymbol{z}]\!]_1$.

$\underline{\mathsf{Vote}(\mathsf{ek}_{\mathsf{id}}, v_{\mathsf{id}}, \mathsf{hpk})}$: For $v_{\mathsf{id}} \in \mathbb{V}$, voter id computes an FH-IPFE encryption:

$$\mathsf{ip.ct}^{\mathsf{id}} \leftarrow \mathsf{ip.Enc}(\mathsf{ip.sk}_{\mathsf{id}}, ([\![v_{\mathsf{id}}]\!]_2, \mathsf{pz}_{\mathsf{id}} \cdot t_{\mathsf{id}}, [\![\mathbf{0}^{\rho(\lambda)}]\!]_2));$$

and casts $b_{\mathsf{id}} = \mathsf{ip.ct}^{\mathsf{id}}$ to BB. Registered voter id can abort voting.

$\underline{\mathsf{Append}(\mathsf{BB}, b_{\mathsf{id}}, \mathsf{L}, \mathsf{hpk})}$: If $\mathsf{id} \in \mathsf{L}$ and there does not exist $b'_{\mathsf{id}} \in \mathsf{BB}$, the authority appends b_{id} to BB.

$\underline{\mathsf{FaultAgg}(\mathsf{BB}, \mathsf{hpk}, \mathsf{id})}$: Let $\mathsf{V} \subseteq [n]$ be the list of each $\mathsf{id}_i \in \mathsf{L}$ whose ballot is contained in BB. If $\mathsf{V} = [n]$, nothing is output. Otherwise, the fault aggregate for the voter $i \in \mathsf{V}$ that corresponds to id is $\mathsf{fa}_i = \sum_{k\in[n]\setminus\mathsf{V}} (-1)^{k<i} [\![t_k]\!]_2$.

$\underline{\mathsf{FaultCrt}(\mathsf{fa}_{\mathsf{id}}, \mathsf{ek}_{\mathsf{id}})}$: The correcting key is output as $\mathsf{ck}_{\mathsf{id}} = \mathsf{fa}_{\mathsf{id}} \cdot t_{\mathsf{id}}$.

$\underline{\mathsf{SelfTally}(\mathsf{BB}, \{\mathsf{ck}_{\mathsf{id}}\}_{\mathsf{id}}, \mathsf{hpk})}$: Let $\mathsf{V} \subseteq [n]$ be the list of each $\mathsf{id}_i \in \mathsf{L}$ whose ballot is contained in BB and ck_i is given (in case $\mathsf{V} \subsetneq [n]$)[b] and let $\boldsymbol{v}^\star$ be an empty tuple. For each $j \in [n]$, it recovers v'_j from $[\![v'_j]\!]_T = \sum_{i\in\mathsf{V}} \mathsf{ip.Dec}(\mathsf{ip.dk}^i_j, \mathsf{ip.ct}^i) - e(\sum_{i\in\mathsf{V}} \mathsf{ck}_i, [\![z_j]\!]_1)$ and appends to $\boldsymbol{v}^\star$ if $[\![v'_j]\!]_T \neq [\![0]\!]_T$[c]. The result res is the equivalence class $[\boldsymbol{v}^\star]_{\sim_{|\mathsf{V}|}}$.

[a] These components are not yet needed in this protocol.
[b] If such a list does not exist, the algorithm outputs $\perp$ as the result.
[c] $v'_j = 0$ corresponds to a null vote $\perp$ in Definition 1.

Fig. 6. A simplified voting protocol with FH-IPFE

- In $\mathsf{Vote}(\mathsf{ek}_{\mathsf{id}}, v_{\mathsf{id}}, \mathsf{hpk})$, the ballot b_{id} has size $O_\lambda(1)$, and the computation cost is that of a single FH-IPFE encryption.
- In $\mathsf{FaultCrt}(\mathsf{fa}_{\mathsf{id}}, \mathsf{ek}_{\mathsf{id}})$, the correcting key $\mathsf{ck}_{\mathsf{id}}$ is a single group element in $\mathbb{G}_2$ and is computed with one exponentiation.

Hence, the protocol satisfies client scalability as defined in Definition 4.

3.3 Ballot Privacy

For the security analysis of the construction, we need the following remark.

Remark 2. Condition $(*)$ in ballot privacy (in Definition 5) when applied to mix-net-based voting (Definition 2) implies that the equivalence class represented by $(v^0_{\mathsf{id}})_{\mathsf{id}\in\mathcal{HS}}$ is equal to that represented by $(v^1_{\mathsf{id}})_{\mathsf{id}\in\mathcal{HS}}$, where $(\mathsf{id}, v^0_{\mathsf{id}}, v^1_{\mathsf{id}})_{\mathsf{id}\in\mathcal{HS}}$ are the recorded vote queries on a subset $\mathcal{HS}$ of honest voters. In other words, there must exist a bijection Π over $\mathcal{HS}$ such that $v^0_{\Pi(\mathsf{id})} = v^1_{\mathsf{id}}$.

Theorem 1. *If FH-IPFE is IND-CPA secure and the SXDH assumption holds in $\mathcal{PG}$, then the proof-of-concept scheme for mix-net voting is symmetric-key and selectively secure, as per Definition 5.*

Proof. At the end of the game, let L be the list of eligible voters, $\mathcal{CS}$ be the set of voters that are corrupted via QCor, and $\mathcal{HS}$ be the subset of honest voters (whose public keys are generated via $\mathsf{QNewHon}$ but not corrupted via QCor) that are queried and recorded in QVote_β. In the selective setting, $\mathcal{HS}, \mathcal{CS}$, and $\{(\mathsf{id}, v^0, v^1)\}_{\mathsf{id}\in\mathcal{HS}}$ are specified beforehand from the $\mathsf{QNewHon}$, QCor and QVote_β queries. In the symmetric-key setting, any (id, v^0, v^1) vote queries for $\mathsf{id} \in \mathcal{CS}$ must satisfy $v^0 = v^1$.

To align with permutations, we assume $n = |\mathsf{L}|$ and index the voters in L using integers from $[n]$. From this point forward, we denote $\mathcal{HI}$ as the set of indices in $[n]$ corresponding to the voters in $\mathcal{HS}$. The indice set $[n] \setminus \mathcal{HI}$ thus corresponds to the voters in $\mathsf{L} \setminus \mathcal{HS}$.

It suffices to prove that it is hard for $\mathcal{A}$ to distinguish between the real game where QVote_β queries are answered based on β and the ideal game where QVote_β queries are answered based on a fixed bit, for example 0. Then we proceed via a hybrid argument of games where $\mathbf{G}_0$ corresponds to the real security game, and $\mathbf{G}_5$ corresponds to the ideal one. We highlight the change between each two adjacent games in a gray box.

Game $\mathbf{G}_1$: If the vote queries $(i, v^0_i, v^1_i)_{i\in\mathcal{HI}}$ satisfy Condition $(*)$ in the security game, then by Remark 2 there exists a bijection Π^β over $\mathcal{HI}$ such that $v^\beta_i = v^0_{\Pi^\beta(i)}$. This bijection can be extended to a permutation over $[n]$ by maintaining $\Pi^\beta(i) = i$ for $i \in [n] \setminus \mathcal{HI}$. The challenger generates a random $\Pi \circ \Pi^\beta$ instead of a random Π in the $\mathsf{QPKProcess}$ oracle. The indistinguishability between $\mathbf{G}_0$ and $\mathbf{G}_1$ is thus perfect.

We have the following remark on the relation between two permutations $(\Pi \circ \Pi^\beta, \Pi)$ and the vote queries $(i, v^0_i, v^1_i)_{i\in\mathcal{HI}}$

Remark 3. As $\Pi \circ \Pi^\beta(i) = \Pi(i)$ for $i \in [n] \setminus \mathcal{HI}$, these two permutations then have their representing column vectors $(\boldsymbol{p}^{\beta,i})_{i\in[n]}$ and $(\boldsymbol{p}^{0,i})_{i\in[n]}$ such that $\boldsymbol{p}^{\beta,i} = \boldsymbol{p}^{0,i}$ for $i \in [n]\setminus\mathcal{HI}$. Given any $(v^\beta_i)_{i\in[n]\setminus\mathcal{HI}} = (v^0_i)_{i\in[n]\setminus\mathcal{HI}}$, then permuting vector $\boldsymbol{v}^\beta = (v^\beta_i)_{i\in[n]}$ by $\Pi \circ \Pi^\beta$ and permuting vector $\boldsymbol{v}^0 = (v^0_i)_{i\in[n]}$ by Π yield the same result, as we have

$$\mathsf{PerVec}_\Pi(\mathsf{PerVec}_{\Pi^\beta}(\boldsymbol{v}^\beta)) = \mathsf{PerVec}_\Pi(\boldsymbol{v}^0).$$

This implies $\sum_{i\in[n]} p^{\beta,i}_j \cdot v^\beta_i = \sum_{i\in[n]} p^{0,i}_j \cdot v^0_i$ for $j \in [n]$. The terms $p^{\beta,i}_j \cdot v^\beta_i = p^{0,i}_j \cdot v^0_i$ can be eliminated for all $i \in [n] \setminus \mathcal{HI}$, implying

$$\sum_{i\in\mathcal{HI}} p^{\beta,i}_j \cdot v^\beta_i = \sum_{i\in\mathcal{HI}} p^{0,i}_j \cdot v^0_i \ \forall j \in [n]. \tag{1}$$

From the above remark, by relying on FH-IPFE and zero shares, we continue to prove that the game $\mathbf{G}_1$ in which the challenger uses $(v^\beta_i)_{i\in\mathcal{HI}}$ and $(\boldsymbol{p}^{\beta,i})_{i\in\mathcal{HI}}$ and the game $\mathbf{G}_5$ in which the challenger uses $(v^0_i)_{i\in\mathcal{HI}}$ and $(\boldsymbol{p}^{0,i})_{i\in\mathcal{HI}}$ to respond to QVote_β and $\mathsf{QPKProcess}$ queries respectively are indistinguishable.

We note that in the following games, the auxiliary key $\mathsf{ip.dk}^i_{(0,0)}$ does not help the adversary in learning the challenge bit. This is because when $\mathsf{ip.dk}^i_{(0,0)}$ is used to decrypt any honest ciphertext, the output is $[\![0]\!]_T$. For the auxiliary key $\mathsf{ip.dk}^i_{(0,1)}$, we parse the underlying vector $\boldsymbol{y}^{i,1}_{n+1}$ as $(p^{\beta,i}_{n+1} := 0, z_{n+1} := 1, 0)$.

Game $\mathbf{G}_2$: The exception is that instead of generating a zero share for each $i \in \mathcal{HI}$ as $\mathsf{share}^{[n]}_i := \mathsf{pz}_i \cdot t_i = [\![\sum_{k\in[n]\setminus\{i\}}(-1)^{k<i} t_k \cdot t_i]\!]_2$ for any honest query on (i, v^0_i, v^1_i), the challenger generates $t_{i,k} \xleftarrow{\$} \mathbb{Z}_p$ for all $i, k \in \mathcal{HI}$ to replace values $t_k \cdot t_i$. This change enables a set of uniform zero shares between voters in $\mathcal{HI}$, each is denoted as $\mathsf{share}^{\mathcal{HI}}_i$ and satisfies $\mathsf{share}^{[n]}_i = [\![\sum_{k\in[n]\setminus\mathcal{HI}}(-1)^{k<i} t_k \cdot t_i + \mathsf{share}^{\mathcal{HI}}_i]\!]_2$.
The indistinguishability is implied by the DDH assumption in $\mathbb{G}_2$ for each instance $i, k \in \mathcal{HI}$ where given $[\![t_i]\!]_2, [\![t_k]\!]_2$, the adversary guesses between $[\![t_i \cdot t_k]\!]_2$ and $[\![t_{i,k}]\!]_2$.

Game $\mathbf{G}_3$: The first exception is instead of using $\boldsymbol{x}^0_i = (v^\beta_i, \mathsf{share}^{[n]}_i, 0, 0)$ as the message to FH-IPFE encryption for any honest query on (i, v^0_i, v^1_i), the challenger uses the message $\boldsymbol{x}^1_i = (0, \sum_{k\in[n]\setminus\mathcal{HI}}(-1)^{k<i} t_k \cdot t_i, 1)$. The second exception is instead of using $\boldsymbol{y}^{i,0}_j = (p^{\beta,i}_j, z_j, 0, 0)$ for all $j \in [n+1]$ to FH-IPFE functional-key generation on the QPKProcess query, the challenger uses $\boldsymbol{y}^{i,1}_j = (0, z_j, z_j \cdot \mathsf{share}^{\mathcal{HI}}_i + v^\beta_i \cdot p^{\beta,i}_j)$ for all $j \in [n+1]$.
The indistinguishability is implied by the IND-CPA security of FH-IPFE, as we have $\langle \boldsymbol{x}^0_i, \boldsymbol{y}^{i,0}_j \rangle = \langle \boldsymbol{x}^1_i, \boldsymbol{y}^{i,1}_j \rangle = v^\beta_i \cdot p^{\beta,i}_j + z_j \cdot \mathsf{share}^{[n]}_i$ for $i \in \mathcal{HI}$, $j \in [n+1]$.

Game $\mathbf{G}_4$: The exception is instead of using $\boldsymbol{y}^i_j = (0, z_j, z_j \cdot \mathsf{share}^{\mathcal{HI}}_i + v^\beta_i \cdot p^{\beta,i}_j)$ for all $j \in [n]$ to FH-IPFE functional-key generation on the QPKProcess query, the challenger uses the vector $\boldsymbol{y}^i_j = (0, z_j, \mathsf{share}^{\mathcal{HI}}_{i,j} + v^\beta_i \cdot p^{\beta,i}_j)$ for all $j \in [n]$, where $\mathsf{share}^{\mathcal{HI}}_{i,j}$ is a zero share between $\mathcal{HI}$ of voter i and random by each j.
The indistinguishability is implied by the DDH assumption in $\mathbb{G}_1$: one re-indexes $\mathcal{HI} = k_1, ..., k_{|\mathcal{HI}|}$, given random $\{[\![\mathsf{share}^{\mathcal{HI}}_{k_i}]\!]_1\}_{i\in[|\mathcal{HI}|-1]}$[6] and random $[\![z_j]\!]_1$ for each j, the adversary guesses if the challenge $\{[\![c_{k_i}]\!]_1\}_{i\in[|\mathcal{HI}|-1]}$ is equal to $\{[\![z_j \cdot \mathsf{share}^{\mathcal{HI}}_{k_i}]\!]_1\}_{i\in[|\mathcal{HI}|-1]}$ or $\{[\![\mathsf{share}^{\mathcal{HI}}_{i,j}]\!]_1\}_{i\in[|\mathcal{HI}|-1]}$. One can simulate either $[\![z_j \cdot \mathsf{share}^{\mathcal{HI}}_{k_{|\mathcal{HI}|}}]\!]_1$ or $[\![\mathsf{share}^{\mathcal{HI}}_{k_{|\mathcal{HI}|},j}]\!]_1$ by computing $-\sum_{i\in[|\mathcal{HI}|-1]}[\![c_{k_i}]\!]_1$.

Game $\mathbf{G}_5$: This game is identical to $\mathbf{G}_4$, except that instead of using $\boldsymbol{y}^i_j = (0, z_j, \mathsf{share}^{\mathcal{HI}}_{i,j} + v^\beta_i \cdot p^{\beta,i}_j)$ for all $j \in [n]$ to FH-IPFE functional-key generation on the QPKProcess query, the challenger uses $\boldsymbol{y}^i_j = (0, z_j, \mathsf{share}^{\mathcal{HI}}_{i,j} + v^0_i \cdot p^{0,i}_j)$. The indistinguishability is perfect: $\{\mathsf{share}^{\mathcal{HI}}_{i,j}\}_{i\in\mathcal{HI}}$ and $\{\mathsf{share}^{\mathcal{HI}}_{i,j} + (v^0_i \cdot p^{0,i}_j - v^\beta_i \cdot p^{\beta,i}_j)\}_{i\in\mathcal{HI}}$ are of the same distribution for all $j \in [n]$, where we have $\sum_{i\in\mathcal{HI}}(v^0_i \cdot p^{0,i}_j - v^\beta_i \cdot p^{\beta,i}_j) = 0$ because of Equation (1).

In game $\mathbf{G}_5$, there is no information on the challenge bit β, which completes the proof. □

In the above security proof, we rely solely on the security of FH-IPFE instances that are run by honest users and the zero shares between them to switch the challenge bit into a fixed one. This implies that malformed or duplicated ballots submitted by the adversary do not contribute to winning the game. Additionally, the proof requires knowledge of the key vectors $(\boldsymbol{p}^{\beta,i})_{i\in\mathcal{HS}}$ and $(\boldsymbol{p}^{0,i})_{i\in\mathcal{HS}}$ to answer the QPKProcess query. These vectors are only defined after the voter set $\mathcal{HS}$ and their corresponding queries are determined, thus requiring a selective setting. The requirement for symmetric security is because the secret key of FH-IPFE enables the ballot decryption.

3.4 Tallying Efficiency

The protocol in Figure 6 involves the SelfTally algorithm for tallying the votes. The computational cost of this step is dominated by up to n^2 pairing-based FE decryptions and n^2 group additions, when all n registered users are honest and send their ballots. While this step is computationally intensive, it is highly parallelisable: assuming pairing and group operations are treated as unit-cost, the computation can be structured as a circuit of depth $O(\log n)$. This indicates that the tallying process is amenable to efficient parallelisation.

[6] These shares do not appear in $\mathbb{G}_2$ in this game, even under the output of the QFaultCrt query, as voters in $\mathcal{HI}$ reveals only their pairwise randomness with dropout voters.

4 Voting with Offline Mix-Net

In this section, we present a more complete version of the proof-of-concept voting protocol in Section 3, designed to support the generation of one-time pads, random permutations, and their corresponding functional encryption keys within a multi-authority framework. The enhanced protocol introduces a key modification: instead of a centralised generation of FH-IPFE secret keys, each voter generates their own secret key. While we still assume a single trusted authority for intuition, its role shifts significantly. During the offline processing phase, the authority now generates functional keys based on *a set of basis functional keys encrypted by voters*. In other words, the functional keys for tallying are derived homomorphically from these encrypted basis keys. To achieve this, we leverage the homomorphic and structure-preserving properties of FH-IPFE.

To demonstrate how this role can be distributed among multiple authorities, we introduce two protocols: OMIX, which enables servers to sequentially embed a permutation in the functional keys, and OPAD, which allows servers to collaboratively embed one-time pads and key randomness. Once these protocols return their outputs, the authority holding the PKE secret key[7] finalises the process by decrypting the encrypted functional keys. We note that the tallying phase remains unchanged from the proof-of-concept scheme, whilst on the voter side the computational and communication costs during registering and voting remain independent of the number of voters.

4.1 OMIX-based Voting Framework

Our protocol in Figure 7 is designed for the mixing result function (see Definition 2) and a voter space $\mathbb{V} = \{1, \ldots, V\} \subset \mathbb{Z}_p$ where V is a polynomially bounded value in the security parameter. The modification to the proof-of-concept scheme lies in the offline phase, specifically in the generation of FH-IPFE secret keys and the derivation of functional keys for tallying. While a formal description is provided in Figure 8, we outline the intuition behind this construction.

KeyGen During key generation, each voter locally generates an FH-IPFE secret key along with a set of basis FH-IPFE functional keys for two-dimensional inner products. The first slot is designated for votes, while the second slot is designated for one-time pads. The basis functional keys consist of three specific types: $(1, 0)$ for decrypting the vote slot, $(0, 1)$ for adding a one-time pad in the second slot, and $(0, 0)$ for re-randomising the key randomness. Thanks to the homomorphic property of functional keys, these basis keys can be combined to produce freshly generated functional keys of the form $(1, z_j)$ or $(0, z_j)$, as in the proof-of-concept scheme. However, to prevent the combiner from directly using $(1, 0)$ to decrypt the vote, each voter applies a layer of public-key encryption onto the key $(1, 0)$, thereby hiding its information. Consequently, the basis functional keys must now include an encrypted version of $(1, 0)$.

OMIX Upon receiving the basis functional keys, the authority's role is divided into two tasks, the first of which involves shuffling the keys according to a permutation. This process begins with an identity matrix of encrypted functional keys, where the diagonal elements correspond to the $(1, 0)$ keys of each voter. Each column is associated with a voter, while the remaining entries in each column are filled with $(0, 0)$ keys that are parsed as encrypted with null randomness. The authority then permutes the rows of this matrix and re-randomises the public-key encryption layer of each functional key to ensure that the applied permutation remains hidden.

OPAD The second task is to embed one-time pads and functional-key randomness into the functional keys, producing an output also structured as a matrix. This step leverages the homomorphic properties of functional encryption: for each matrix entry, the authority multiplies the $(0, 1)$ keys by the one-time pads and the $(0, 0)$ keys by the randomness, then sums the results encrypted under the public key, ensuring that the embedded one-time pads and randomness remain hidden before they are added to keys from OMIX. To collaboratively generate the one-time pads $[\![\boldsymbol{z}]\!]_1$ in a multi-authority setting, we additionally require that they be encrypted to ensure independence.

To finalise the offline processing phase, the authority sums the matrices of public-key ciphertexts

[7] As in standard approaches, the PKE scheme can be extended to a threshold variant to distribute the secret-key authority.

obtained from the previous tasks and performs a decryption. Due to the structure-preserving properties of functional keys under the homomorphic public-key encryption scheme, the decrypted functional keys are identical to those that would have been generated in the proof-of-concept scheme.

Although the above intuition of offline processing works correctly, it additionally introduces a vulnerability to copy-ballot attacks. In this attack, an adversary can target a voter by submitting an identical set of basis functional keys and later casting an identical ciphertext as its ballot. This succeeds because the processed functional keys corresponding to the adversary can then decrypt the copied ballot. To prevent this, we impose non-malleability on basis functional keys by requiring each voter to provide a proof of knowledge for the vote-slot decrypting key $(1, 0)$, i.e. the plaintext under the public-key encryption. Adapted from the idea in [29], this proof is constructed using a non-interactive Σ-protocol, which is hardwired with the voter's identity

From the above analysis, the building blocks to ensure ballot privacy of the voting protocol in Figure 7 will contain an FH-IPFE scheme $(\mathsf{ip.Setup}, \mathsf{ip.DKGen}, \mathsf{ip.Enc}, \mathsf{ip.Dec})$, a PKE scheme $(\mathsf{PKE.SetUp}, \mathsf{PKE.Enc}, \mathsf{PKE.Dec})$, and a Σ-protocol denoted by $\mathsf{NIZK}_{\mathcal{R}_{\mathsf{FS}}}$ (described in Figure 13). Furthermore, in order to complete the protocol with universal verifiability, we will also use NIZK proof systems to ensure correct execution of parties, namely for the following relations: $\mathcal{R}_{\mathsf{dk}}$ for voter public keys, $\mathcal{R}_{\mathsf{ct}}$ for voter encrypted ballots, $\mathcal{R}_{\mathsf{Mix}}$ for shuffling functional keys, $\mathcal{R}_{\mathsf{otp}}$ for generating one-time pads and key randomness, $\mathcal{R}_{\mathsf{Dec}}$ for decrypting in the offline phase, and $\mathcal{R}_{\mathsf{ck}}$ for generating correcting keys in case of dropout voters.

Proofs of correctness, corrective fault tolerance, and client scalability are provided in Section C. Ballot privacy is proved under the assumption that the permutation embedding (OMIX) and one-time-pad embedding (OPAD) tasks are executed by a trusted server. In a later section, we introduce constructions of OMIX and OPAD that account for the presence of corrupted servers and show that these protocols can be securely integrated into our voting protocol.

From a usability standpoint, our protocol only involves a single-pass voter registration, in which each voter locally generates and sends the basis keys. To further enhance usability, the registration phase can be extended to include a distributed FE-key generation mechanism among authorities. This would eliminate the need for active voter involvement during registration and facilitate secure key recovery in the event of loss.

4.2 Security Analysis

Theorem 2 (Universal Verifiability). *The voting protocol described in Figure 7 achieves universal verifiability as per Definition 21, assuming the soundness property of the NIZK proof systems for the relations $\mathcal{R}_{\mathsf{dk}}$, $\mathcal{R}_{\mathsf{ct}}$, $\mathcal{R}_{\mathsf{Mix}}$, $\mathcal{R}_{\mathsf{otp}}$, $\mathcal{R}_{\mathsf{Dec}}$ and $\mathcal{R}_{\mathsf{ck}}$.*

Sketch of the Proof. The soundness of NIZK proof systems ensures the correct generation of components in the protocol with respect to consistent secret keys. A complete proof is provided in Section C.

Theorem 3 (Ballot Privacy). *The voting protocol described in Figure 7 achieves ballot privacy in the selective and symmetric-key setting (as per Definition 5), assuming the zero-knowledge property of the NIZK proof systems for the relations in $\{\mathcal{R}_{\mathsf{dk}}, \mathcal{R}_{\mathsf{ct}}, \mathcal{R}_{\mathsf{Mix}}, \mathcal{R}_{\mathsf{otp}}, \mathcal{R}_{\mathsf{Dec}}, \mathcal{R}_{\mathsf{ck}}\}$, the random oracle in the $\mathsf{NIZK}_{\mathcal{R}_{\mathsf{FS}}}$ proof system (described in Figure 13), the IND-CPA security of the PKE scheme, the SXDH assumption in $\mathcal{PG}$ and the IND-CPA security of the FH-IPFE scheme.*

Sketch of the Proof. Compared to the proof-of-concept scheme in Figure 6, the protocol in Figure 7 exposes more information about secrets of honest voters i that have ballots recorded in the ballot box. The adversary gains additional access to: the functional key $\mathsf{ip.dk}^i_{(1,0)}$ for decrypting the vote slot, which is encrypted under PKE as ct_i; the functional keys embedding $\Pi(i)$ that are encrypted under PKE as the i-th column in MBox; the functional keys embedding the one-time pads $\{z_j\}_{j\in[n]}$ and the key randomness $\{\gamma_{\mathsf{IPE},j,i}\}_{j\in[n]}$, encrypted under PKE as the i-th column in OBox.

To eliminate this additional leakage, we modify the protocol via hybrids so that all the above PKE ciphertexts instead encrypt a null value $\mathbf{0}$. By doing so, the adversary's view contains no more information on these voters than in the proof-of-concept scheme. A complete proof is provided in Section C.

$\underline{\mathsf{SetUp}(1^\lambda)}$: The authority specifies $\mathbb{V} = \{1, \dots V\}$ and generates $(\mathsf{PKE.sk}, \mathsf{PKE.pk}) \leftarrow \mathsf{PKE.SetUp}(1^\lambda)$ and $\mathsf{NIZK.pp} \leftarrow \mathsf{NIZK}_{\{\mathcal{R}_{\mathsf{dk}}, \mathcal{R}_{\mathsf{FS}}, \mathcal{R}_{\mathsf{ct}}, \mathcal{R}_{\mathsf{Mix}}, \mathcal{R}_{\mathsf{otp}}, \mathcal{R}_{\mathsf{Dec}}, \mathcal{R}_{\mathsf{ck}}\}}.\mathsf{SetUp}(1^\lambda)$. It outputs $\mathsf{mpk} = (\mathbb{V}, \mathsf{NIZK.pp}, \mathsf{PKE.pk}, \mathcal{PG})$ and stores $\mathsf{msk} = \mathsf{PKE.sk}$.

Apart from SetUp, the offline phase is described with details in Figure 8:

- Each voter id computes $(\mathsf{pk}_{\mathsf{id}}, \mathsf{ek}_{\mathsf{id}}) \leftarrow \mathsf{KeyGen}(\mathsf{id}, \mathsf{mpk})$[a].
- The authority runs $\mathsf{Register}(\mathsf{L}, \mathsf{pk}_{\mathsf{id}})$ to update L with $\mathsf{pk}_{\mathsf{id}}$.
- The authority outputs $\mathsf{hpk} \leftarrow \mathsf{PKProcess}(\mathsf{L}, \mathsf{msk})$.

$\underline{\mathsf{Vote}(\mathsf{ek}_{\mathsf{id}}, v, \mathsf{hpk})}$: Each voter id parses $\mathsf{ek}_{\mathsf{id}} = (\mathsf{pk}_{\mathsf{id}}, \mathsf{ip.sk}_{\mathsf{id}}, t_{\mathsf{id}})$ and computes

- an FH-IPFE encryption: $\mathsf{ip.ct}^{\mathsf{id}} \leftarrow \mathsf{ip.Enc}(\mathsf{ip.sk}_{\mathsf{id}}, ([\![v]\!]_2, \mathsf{pz}_{\mathsf{id}} \cdot t_{\mathsf{id}}, [\![\mathbf{0}^{\rho(\lambda)}]\!]_2))$;
- a proof for correct ciphertext: $\pi^{\mathsf{ct}}_{\mathsf{id}} \leftarrow \mathsf{NIZK}_{\mathcal{R}_{\mathsf{ct}}}.\mathsf{Prove}(\mathsf{ip.ct}^{\mathsf{id}}, \mathsf{pk}_{\mathsf{id}}, \mathsf{pz}_{\mathsf{id}}; \mathsf{ip.sk}_{\mathsf{id}}, t_{\mathsf{id}}, v)$;

and casts $b_{\mathsf{id}} = (\mathsf{ip.ct}^{\mathsf{id}}, \pi^{\mathsf{ct}}_{\mathsf{id}})$ to BB. Registered voter id can abort voting.

$\underline{\mathsf{Append}(\mathsf{BB}, b_{\mathsf{id}}, \mathsf{L}, \mathsf{hpk})}$: Parsing $b_{\mathsf{id}} = (\mathsf{ip.ct}^{\mathsf{id}}, \pi^{\mathsf{ct}}_{\mathsf{id}})$, the authority appends b_{id} to BB if

$$\mathsf{id} \text{ is in } \mathsf{L} \text{ and there does not exist } b'_{\mathsf{id}} \in \mathsf{BB} \text{ and } \mathsf{NIZK}_{\mathcal{R}_{\mathsf{ct}}}.\mathsf{Verify}(\pi^{\mathsf{ct}}_{\mathsf{id}}) = 1;$$

and rejects otherwise.

$\underline{\mathsf{FaultAgg}(\mathsf{BB}, \mathsf{hpk}, \mathsf{id})}$: It parses $\mathsf{V} \subseteq [n]$ as the list of voters id in L whose ballots are in BB. If $\mathsf{V} = [n]$, nothing is output. Otherwise, the fault aggregate for the voter $i \in \mathsf{V}$ that corresponds to id is $\mathsf{fa}_i = \sum_{k \in [n] \setminus \mathsf{V}} (-1)^{k<i} [\![t_k]\!]_2$.

$\underline{\mathsf{FaultCrt}(\mathsf{fa}_{\mathsf{id}}, \mathsf{ek}_{\mathsf{id}})}$: The correcting key is output as $\mathsf{ck}_{\mathsf{id}} = \mathsf{fa}_{\mathsf{id}} \cdot t_{\mathsf{id}}$. It generates a proof for correcting key $\pi^{\mathsf{id}}_{\mathsf{ck}} \leftarrow \mathsf{NIZK}_{\mathcal{R}_{\mathsf{ck}}}.\mathsf{Prove}(\mathsf{fa}_{\mathsf{id}}, [\![t_{\mathsf{id}}]\!]_2, \mathsf{ck}_{\mathsf{id}}; t_{\mathsf{id}})$ and outputs $(\mathsf{ck}_{\mathsf{id}}, \pi^{\mathsf{id}}_{\mathsf{ck}})$.

$\underline{\mathsf{SelfTally}(\mathsf{BB}, \{\mathsf{ck}_{\mathsf{id}}\}_{\mathsf{id}}, \mathsf{hpk})}$: Let $\mathsf{V} \subseteq [n]$ be the list of each voter $\mathsf{id}_i \in \mathsf{L}$ whose ballot is in BB and for which ck_i is given (in case $\mathsf{V} \subsetneq [n]$) and let $\boldsymbol{v}^\star$ be an empty tuple. It obtains $\{\mathsf{ip.dk}^i_j\}_{j,i \in [n]}$ from hpk, $\{\mathsf{ip.ct}^i\}_{i \in \mathsf{V}}$ from BB. For each $j \in [n]$, it appends v'_j from $[\![v'_j]\!]_T = \sum_{i \in \mathsf{V}} \mathsf{ip.Dec}(\mathsf{ip.dk}^i_j, \mathsf{ip.ct}^i) - e([\![z_j]\!]_1, \sum_{i \in \mathsf{V}} \mathsf{ck}_i)$ to $\boldsymbol{v}^\star$ if $[\![v'_j]\!]_T \neq [\![0]\!]_T$ and v'_j can be recovered. The result res is the equivalence class $[\boldsymbol{v}^\star]_\sim$.

$\underline{\mathsf{Verify}(\mathsf{BB}, \{\mathsf{ck}_{\mathsf{id}}\}_{\mathsf{id}}, \mathsf{res}, \mathsf{L}, \mathsf{hpk})}$: It parses $\mathsf{L} = (\mathsf{pk}_i)_{i \in [n]}$ and $\mathsf{V} \subseteq [n]$ as the list of each voter $\mathsf{id}_i \in \mathsf{L}$ whose ballot is in BB and for which ck_i is given (in case $\mathsf{V} \subsetneq [n]$). Let L_i (and BB_i) be the state of L (and BB) before pk_i (and b_i) is appended. It verifies the following phases

- registering: it verifies if $\mathsf{Register}(\mathsf{L}_i, \mathsf{pk}_i)$ outputs 1 for $i \in [n]$;
- processing: it verifies π^{Proc} and $\mathsf{pz}_i = \sum_{k \in [n] \setminus \{i\}} (-1)^{k<i} \cdot [\![t_k]\!]_2$ for $i \in [n]$ in hpk;
- ballot appending: it verifies if $\mathsf{Append}(\mathsf{BB}_i, b_i, \mathsf{L}, \mathsf{hpk})$ outputs 1 for $i \in \mathsf{V}$;
- fault correcting: it computes $\mathsf{fa}_i = \mathsf{FaultAgg}(\mathsf{BB}, \mathsf{hpk}, \mathsf{id})$ and verifies π^i_{ck} for $i \in \mathsf{V}$;
- tallying: it verifies if $\mathsf{res} = \mathsf{SelfTally}(\mathsf{BB}, \{\mathsf{ck}_{\mathsf{id}}\}_{\mathsf{id}}, \mathsf{hpk})$;

and outputs 1 if all the phases are valid, and 0 otherwise.

[a] The argument msk is not required in this construction.

Fig. 7. OMIX-based voting framework

KeyGen(id, mpk): Each voter id processes as follows

1. It generates a zero-sharing secret key $t_{\mathsf{id}} \xleftarrow{\$} \mathbb{Z}_p$.
2. It generates an FH-IPFE secret key $\mathsf{ip.sk}_{\mathsf{id}} \leftarrow \mathsf{ip.Setup}(1^\lambda)$ and functional keys
   (a) for vote-slot decrypting: $\mathsf{ip.dk}^{\mathsf{id}}_{(1,0)} \leftarrow \mathsf{ip.DKGen}(\mathsf{ip.sk}_{\mathsf{id}}, [\![1, 0, \mathbf{0}^{\rho(\lambda)}]\!]_1)$;
   (b) for one-time-pad adding: $\mathsf{ip.dk}^{\mathsf{id}}_{(0,1)} \leftarrow \mathsf{ip.DKGen}(\mathsf{ip.sk}_{\mathsf{id}}, [\![0, 1, \mathbf{0}^{\rho(\lambda)}]\!]_1)$;
   (c) for re-randomising: $\mathsf{ip.dk}^{\mathsf{id}}_{(0,0)} \leftarrow \mathsf{ip.DKGen}(\mathsf{ip.sk}_{\mathsf{id}}, [\![0, 0, \mathbf{0}^{\rho(\lambda)}]\!]_1)$.
3. It encrypts the functional key for vote-slot decrypting under PKE as

$$\mathsf{ct}_{\mathsf{id}} \leftarrow \mathsf{PKE.Enc}(\mathsf{PKE.pk}, \mathsf{ip.dk}^{\mathsf{id}}_{(1,0)});\ \pi^{\mathsf{FS}}_{\mathsf{id}} \leftarrow \mathsf{NIZK}_{\mathcal{R}_{\mathsf{FS}}}.\mathsf{Prove}(\mathsf{id}, \mathsf{PKE.pk}, \mathsf{ct}_{\mathsf{id}}; \mathsf{ip.dk}^{\mathsf{id}}_{(1,0)}).$$

4. It generates $\pi^{\mathsf{dk}}_{\mathsf{id}} \leftarrow \mathsf{NIZK}_{\mathcal{R}_{\mathsf{dk}}}.\mathsf{Prove}(\mathsf{ip.dk}^{\mathsf{id}}_{(0,1)}, \mathsf{ip.dk}^{\mathsf{id}}_{(0,0)}, \mathsf{ct}_{\mathsf{id}}; \mathsf{ip.sk}_{\mathsf{id}})$ for the correct generation of (encrypted) functional keys.
5. It outputs $\mathsf{pk}_{\mathsf{id}} = \{\mathsf{id}, \mathsf{ct}_{\mathsf{id}}, \mathsf{ip.dk}^{\mathsf{id}}_{(0,1)}, \mathsf{ip.dk}^{\mathsf{id}}_{(0,0)}, \pi^{\mathsf{dk}}_{\mathsf{id}}, \pi^{\mathsf{FS}}_{\mathsf{id}}, [\![t_{\mathsf{id}}]\!]_2\}$ and stores $\mathsf{ek}_{\mathsf{id}} = (\mathsf{pk}_{\mathsf{id}}, \mathsf{ip.sk}_{\mathsf{id}}, t_{\mathsf{id}})$.

Register(L, $\mathsf{pk}_{\mathsf{id}}$): Parsing $\mathsf{pk}_{\mathsf{id}} = \{\mathsf{id}, \mathsf{ct}_{\mathsf{id}}, \mathsf{ip.dk}^{\mathsf{id}}_{(0,1)}, \mathsf{ip.dk}^{\mathsf{id}}_{(0,0)}, \pi^{\mathsf{dk}}_{\mathsf{id}}, \pi^{\mathsf{FS}}_{\mathsf{id}}, [\![t_{\mathsf{id}}]\!]_2\}$, the authority appends $\mathsf{pk}_{\mathsf{id}}$ to L if there does not exist $\mathsf{pk}'_{\mathsf{id}} \in \mathsf{L}$ and $\mathsf{NIZK}_{\mathcal{R}_{\mathsf{FS}}}.\mathsf{Verify}(\pi^{\mathsf{FS}}_{\mathsf{id}}) = 1$ and $\mathsf{NIZK}_{\mathcal{R}_{\mathsf{dk}}}.\mathsf{Verify}(\pi^{\mathsf{dk}}_{\mathsf{id}}) = 1$.

PKProcess(L, msk): The authority processes as follows

1. It parses $\mathsf{L} = (\mathsf{pk}_i)_{i\in[n]} = (\mathsf{id}_i, \mathsf{ct}_i, \mathsf{ip.dk}^i_{(0,1)}, \mathsf{ip.dk}^i_{(0,0)}, \pi^{\mathsf{dk}}_i, \pi^{\mathsf{FS}}_i, [\![t_i]\!]_2)_{i\in[n]}$.
2. For $i \in [n]$, it computes a zero-sharing public key: $\mathsf{pz}_i = \sum_{k\in[n]\setminus\{i\}} (-1)^{k<i} \cdot [\![t_k]\!]_2$
3. It generates random $\Pi : [n] \to [n]$, $\boldsymbol{z} = (z_j)_{j\in[n]}$ and $(\gamma_{\mathsf{IPE},j,i})_{j,i\in[n]}$.
4. The initial $n \times n$-mix box is parsed by default as MBox_I where

$$\mathsf{MBox}_I(j,i) = \begin{cases} \mathsf{ct}_i & \text{if } j = i, \\ \mathsf{PKE.Enc}(\mathsf{PKE.pk}, \mathsf{ip.dk}^i_{(0,0)}; 0) & \text{if } j \neq i. \end{cases}$$

5. OMIX: It generates random $(\gamma_{\mathsf{PKE},j,i})_{j,i\in[n]}$ and mixes MBox_I into MBox as

$$\mathsf{MBox}(\Pi(j), i) = \mathsf{MBox}_I(j,i) + \gamma_{\mathsf{PKE},j,i} \cdot \mathsf{PKE.Enc}(\mathsf{PKE.pk}, [\![\mathbf{0}]\!]_1; 1).$$

6. OPAD: It generates $\mathsf{ct}^{\mathsf{PAD}}_j = \mathsf{PKE.Enc}(\mathsf{PKE.pk}, [\![z_j]\!]_1)$ and OBox as

$$\mathsf{OBox}(j,i) = \mathsf{PKE.Enc}(\mathsf{PKE.pk}, z_j \cdot \mathsf{ip.dk}^i_{(0,1)} + \gamma_{\mathsf{IPE},j,i} \cdot \mathsf{ip.dk}^i_{(0,0)}).$$

7. The authority decrypts $\mathsf{ct}^{\mathsf{PAD}}_j$ into $[\![z_j]\!]_1$ and obtains the functional keys as

$$\mathsf{ip.dk}^i_j = \mathsf{PKE.Dec}(\mathsf{PKE.sk}, \mathsf{OBox}(j,i) + \mathsf{MBox}(j,i))\ \forall j, i \in [n]$$

8. It generates a proof for correct processing as π^{Proc}, consisting of

$$\pi^{\mathsf{Mix}} \leftarrow \mathsf{NIZK}_{\mathcal{R}_{\mathsf{Mix}}}.\mathsf{Prove}(\mathsf{MBox}, (\mathsf{pk}_i)_{i\in[n]}, \mathsf{PKE.pk}; \Pi, (\gamma_{\mathsf{PKE},j,i})_{j,i\in[n]})$$
$$\pi^{\mathsf{otp}} \leftarrow \mathsf{NIZK}_{\mathcal{R}_{\mathsf{otp}}}.\mathsf{Prove}(\mathsf{OBox}, (\mathsf{ct}^{\mathsf{PAD}}_j)_{j\in[n]}, (\mathsf{pk}_i)_{i\in[n]}, \mathsf{PKE.pk}; (z_j)_{j\in[n]}, (\gamma_{\mathsf{IPE},j,i})_{j,i\in[n]})$$
$$\pi^{\mathsf{Dec}} \leftarrow \mathsf{NIZK}_{\mathcal{R}_{\mathsf{Dec}}}.\mathsf{Prove}([\![\boldsymbol{z}]\!]_1, (\mathsf{ip.dk}^i_j)_{j,i\in[n]}, (\mathsf{ct}^{\mathsf{PAD}}_j)_{j\in[n]}, \mathsf{MBox}, \mathsf{OBox}; \mathsf{PKE.sk})$$

9. It publishes $\mathsf{hpk} = \mathsf{mpk}||(\mathsf{ct}^{\mathsf{PAD}}_j)_{j\in[n]}, \mathsf{MBox}, \mathsf{OBox}, (\mathsf{ip.dk}^i_j)_{j,i\in[n]}, \pi^{\mathsf{Proc}}, \{\mathsf{pz}_i\}_{i\in[n]}, [\![\boldsymbol{z}]\!]_1$

Fig. 8. Description of offline phase algorithms in Figure 7.

5 OMIX from Linearly-Homomorphic Signature

In the multi-server setting, the verifiability of an OMIX protocol is crucial for both ballot privacy and universal verifiability within the voting framework presented in the previous section. Although OMIX operates in the offline phase, the permutation-contributing process still requires sequential execution by mix-servers on encrypted basis functional keys.

On the other hand, unlike traditional mixes, OMIX shuffles a matrix of ciphertexts, requiring mix-servers to re-randomise all ciphertexts and permute matrix rows. This introduces a novel shuffling language for verification, for which no dedicated proof systems currently exist.

To address this, we enhance a tag-randomisable linear homomorphic signature (LHS)[8] with two key properties: tag-binding unforgeability (a stronger security notion) and two-dimensional tags (a novel feature in its tags). The resulting scheme not only enables efficient verification for this new shuffling language but also serves as a building block in scalable OMIX.

With this novel tool, we construct an OMIX protocol where verification time remains independent of the number of mix servers. Importantly, this construction enables a secure integration in the voting framework.

5.1 Linearly-Homomorphic Signature

In this work, we rely on the concept of linearly-homomorphic signatures with randomisable tags, as introduced in [35], with an instantiation based on the construction from [26].

Definition 9 (Linearly-Homomorphic Signature (LH-Sign)). *A linearly-homomorphic signature scheme with messages in $\mathcal{M} \in \mathbb{G}^\rho$, for a cyclic group $\mathbb{G}$ of prime order p, some dimension $\rho \in \mathsf{poly}(\lambda)$, and some tag set $\mathcal{T}$, consists of eight algorithms:*

- $\mathsf{SetUp}(1^\lambda)$*: Given a security parameter λ, it outputs the global parameter* pp*, which includes the tag space $\mathcal{T}$;*
- $\mathsf{KeyGen}(\mathsf{pp}, \rho)$*: Given a public parameter* pp *and an integer ρ, it outputs a key pair* $(\mathsf{sk}, \mathsf{vk})$*, where* vk *implicitly contains* pp *and* sk *implicitly contains* vk*;*
- $\mathsf{NewTag}(\mathsf{sk})$*: Given a signing key* sk*, it outputs a tag τ and its associated secret key $\tilde{\tau}$;*
- $\mathsf{VerifTag}(\mathsf{vk}, \tau)$*: Given a verification key* vk *and a tag τ, it outputs 1 if the tag is valid and 0 otherwise;*
- $\mathsf{Sign}(\mathsf{sk}, \tilde{\tau}, \boldsymbol{M})$*: Given a signing key, a secret key tag $\tilde{\tau}$ and a vector-message $\boldsymbol{M} = (M_i)_i \in \mathbb{G}^\rho$, it outputs the signature σ under the tag τ;*
- $\mathsf{DerivSign}(\mathsf{vk}, \tau, (\omega_i, \boldsymbol{M}_i, \sigma_i)_{i\in[\ell]})$*: Given a public key* vk*, a tag τ and ℓ tuples of weights $\omega_i \in \mathbb{Z}_p$ and signed messages $\boldsymbol{M}_i$ in σ_i, it outputs a signature σ on the vector $\boldsymbol{M} = \sum_{i=1}^{\ell} \omega_i \cdot \boldsymbol{M_i}$ under the tag τ;*
- $\mathsf{Verif}(\mathsf{vk}, \tau, \boldsymbol{M}, \sigma)$*: Given a verification key* vk*, a tag τ, a vector-message $\boldsymbol{M}$ and a signature σ, it outputs 1 if* $\mathsf{VerifTag}(\mathsf{vk}, \tau) = 1$ *and σ is also valid relative to* vk *and τ, and 0 otherwise.*
- $\mathsf{RandTag}(\mathsf{vk}, \tau, \boldsymbol{M}, \sigma)$*: Given a verification key* vk*, a tag τ and a signature σ on a vector-message $\boldsymbol{M} = (M_i)_i \in \mathbb{G}^\rho$, it outputs τ' and an adapted signature σ'.*

Correctness. For all security parameters $\lambda \in \mathbb{N}$, parameters pp generated by $\mathsf{SetUp}(1^\lambda)$, key pairs $(\mathsf{sk}, \mathsf{vk})$ generated by $\mathsf{KeyGen}(\mathsf{pp}, \rho)$, and for any tags $(\tau, \tilde{\tau})$ generated by $\mathsf{NewTag}(\mathsf{sk})$, if $\sigma_i = \mathsf{Sign}(\mathsf{sk}, \tilde{\tau}, \boldsymbol{M}_i)$ are valid signatures for $i = 1, \ldots, \ell$ and $\sigma = \mathsf{DerivSign}(\mathsf{vk}, \tau, \{\omega_i, \boldsymbol{M}_i, \sigma_i\}_{i=1}^{\ell})$ from some scalars ω_i, then both

$$\mathsf{VerifTag}(\mathsf{vk}, \tau) = 1 \qquad \mathsf{Verify}(\mathsf{vk}, \tau, \boldsymbol{M}, \sigma) = 1.$$

Tag Randomisability. Given any valid tuple $(\mathsf{vk}, \tau, \boldsymbol{M}, \sigma)$, then the tuple $(\mathsf{vk}, \tau', \boldsymbol{M}, \sigma')$ generated by $\mathsf{RandTag}(\mathsf{vk}, \tau, \boldsymbol{M}, \sigma)$ has a tag τ' unlinkable to τ.

[8] In this work, we consider only linearly-homomorphic signatures with randomisable tags (see Section 5.1). From this point forward, we refer to them simply as LHS.

Definition 10 (Unforgeability for LH-Sign). *For a LH-Sign scheme with messages in $\mathbb{G}^\rho$, for any adversary $\mathcal{A}$ that, given tags and signatures on messages $(\boldsymbol{M}_i)_i$ under tags $(\tau_i)_i$ both of its choice (for Chosen-Message Attacks), outputs a valid tuple $(\mathsf{vk}, \tau, \boldsymbol{M}, \sigma)$ with $\tau \in \mathcal{T}$, there must exist some tag $\tau' \in \{\tau_i\}_i$ and coefficients $(\omega_i)_{i \in I_{\tau'}}$, where $I_{\tau'}$ is the index set of messages already signed under τ', such that*

$$\boldsymbol{M} = \sum_{i \in I_{\tau'}} \omega_i \cdot \boldsymbol{M}_i$$

with overwhelming probability.

5.2 Enhancing Linearly-Homomorphic Signature

Our improvements to LHS consist of a stronger unforgeability notion and a novel two-dimensional tagging structure.

Extension of Unforgeability The standard unforgeability of LHS, as in Definition 10, falls short in ensuring a correct mix when we sign all entries within the same row under a (row) tag. It does not guarantee that valid messages signed under a re-randomised tag are all derived from messages signed under the same original tag. This limitation leaves room for a malicious mix-server to combine entries from different rows to forge valid rows in the output. Addressing this issue necessitates an extension of the security notion of LHS, which we define below.

Definition 11 (Tag-Binding Unforgeability for LH-Sign). *For a* LHS *scheme with messages in $\mathbb{G}^\rho$, for any adversary $\mathcal{A}$ that, given tags and signatures on messages $(\boldsymbol{M}_i)_i$ under tags $(\tau_i)_i$ both of its choice (for Chosen-Message Attacks), outputs a set of valid tuples $(\mathsf{vk}, \tau, \boldsymbol{M}'_j, \sigma_j)_j$ with $\tau \in \mathcal{T}$, there must exist some queried tag $\tau' \in (\tau_i)_i$ and coefficients $(\omega_{i,j})_{i \in I_{\tau'}}$ corresponding to each j, where $I_{\tau'}$ is the index set of messages already signed under τ', such that $\boldsymbol{M}'_j = \sum_{i \in I_{\tau'}} \omega_{i,j} \cdot \boldsymbol{M}_i$ with overwhelming probability.*

In Theorem 7, we show that any LHS scheme that has *homomorphic verification* property (see Definition 24) will also satisfy tag-binding unforgeability.

Two-Dimensional Tags We want to ensure that elements can be combined (e.g. for re-randomisation) only if they correspond to the same position in the matrix. Running two LHS instances, one for rows and one for columns, fails to meet this security requirement. We then introduce LHS with two-dimensional tags, where one dimension corresponds to the row and the other to the column. As for one-dimensional tags, the security guarantees that only messages signed under tags of the same row and column can be combined (e.g. equal tags). At the same time, public randomisation of row (sub-)tags is available, and the tag-binding property (see Definition 26) still hold with respect to these row tags.

Definition 12 (Two-Dimensional Tags). *A* LHS *scheme (as defined in Definition 9) admits two-dimensional tags if the tag set $\mathcal{T}$ can be split into $\mathcal{T}_1 \times \mathcal{T}_2$ where $\mathcal{T}_1$ is a randomisable set. Specifically, the scheme admits a* RandTag *algorithm as follows:* $\mathsf{RandTag}(\mathsf{vk}, \tau, \boldsymbol{M}, \sigma)$ *takes as input a verification key* vk*, a tag $\tau = (\tau_1, \tau_2)$ where $\tau_1 \in \mathcal{T}_1$ and $\tau_2 \in \mathcal{T}_2$, and a signature σ on a vector-message $\boldsymbol{M} = (M_i)_i \in \mathbb{G}^\rho$; it outputs $\tau' = (\tau'_1, \tau_2)$ where τ'_1 is a re-randomisation of τ_1 and an adapted signature σ'.*

To obtain a scheme that achieves both above properties, we enhance the LHS scheme in [35] (see Section D.4). The resulting scheme integrates two-dimensional tags, making it well-suited for use in OMIX.

5.3 OMIX Construction

Given computations that an OMIX protocol should perform in Section 4, we propose an OMIX construction detailed in Figure 9, Figure 10, and Figure 11. This construction enables verifiable execution across multiple mix-servers. The intuition behind our approach is outlined in the following phases.

Setup The setup outputs a pairing group $\mathcal{PG}$[9] as parameters for the two-dimensional-tag LHS (2-tag-LHS) scheme. OMIX adopts the PKE scheme, which is instantiated in $\mathbb{G}_1$ of $\mathcal{PG}$ from the voting protocol, as a public parameter. Additionally, the setup provides a $\mathcal{RO}$-modeled hash function H mapping onto $\mathbb{G}_1$ and a common reference string for the $\mathsf{NIZK}_{\mathsf{DH}}$ proof system to verify Diffie-Hellman (DH) tuples.

Key Generation This phase aligns with the local secret-key generation in the voting protocol, each voter generates an FH-IPFE secret key and submits encrypted basis functional keys for registration. We designate the vote-slot decrypting key $(1, 0)$, encrypted under PKE, as the voter's secret key. Additionally, each mix-server registers for participation in OMIX by submitting its verification key for a standard signature scheme.

The OMIX protocol requires a Certificate Authority ($\mathcal{CA}$) that holds the secret signing key for the 2-tag-LHS scheme. The role of $\mathcal{CA}$ is to certify (by signing) mix inputs contributed by voters during registration. The soundness of OMIX verification relies on the secrecy of this signing key, which is considered the master secret key of the protocol. Consequently, the corresponding 2-tag-LHS verification key serves as the master public key, later used for mix verification.

Certification Once the voter and mix-server registration phase has concluded, $\mathcal{CA}$ certifies mix-servers' verification keys, for example by publishing a list. Certifying mix inputs from voters consists of two steps.

The first step is to arrange the basis functional keys following an identity matrix (initial permutation). The keys $(1, 0)$ of voters are placed along the diagonal. Each column i is filled with the key $(0, 0)$ of voter i. To match the encrypted key $(1, 0)$, each key $(0, 0)$ is parsed as encrypted under PKE with null randomness. Additionally, a PKE-randomising ciphertext (or PKE randomiser) is set. This step is fully transparent and deterministic.

In the second step, $\mathcal{CA}$ signs each encrypted key in the matrix using the 2-tag-LHS secret key, assigning a two-dimensional tag corresponding to the key's row and column indices. To enable verifiable re-randomisation of the PKE layer of these keys, $\mathcal{CA}$ also signs the PKE randomiser under tags corresponding to each matrix position. One has a matrix of 2-tag-LHS signatures for the PKE randomiser.

An important remark is that the soundness cannot be obtained with 2-tag-LHS only, as the adversary can mount a remove-and-replace attack: it removes one row, duplicates another row to fill in, re-randomises row tags and shuffles. Due to row tag unlinkability, such an attack would pass verification, as all signatures are valid. To counter this, we key the rows by introducing an additional column of their hash outputs, which $\mathcal{CA}$ also signs. Mix-servers re-randomise these row keys by multiplying them with a secret exponent. By the Diffie Hellman assumption, unlinkability is preserved, while a DH-tuple proof ensures that the sum of the initial keys and that of the mixed keys are the same up to this exponent. This prevents remove-and-replace attacks unless the adversary can compute the discrete-logarithm relation between the hash outputs.

The mix box is initialised with the identity matrix of functional keys and their signatures; the matrix of PKE-randomiser signatures; and the column of row-key signatures. All signed elements are tagged by their respective row and column indices.

Mix On input a mix box, each server re-randomises the functional-key ciphertexts and the row keys; and adapt their signatures correspondingly. The server re-randomises the row tags and shuffles the row indices of all signed elements and their signatures. It generates a proof for the DH-tuple formed by the input row keys and the shuffled row keys. The last step is to authenticate the mixed output by signing the proof using a standard signature scheme. The mix-server passes the shuffled box to the next server, along with the proof and its signature.

Verification This phase is described in Figure 11, where any party can verify the following: the output mix box is contributed by all registered servers (via standard signature verification), the initial mix box is correctly signed (via 2-tag-LHS verification), and the output mix box is correctly permuted and re-randomised from the initial mix box (via 2-tag-LHS and DH-proof verification).

To extend that the proof size and verification cost remain independent of the number of mix-servers, one can adopt the strategy proposed in [35]. This approach utilises malleable NIZK systems,

[9] This is the same group in which the PKE and FH-IPFE schemes are instantiated.

such as Groth-Sahai proofs [31], allowing servers to sequentially update a global proof for the re-randomised DH-tuple in each round. At the end of the mixing process, all servers verify and sign this global proof using the multi-signature scheme of Boneh-Drijvers-Neven [13].

We prove the soundness of the protocol in Theorem 9 and unlinkability in Theorem 10. A stronger form of unlinkability, which enables permutation extractability during the simulation of the unlinkability proof, ensures that the protocol can be securely integrated into the voting system. Additionally, as the voting protocol requires OPAD, we provide a construction in Figure 16 and respective security proofs in Section E.3.

We also note that by relying on 2-tag-LHS, the OMIX protocol requires trust in the certificate authority. However, its role is static, akin to generating a common reference string (CRS) for a succinct proof system (e.g., SNARKs): the authority signs the initial mix inputs once and remains offline for the rest of the protocol. These certification signatures effectively serve as a CRS for a shuffling-proof system.

6 Conclusion

In this work, we propose the first construction of mix-net-based elections with self-tallying and client scalability. This approach opens several directions for future research, as outlined below.

Post-quantum security Recent scalable mix-net frameworks rely upon pairing-friendly groups [1, 35], rendering them incompatible with post-quantum security. By foregoing this particular notion of scalability, our framework employs only one-time-secure encryption from FH-IPFE, enabling a plausibly post-quantum secure construction. We leave the development of such a construction as an open question.

Receipt-freeness In this work, we concentrate on building a novel theoretical approach to using offline mix-nets within the self-tallying setting. Receipt-freeness is a valuable additional property that prevents vote selling. It would be interesting to investigate this property in the context of offline mix-nets and self-tallying, for instance by further leveraging the homomorphic properties of cryptographic primitives. We leave a detailed realisation and thorough analysis of this for future work.

Sub-quadratic tallying Our current tallying procedure exhibits quadratic complexity in the number of ballots. However, this does not represent a lower bound for tallying, and as such we highlight as a new research direction the construction of a more efficient offline mix-net.

Acknowledgements. This work was supported in part by the France 2030 ANR Project ANR-22-PECY-003 SecureCompute.

References

1. Abe, M., Nanri, M., Ohkubo, M., Kempner, O.P., Slamanig, D., Tibouchi, M.: Scalable mixnets from mercurial signatures on randomizable ciphertexts. Cryptology ePrint Archive, Paper 2024/1503 (2024), https://eprint.iacr.org/2024/1503
2. Adida, B.: Helios: Web-based open-audit voting. In: van Oorschot, P.C. (ed.) USENIX Security 2008: 17th USENIX Security Symposium. pp. 335–348. USENIX Association, San Jose, CA, USA (Jul 28 – Aug 1, 2008)
3. Adida, B., Wikström, D.: How to shuffle in public. In: Vadhan, S.P. (ed.) TCC 2007: 4th Theory of Cryptography Conference. Lecture Notes in Computer Science, vol. 4392, pp. 555–574. Springer, Heidelberg, Germany, Amsterdam, The Netherlands (Feb 21–24, 2007). https://doi.org/10.1007/978-3-540-70936-7_30
4. Adida, B., Wikström, D.: Offline/online mixing. In: Arge, L., Cachin, C., Jurdzinski, T., Tarlecki, A. (eds.) ICALP 2007: 34th International Colloquium on Automata, Languages and Programming. Lecture Notes in Computer Science, vol. 4596, pp. 484–495. Springer, Heidelberg, Germany, Wroclaw, Poland (Jul 9–13, 2007). https://doi.org/10.1007/978-3-540-73420-8_43
5. Arapinis, M., Lamprou, N., Mareková, L., Zacharias, T., Ackermann, L., Georgiou, P.: E-cclesia: Universally composable self-tallying elections. Cryptology ePrint Archive, Paper 2020/513 (2020), https://eprint.iacr.org/2020/513

$\underline{\mathsf{OMIX.Setup}(1^\lambda)}$:
$\mathsf{LHS.pp} := \mathcal{PG} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, e) \leftarrow \text{2-tag-}\mathsf{LHS.SetUp}(1^\lambda)$;
$\mathsf{NIZK_{DH}.pp} \leftarrow \mathsf{NIZK_{DH}.SetUp}(1^\lambda)$; $(\mathsf{PKE.sk}, \mathsf{PKE.pk}) \leftarrow \mathsf{PKE.SetUp}(1^\lambda, \mathbb{G}_1)$;
$H : \{0,1\}^* \to \mathbb{G}_1$;
Let $\mathsf{Mix.pp} = (\mathcal{PG}, \mathsf{LHS.pp}, \mathsf{NIZK_{DH}.pp}, \mathsf{PKE.pk}, H)$.

$\underline{\mathsf{OMIX.KeyGen}(\mathsf{Mix.pp})}$:
Certificate Authority $\mathcal{CA}$ generates $(\mathsf{LHS.sk}, \mathsf{LHS.vk}) \leftarrow \text{2-tag-}\mathsf{LHS.KeyGen}()$ and sets $\mathsf{msk} = \mathsf{LHS.sk}$ and $\mathsf{mpk} = \mathsf{LHS.vk}$.
Each voter i for $i \in [n]$ generates FH-IPFE keys

$$\mathsf{sk}_i := \mathsf{ip.dk}^i_{(1,0)};\ \mathsf{pk}_i := (\mathsf{id}_i, \mathsf{ct}_i, \mathsf{ip.dk}^i_{(0,0)})$$

where $\mathsf{ct}_i = \mathsf{PKE.Enc}(\mathsf{PKE.pk}, \mathsf{ip.dk}^i_{(1,0)})$.
Each mix-server $\mathcal{S}^{\mathsf{mix}}_s$ for $s \in [N_{\mathsf{mix}}]$ generates Sig keys $(\mathsf{SK}^{\mathsf{mix}}_s, \mathsf{VK}^{\mathsf{mix}}_s) \leftarrow \mathsf{Sig.SetUp}(1^\lambda)$.

$\underline{\mathsf{OMIX.Certify}(\mathsf{msk}, (\mathsf{pk}_i)_{i\in[n]}, (\mathsf{VK}^{\mathsf{mix}}_s)_{s\in[N_{\mathsf{mix}}]})}$[a]:
Starting from an identity matrix $\boldsymbol{I}_{n\times n}$, each column vector $\boldsymbol{e}_i$ corresponds to each voter i, initially with 1 at positions (i,i), and 0 in all other entries of the column.
Transparently Arrange: for each $i \in [n]$,

1. $\mathcal{CA}$ parses [b]$C_{i,i} = \mathsf{ct}_i$;
2. for each row $j \neq i$, $\mathcal{CA}$ forms $C_{j,i} \leftarrow \mathsf{PKE.Enc}(\mathsf{PKE.pk}, \mathsf{ip.dk}^i_{(0,0)}; 0)$;
3. $\mathcal{CA}$ forms a PKE randomiser $C_{\mathsf{PKE}} \leftarrow \mathsf{PKE.Enc}(\mathsf{PKE.pk}, \mathbf{0}; 1)$;

Sign with msk

1. For each $j \in [n]$ and each $i \in [n+1]$, $\mathcal{CA}$ generates a row tag and a column tag

$$(\tau_j, \tilde{\tau}_j) \leftarrow \text{2-tag-}\mathsf{LHS.NewTag}_{\mathcal{T}_1}(\mathsf{LHS.sk});\ (\theta_i, \tilde{\theta}_i) \leftarrow \text{2-tag-}\mathsf{LHS.NewTag}_{\mathcal{T}_2}(\mathsf{LHS.sk}).$$

2. For each $(j,i) \in [n] \times [n]$, $\mathcal{CA}$ signs $\bar{C}_{j,i} = ([\![1]\!]_1, C_{j,i})$ and $\bar{C}_{\mathsf{PKE}} = ([\![0]\!]_1, C_{\mathsf{PKE}})$ under the two-dimensional tag (τ_j, θ_i):

$$\sigma_{j,i} \leftarrow \text{2-tag-}\mathsf{LHS.Sign}(\mathsf{LHS.sk}, \tilde{\tau}_j, \tilde{\theta}_i, \bar{C}_{j,i});$$
$$\Sigma_{\mathsf{PKE},j,i} \leftarrow \text{2-tag-}\mathsf{LHS.Sign}(\mathsf{LHS.sk}, \tilde{\tau}_j, \tilde{\theta}_i, \bar{C}_{\mathsf{PKE}}).$$

3. $\mathcal{CA}$ signs each $\boldsymbol{G}_j := (G_0 = H(0), G_{1,j} = H(j))$ under the tag (τ_j, θ_{n+1}):

$$\Sigma_{\boldsymbol{G}_j} \leftarrow \text{2-tag-}\mathsf{LHS.Sign}(\mathsf{LHS.sk}, \tilde{\tau}_j, \tilde{\theta}_{n+1}, \boldsymbol{G}_j)$$

The certification to a voter i is $\mathcal{CA}.\Sigma_i = (C_{j,i}, \sigma_{j,i}, \Sigma_{\mathsf{PKE},j,i})_{j\in[n]}, \theta_i$.
The initial box is $\mathsf{MBox}^{(0)} = \{(\mathcal{CA}.\Sigma_i)_{i\in[n]}, (\tau_j, G_{1,j}, \Sigma_{\boldsymbol{G}_j})_{j\in[n]}, \theta_{n+1}, G_0, C_{\mathsf{PKE}}\}$.

[a] We assume that all servers' verification keys $\mathsf{Sig.vk}_s$ are implicitly certified.
[b] In this protocol, we use the uppercase letter C to denote ElGamal-ciphertext vectors.

Fig. 9. Setup, key generation and certification of OMIX.

$\underline{\mathsf{OMIX.Shuffle}(\mathsf{SK}_s^{\mathsf{mix}}, \mathsf{MBox}^{(s-1)}, (\mathsf{proof}^{(k)}, \mathsf{sig}_{\mathsf{mix}}^{(k)})_{k\in[s-1]})}$:

Each server $\mathcal{S}_s$ takes input $\mathsf{MBox}^{(s-1)}$ as

$$\{(C_{j,i}, \sigma_{j,i}, \Sigma_{\mathsf{PKE},j,i})_{j,i\in[n]}, (\tau_j, G_{1,j}, \Sigma_{\boldsymbol{G}_j})_{j\in[n]}, (\theta_i)_{i\in[n+1]}, G_0, C_{\mathsf{PKE}}\}$$

1. It generates random $\gamma_{\mathsf{PKE},j,i}$ and randomises $C_{j,i}$ for each (j,i):
$$C'_{j,i} = C_{j,i} + \gamma_{\mathsf{PKE},j,i} \cdot C_{\mathsf{PKE}}$$
2. It adapts each signature $\sigma_{j,i}$ into $\sigma'_{j,i}$ for $\bar{C}'_{j,i} = ([\![1]\!]_1, C'_{j,i})$ accordingly
$$\sigma'_{j,i} \leftarrow \text{2-tag-}\mathsf{LHS.DerivSign}(\mathsf{LHS.vk}, (\tau_j, \theta_i), (\gamma_{\mathsf{PKE},j,i}, \bar{C}_{\mathsf{PKE}}, \Sigma_{\mathsf{PKE},j,i}), (1, \bar{C}_{j,i}, \sigma_{j,i})).$$
3. It generates random β and randomises G_0 and $G_{1,j}$ for each $j \in [n]$:
$$G'_0 = \beta \cdot G_0; \qquad G'_{1,j} = \beta \cdot G_{1,j}$$
4. It adapts each signature $\Sigma_{\boldsymbol{G}_j}$ into $\Sigma'_{\boldsymbol{G}_j}$ for $\boldsymbol{G}'_j = (G'_0, G'_{1,j})$:
$$\Sigma'_{\boldsymbol{G}_j} \leftarrow \text{2-tag-}\mathsf{LHS.DerivSign}(\mathsf{LHS.vk}, (\tau_j, \theta_{n+1}), (\beta, \boldsymbol{G}_j, \Sigma_{\boldsymbol{G}_j})).$$
5. It randomises each row tag τ_j into $\tau_j^\star$ and adapts the signatures $\sigma'_{j,i}, \Sigma_{\mathsf{PKE},j,i}, \Sigma'_{\boldsymbol{G}_j}$ accordingly
$$\begin{aligned}(\tau_j^\star, \bar{C}'_{j,i}, \sigma'^\star_{j,i}) &\leftarrow \text{2-tag-}\mathsf{LHS.RandTag}(\mathsf{LHS.vk}, \tau_j, \bar{C}'_{j,i}, \sigma'_{j,i})\\ (\tau_j^\star, \bar{C}_{\mathsf{PKE}}, \Sigma^\star_{\mathsf{PKE},j,i}) &\leftarrow \text{2-tag-}\mathsf{LHS.RandTag}(\mathsf{LHS.vk}, \tau_j, \bar{C}_{\mathsf{PKE}}, \Sigma_{\mathsf{PKE},j,i})\\ (\tau_j^\star, \boldsymbol{G}'_j, \Sigma'^\star_{\boldsymbol{G}_j}) &\leftarrow \text{2-tag-}\mathsf{LHS.RandTag}(\mathsf{LHS.vk}, \tau_j, \boldsymbol{G}'_j, \Sigma'_{\boldsymbol{G}_j})\end{aligned}$$
6. It samples a random permutation $\Pi : [n] \to [n]$, and assigns
$$\begin{aligned}&C^{(s)}_{\Pi(j),i} = C'_{j,i},\ \sigma^{(s)}_{\Pi(j),i} = \sigma'^\star_{j,i},\ \Sigma^{(s)}_{\mathsf{PKE},\Pi(j),i} = \Sigma^\star_{\mathsf{PKE},j,i},\ \tau^{(s)}_{\Pi(j)} = \tau_j^\star;\\ &G^{(s)}_{1,\Pi(j)} = G'_{1,j};\ \Sigma^{(s)}_{\boldsymbol{G}_{\Pi}(j)} = \Sigma'^\star_{\boldsymbol{G}_j};\ G^{(s)}_0 = G'_0\end{aligned}$$
7. It computes $\mathsf{proof}^{(s)} \leftarrow \mathsf{NIZK}_{\mathsf{DH}}.\mathsf{Prove}\left(G_0, G_0^{(s)}, \sum_{j\in[n]} G_{1,j}, \sum_{j\in[n]} G^{(s)}_{1,j}; \beta\right)$
8. It signs the proofs as $\mathsf{sig}^{(s)}_{\mathsf{mix}} \leftarrow \mathsf{Sig.Sign}(\mathsf{SK}_s^{\mathsf{mix}}, \mathsf{proof}^{(s)})$.

The output box is

$$\mathsf{MBox}^{(s)} = \{(C^{(s)}_{j,i}, \sigma^{(s)}_{j,i}, \Sigma^{(s)}_{\mathsf{PKE},j,i})_{j,i\in[n]}, (\tau_j^{(s)}, G^{(s)}_{1,j}, \Sigma^{(s)}_{\boldsymbol{G}_j})_{j\in[n]}, (\theta_i)_{i\in[n+1]}, G^{(s)}_0, C_{\mathsf{PKE}}\}$$

and $\mathsf{proof}^{(s)}, \mathsf{sig}^{(s)}_{\mathsf{mix}}$.

Fig. 10. Shuffle and re-randomisation of OMIX.

$\mathsf{OMIX.VerifShuffle}(\mathsf{MBox}^{(0)}, \mathsf{MBox}^{(N_{\mathsf{mix}})}, (\mathsf{proof}^{(s)}, \mathsf{sig}^{(s)}_{\mathsf{mix}}, \mathsf{VK}^{\mathsf{mix}}_s)_{s\in[N_{\mathsf{mix}}]})$:

The verification continues unless the inputs $\mathsf{MBox}^{(0)}, \mathsf{MBox}^{(N_{\mathsf{mix}})}$ can not be parsed in the following form

$$\mathsf{MBox}^{(0)} = \{(C^{(0)}_{j,i}, \sigma^{(0)}_{j,i}, \Sigma^{(0)}_{\mathsf{PKE},j,i})_{j,i\in[n]}, (\tau^{(0)}_j, G^{(0)}_{1,j}, \Sigma^{(0)}_{\boldsymbol{G}_j})_{j\in[n]}, (\theta_i)_{i\in[n+1]}, G^{(0)}_0, C_{\mathsf{PKE}}\};$$

$$\mathsf{MBox}^{(N_{\mathsf{mix}})} = \{(C^{(N_{\mathsf{mix}})}_{j,i}, \sigma^{(N_{\mathsf{mix}})}_{j,i}, \Sigma^{(N_{\mathsf{mix}})}_{\mathsf{PKE},j,i})_{j,i\in[n]}, (\tau^{(N_{\mathsf{mix}})}_j, G^{(N_{\mathsf{mix}})}_{1,j}, \Sigma^{(N_{\mathsf{mix}})}_{\boldsymbol{G}_j})_{j\in[n]},$$
$$(\theta_i)_{i\in[n+1]}, G^{(N_{\mathsf{mix}})}_0, C_{\mathsf{PKE}}\}.$$

The verification outputs 1 for validity if:

- $\mathsf{Sig.Verify}(\mathsf{VK}^{\mathsf{mix}}_s, \mathsf{proof}^{(s)}, \mathsf{sig}^{(s)}_{\mathsf{mix}}) = 1$ for each $s \in [N_{\mathsf{mix}}]$[a];
- $\mathsf{NIZK}_{\mathsf{DH}}.\mathsf{Verify}(\mathsf{proof}^{(s)}) = 1$ for each $s \in [N_{\mathsf{mix}}]$[b];
- For $N \in \{0, N_{\mathsf{mix}}\}$:
  - for each (j,i), parse $\bar{C}^{(N)}_{j,i} = ([\![1]\!]_1, C^{(N)}_{j,i})$ then
  $$2\text{-tag-}\mathsf{LHS.Verify}(\mathsf{LHS.vk}, (\tau^{(N)}_j, \theta_i), \bar{C}^{(N)}_{j,i}, \sigma^{(N)}_{j,i}) = 1$$
  - for each j, parse $\boldsymbol{G}^{(N)}_j = (G^{(N)}_0, G^{(N)}_{1,j})$, then
  $$2\text{-tag-}\mathsf{LHS.Verify}(\mathsf{LHS.vk}, (\tau^{(N)}_j, \theta_{n+1}), \boldsymbol{G}^{(N)}_j, \Sigma^{(N)}_{\boldsymbol{G}_j}) = 1$$

[a] These N_{mix} signatures are batched by using in parallel a multi-signature.
[b] These N_{mix} proofs are batched by using in parallel a malleable NIZK system.

Fig. 11. Verification of OMIX.

6. Bayer, S., Groth, J.: Efficient zero-knowledge argument for correctness of a shuffle. In: Pointcheval, D., Johansson, T. (eds.) Advances in Cryptology – EUROCRYPT 2012. Lecture Notes in Computer Science, vol. 7237, pp. 263–280. Springer, Heidelberg, Germany, Cambridge, UK (Apr 15–19, 2012). https://doi.org/10.1007/978-3-642-29011-4_17
7. Bellare, M., Rogaway, P.: Random oracles are practical: A paradigm for designing efficient protocols. In: Denning, D.E., Pyle, R., Ganesan, R., Sandhu, R.S., Ashby, V. (eds.) ACM CCS 93: 1st Conference on Computer and Communications Security. pp. 62–73. ACM Press, Fairfax, Virginia, USA (Nov 3–5, 1993). https://doi.org/10.1145/168588.168596
8. Benaloh, J.: Verifiable Secret-Ballot Elections. Ph.D. thesis (September 1987), https://www.microsoft.com/en-us/research/publication/verifiable-secret-ballot-elections/
9. Benaloh, J.: Simple verifiable elections. In: Proceedings of the USENIX/Accurate Electronic Voting Technology Workshop 2006 on Electronic Voting Technology Workshop. p. 5. EVT'06, USENIX Association, USA (2006)
10. Bernhard, D., Pereira, O., Warinschi, B.: On necessary and sufficient conditions for private ballot submission. Cryptology ePrint Archive, Paper 2012/236 (2012), https://eprint.iacr.org/2012/236
11. Bernhard, D., Cortier, V., Galindo, D., Pereira, O., Warinschi, B.: SoK: A comprehensive analysis of game-based ballot privacy definitions. In: 2015 IEEE Symposium on Security and Privacy. pp. 499–516. IEEE Computer Society Press, San Jose, CA, USA (May 17–21, 2015). https://doi.org/10.1109/SP.2015.37
12. Bishop, A., Jain, A., Kowalczyk, L.: Function-hiding inner product encryption. In: Iwata, T., Cheon, J.H. (eds.) Advances in Cryptology – ASIACRYPT 2015, Part I. Lecture Notes in Computer Science, vol. 9452, pp. 470–491. Springer, Heidelberg, Germany, Auckland, New Zealand (Nov 30 – Dec 3, 2015). https://doi.org/10.1007/978-3-662-48797-6_20
13. Boneh, D., Drijvers, M., Neven, G.: Compact multi-signatures for smaller blockchains. In: Peyrin, T., Galbraith, S. (eds.) Advances in Cryptology – ASIACRYPT 2018, Part II. Lecture Notes in Computer Science, vol. 11273, pp. 435–464. Springer, Heidelberg, Germany, Brisbane, Queensland, Australia (Dec 2–6, 2018). https://doi.org/10.1007/978-3-030-03329-3_15
14. Boneh, D., Sahai, A., Waters, B.: Functional encryption: Definitions and challenges. In: Ishai, Y. (ed.) TCC 2011: 8th Theory of Cryptography Conference. Lecture Notes in Computer Science, vol. 6597, pp. 253–273. Springer, Heidelberg, Germany, Providence, RI, USA (Mar 28–30, 2011). https://doi.org/10.1007/978-3-642-19571-6_16
15. Chaidos, P., Cortier, V., Fuchsbauer, G., Galindo, D.: BeleniosRF: A non-interactive receipt-free electronic voting scheme. In: Weippl, E.R., Katzenbeisser, S., Kruegel, C., Myers, A.C., Halevi, S. (eds.) ACM CCS 2016: 23rd Conference on Computer and Communications Security. pp. 1614–1625. ACM Press, Vienna, Austria (Oct 24–28, 2016). https://doi.org/10.1145/2976749.2978337

16. Chaum, D.L.: Untraceable electronic mail, return addresses, and digital pseudonyms. Commun. ACM **24**(2), 84–90 (Feb 1981). https://doi.org/10.1145/358549.358563, https://doi.org/10.1145/358549.358563
17. Clarkson, M.R., Chong, S., Myers, A.C.: Civitas: Toward a secure voting system. In: 2008 IEEE Symposium on Security and Privacy. pp. 354–368. IEEE Computer Society Press, Oakland, CA, USA (May 18–21, 2008). https://doi.org/10.1109/SP.2008.32
18. Cohen, J.D., Fischer, M.J.: A robust and verifiable cryptographically secure election scheme (extended abstract). In: 26th Annual Symposium on Foundations of Computer Science. pp. 372–382. IEEE Computer Society Press, Portland, Oregon (Oct 21–23, 1985). https://doi.org/10.1109/SFCS.1985.2
19. Cortier, V., Filipiak, A., Lallemand, J.: BeleniosVS: Secrecy and verifiability against a corrupted voting device. In: Delaune, S., Jia, L. (eds.) CSF 2019: IEEE 32nd Computer Security Foundations Symposium. pp. 367–381. IEEE Computer Society Press, Hoboken, NJ, USA (Jun 25–28, 2019). https://doi.org/10.1109/CSF.2019.00032
20. Cortier, V., Gaudry, P., Glondu, S.: Belenios: A Simple Private and Verifiable Electronic Voting System, pp. 214–238. Springer International Publishing, Cham (2019). https://doi.org/10.1007/978-3-030-19052-1_14, https://doi.org/10.1007/978-3-030-19052-1_14
21. Cortier, V., Smyth, B.: Attacking and fixing helios: An analysis of ballot secrecy. In: Backes, M., Zdancewic, S. (eds.) CSF 2011: IEEE 24th Computer Security Foundations Symposium. pp. 297–311. IEEE Computer Society Press, Domaine de l'Abbaye des Vaux de Cernay, France (Jun 27–29, 2011). https://doi.org/10.1109/CSF.2011.27
22. ElSheikh, M., Youssef, A.M.: Dispute-free scalable open vote network using zk-SNARKs. Cryptology ePrint Archive, Paper 2022/310 (2022), https://eprint.iacr.org/2022/310
23. ElSheikh, M., Youssef, A.M., Hasan, M.A.: Self-tallying e-voting using homomorphic time-lock puzzles and zk-snarks. IEEE Transactions on Network Science and Engineering **12**(4), 2566–2581 (2025). https://doi.org/10.1109/TNSE.2025.3550290
24. Faonio, A., Russo, L.: Mix-nets from re-randomizable and replayable cca-secure public-key encryption. In: Galdi, C., Jarecki, S. (eds.) Security and Cryptography for Networks. pp. 172–196. Springer International Publishing, Cham (2022)
25. Fauzi, P., Lipmaa, H., Siim, J., Zając, M.: An efficient pairing-based shuffle argument. In: Takagi, T., Peyrin, T. (eds.) Advances in Cryptology – ASIACRYPT 2017. pp. 97–127. Springer International Publishing, Cham (2017)
26. Fuchsbauer, G., Hanser, C., Slamanig, D.: Structure-preserving signatures on equivalence classes and constant-size anonymous credentials. Journal of Cryptology **32**(2), 498–546 (Apr 2019). https://doi.org/10.1007/s00145-018-9281-4
27. Furukawa, J., Sako, K.: An efficient scheme for proving a shuffle. In: Kilian, J. (ed.) Advances in Cryptology – CRYPTO 2001. Lecture Notes in Computer Science, vol. 2139, pp. 368–387. Springer, Heidelberg, Germany, Santa Barbara, CA, USA (Aug 19–23, 2001). https://doi.org/10.1007/3-540-44647-8_22
28. Groth, J.: Efficient maximal privacy in boardroom voting and anonymous broadcast. In: Juels, A. (ed.) FC 2004: 8th International Conference on Financial Cryptography. Lecture Notes in Computer Science, vol. 3110, pp. 90–104. Springer, Heidelberg, Germany, Key West, USA (Feb 9–12, 2004)
29. Groth, J.: Evaluating security of voting schemes in the universal composability framework. In: Jakobsson, M., Yung, M., Zhou, J. (eds.) ACNS 04: 2nd International Conference on Applied Cryptography and Network Security. Lecture Notes in Computer Science, vol. 3089, pp. 46–60. Springer, Heidelberg, Germany, Yellow Mountain, China (Jun 8–11, 2004). https://doi.org/10.1007/978-3-540-24852-1_4
30. Groth, J., Ishai, Y.: Sub-linear zero-knowledge argument for correctness of a shuffle. In: Smart, N.P. (ed.) Advances in Cryptology – EUROCRYPT 2008. Lecture Notes in Computer Science, vol. 4965, pp. 379–396. Springer, Heidelberg, Germany, Istanbul, Turkey (Apr 13–17, 2008). https://doi.org/10.1007/978-3-540-78967-3_22
31. Groth, J., Sahai, A.: Efficient non-interactive proof systems for bilinear groups. In: Smart, N.P. (ed.) Advances in Cryptology – EUROCRYPT 2008. Lecture Notes in Computer Science, vol. 4965, pp. 415–432. Springer, Heidelberg, Germany, Istanbul, Turkey (Apr 13–17, 2008). https://doi.org/10.1007/978-3-540-78967-3_24
32. Haines, T., MUELLER, J., MOSAHEB, R., PRYVALOV, I.: Sok: Secure e-voting with everlasting privacy. In: Proceedings on Privacy Enhancing Technologies (PoPETs) (2023)
33. Haines, T., Müller, J.: SoK: Techniques for verifiable mix nets. In: Jia, L., Küsters, R. (eds.) CSF 2020: IEEE 33rd Computer Security Foundations Symposium. pp. 49–64. IEEE Computer Society Press, Boston, MA, USA (Jun 22–26, 2020). https://doi.org/10.1109/CSF49147.2020.00012
34. Hao, F., Ryan, P., Zielinski, P.: Anonymous voting by two-round public discussion. Information Security, IET **4**, 62 – 67 (07 2010). https://doi.org/10.1049/iet-ifs.2008.0127
35. Hébant, C., Phan, D.H., Pointcheval, D.: Linearly-homomorphic signatures and scalable mix-nets. In: Kiayias, A., Kohlweiss, M., Wallden, P., Zikas, V. (eds.) PKC 2020: 23rd International Conference on Theory and Practice of Public Key Cryptography, Part II. Lecture Notes in Computer Science, vol.

12111, pp. 597–627. Springer, Heidelberg, Germany, Edinburgh, UK (May 4–7, 2020). https://doi.org/10.1007/978-3-030-45388-6_21
36. Juels, A., Catalano, D., Jakobsson, M.: Coercion-resistant electronic elections. In: Proceedings of the 2005 ACM Workshop on Privacy in the Electronic Society. p. 61–70. WPES '05, Association for Computing Machinery, New York, NY, USA (2005). https://doi.org/10.1145/1102199.1102213, https://doi.org/10.1145/1102199.1102213
37. Khader, D., Smyth, B., Ryan, P., Hao, F.: A fair and robust voting system by broadcast. Lecture Notes in Informatics (LNI), Proceedings - Series of the Gesellschaft fur Informatik (GI) **205** (01 2012)
38. Kiayias, A., Yung, M.: Self-tallying elections and perfect ballot secrecy. In: Naccache, D., Paillier, P. (eds.) PKC 2002: 5th International Workshop on Theory and Practice in Public Key Cryptography. Lecture Notes in Computer Science, vol. 2274, pp. 141–158. Springer, Heidelberg, Germany, Paris, France (Feb 12–14, 2002). https://doi.org/10.1007/3-540-45664-3_10
39. Li, Y., Susilo, W., Yang, G., Yu, Y., Liu, D., Du, X., Guizani, M.: A blockchain-based self-tallying voting protocol in decentralized iot. IEEE Transactions on Dependable and Secure Computing **19**(1), 119–130 (2022). https://doi.org/10.1109/TDSC.2020.2979856
40. Lin, H.: Indistinguishability obfuscation from SXDH on 5-linear maps and locality-5 PRGs. In: Katz, J., Shacham, H. (eds.) Advances in Cryptology – CRYPTO 2017, Part I. Lecture Notes in Computer Science, vol. 10401, pp. 599–629. Springer, Heidelberg, Germany, Santa Barbara, CA, USA (Aug 20–24, 2017). https://doi.org/10.1007/978-3-319-63688-7_20
41. McCorry, P., Shahandashti, S.F., Hao, F.: A smart contract for boardroom voting with maximum voter privacy. In: Kiayias, A. (ed.) FC 2017: 21st International Conference on Financial Cryptography and Data Security. Lecture Notes in Computer Science, vol. 10322, pp. 357–375. Springer, Heidelberg, Germany, Sliema, Malta (Apr 3–7, 2017)
42. O'Neill, A.: Definitional issues in functional encryption. Cryptology ePrint Archive, Report 2010/556 (2010), https://eprint.iacr.org/2010/556
43. Panja, S., Bag, S., Hao, F., Roy, B.: A smart contract system for decentralized borda count voting. IEEE Transactions on Engineering Management **67**(4), 1323–1339 (2020). https://doi.org/10.1109/TEM.2020.2986371
44. Parampalli, U., Ramchen, K., Teague, V.: Efficiently shuffling in public. In: Fischlin, M., Buchmann, J., Manulis, M. (eds.) PKC 2012: 15th International Conference on Theory and Practice of Public Key Cryptography. Lecture Notes in Computer Science, vol. 7293, pp. 431–448. Springer, Heidelberg, Germany, Darmstadt, Germany (May 21–23, 2012). https://doi.org/10.1007/978-3-642-30057-8_26
45. Pointcheval, D.: Efficient universally-verifiable electronic voting with everlasting privacy. In: Galdi, C., Phan, D.H. (eds.) Security and Cryptography for Networks. pp. 323–344. Springer Nature Switzerland, Cham (2024)
46. Sahai, A., Waters, B.R.: Fuzzy identity-based encryption. In: Cramer, R. (ed.) Advances in Cryptology – EUROCRYPT 2005. Lecture Notes in Computer Science, vol. 3494, pp. 457–473. Springer, Heidelberg, Germany, Aarhus, Denmark (May 22–26, 2005). https://doi.org/10.1007/11426639_27
47. Sako, K., Kilian, J.: Receipt-free mix-type voting scheme - a practical solution to the implementation of a voting booth. In: Guillou, L.C., Quisquater, J.J. (eds.) Advances in Cryptology – EUROCRYPT'95. Lecture Notes in Computer Science, vol. 921, pp. 393–403. Springer, Heidelberg, Germany, Saint-Malo, France (May 21–25, 1995). https://doi.org/10.1007/3-540-49264-X_32
48. Shi, E., Wu, K.: Non-interactive anonymous router. In: Canteaut, A., Standaert, F.X. (eds.) Advances in Cryptology – EUROCRYPT 2021, Part III. Lecture Notes in Computer Science, vol. 12698, pp. 489–520. Springer, Heidelberg, Germany, Zagreb, Croatia (Oct 17–21, 2021). https://doi.org/10.1007/978-3-030-77883-5_17
49. Terelius, B., Wikström, D.: Proofs of restricted shuffles. In: Bernstein, D.J., Lange, T. (eds.) AFRICACRYPT 10: 3rd International Conference on Cryptology in Africa. Lecture Notes in Computer Science, vol. 6055, pp. 100–113. Springer, Heidelberg, Germany, Stellenbosch, South Africa (May 3–6, 2010)
50. The State of Geneva: Chvote. https://github.com/republique-et-canton-de-geneve/chvote-protocol-poc (2018)
51. Tomida, J., Abe, M., Okamoto, T.: Efficient functional encryption for inner-product values with full-hiding security. In: Bishop, M., Nascimento, A.C.A. (eds.) ISC 2016: 19th International Conference on Information Security. Lecture Notes in Computer Science, vol. 9866, pp. 408–425. Springer, Heidelberg, Germany, Honolulu, HI, USA (Sep 3–6, 2016). https://doi.org/10.1007/978-3-319-45871-7_24
52. Tomida, J., Abe, M., Okamoto, T.: Efficient functional encryption for inner-product values with full-hiding security. In: Bishop, M., Nascimento, A.C.A. (eds.) Information Security. pp. 408–425. Springer International Publishing, Cham (2016)
53. Wikström, D.: A commitment-consistent proof of a shuffle. In: Boyd, C., Nieto, J.M.G. (eds.) ACISP 09: 14th Australasian Conference on Information Security and Privacy. Lecture Notes in Computer Science, vol. 5594, pp. 407–421. Springer, Heidelberg, Germany, Brisbane, Australia (Jul 1–3, 2009)

54. Wikström, D.: Verificatum. https://github.com/verificatum/verificatum-vcr (2018)
55. Yang, Y., Guan, Z., Wan, Z., Weng, J., Pang, H.H., Deng, R.H.: Priscore: Blockchain-based self-tallying election system supporting score voting. IEEE Transactions on Information Forensics and Security **16**, 4705–4720 (2021). https://doi.org/10.1109/TIFS.2021.3108494
56. Zhou, Z., Zhang, Z., Hao, F., Zheng, B., Masyhur, Z.: QV-net: Decentralized self-tallying quadratic voting with maximal ballot secrecy. Cryptology ePrint Archive, Paper 2025/1146 (2025). https://doi.org/10.1145/3719027.3744810, https://eprint.iacr.org/2025/1146

A Notation and Standard Primitives

A.1 Notation

Sets. Given any $n \in \mathbb{N}$, we denote by $[n]$ the set of integers $\{1, \ldots, n\}$. We denote by $|\mathcal{S}|$ the cardinal of any finite set $\mathcal{S}$. For any vector $\boldsymbol{a} \in \mathbb{A}^n$, we use a subscript i to denote by a_i the i-th component of $\boldsymbol{a}$. Similarly, for any matrix $\boldsymbol{A} \in \mathbb{Z}_p^{m \times n}$, we use subscripts (i, j) to denote by $A_{i,j}$ the component at the i-th row and the j-column of $\boldsymbol{A}$. The identity matrix is denoted by $\boldsymbol{I}$.

Permutations. Let $\Pi : [n] \to [n]$ be a bijective map. For every $\boldsymbol{a} \in \mathbb{A}^n$, we let $\mathsf{PerVec}_\Pi(\boldsymbol{a})$ be the vector generated by permuting n entries of $\boldsymbol{a}$ with respect to Π, i.e. if $\boldsymbol{a'} = \mathsf{PerVec}_\Pi(\boldsymbol{a})$ then $a'_{\Pi(i)} = a_i$ for all $i \in [n]$. We also denote by $\mathsf{PerMatRow}_\Pi(\boldsymbol{I})$ the matrix generated by permuting n rows of an identity matrix $\boldsymbol{I}$ with respect to Π so that $\boldsymbol{a'} = \mathsf{PerMatRow}_\Pi(\boldsymbol{I}) \cdot \boldsymbol{a}$. Columns of $\mathsf{PerMatRow}_\Pi(\boldsymbol{I})$ can be parsed as $(\boldsymbol{p}^i)_{i \in [n]}$ where $p^i_j = 1$ if $j = \Pi(i)$ and $p^i_j = 0$ otherwise.

A.2 Groups

Groups. Let GGen be a prime-order group generator, a probabilistic polynomial time (PPT) algorithm that on input the security parameter 1^λ returns a description $\mathcal{G} = (\mathbb{G}, p, P)$ of an additive cyclic group $\mathbb{G}$ of order a 2λ-bit prime p, whose generator is P:

- for $a \in \mathbb{Z}_p$, we define $[\![a]\!] = a \cdot P \in \mathbb{G}$ as the *implicit representation* of a in $\mathbb{G}$;
- the group $\mathbb{Z}_p$ is equipped with element addition $+_{\mathbb{Z}_p}$ and scalar multiplication $\cdot_{\mathbb{Z}_p}$. Similarly, $\mathbb{G}$ is equipped with element addition $+_{\mathbb{G}}$ and scalar multiplication $\cdot_{\mathbb{G}}$. Since the context makes it clear whether the operations occur in $\mathbb{G}$ or $\mathbb{Z}_p$, the subscripts for these operations are omitted;
- scalar products between vectors $\boldsymbol{a} \in \mathbb{Z}_p^\rho$ and vectors $\boldsymbol{b} \in \mathbb{Z}_p^\rho$ (or between $\boldsymbol{a} \in \mathbb{Z}_p^\rho$ and $[\![\boldsymbol{b}]\!] \in \mathbb{G}^\rho$) are denoted by $\langle \boldsymbol{a}, \boldsymbol{b} \rangle$ (or $\langle \boldsymbol{a}, [\![\boldsymbol{b}]\!] \rangle$, respectively).

Given $[\![a]\!], [\![b]\!] \in \mathbb{G}$ and a scalar $x \in \mathbb{Z}_p$, one can efficiently compute $[\![a \cdot x]\!] \in \mathbb{G}$ and $[\![a + b]\!] = [\![a]\!] + [\![b]\!] \in \mathbb{G}$. We rely on the following standard assumptions.

Definition 13 (Discrete Logarithm (DL) Assumption). *In a group $\mathbb{G}$ of prime order p, it states that for any random element $[\![x]\!]$, it is computationally hard to recover x.*

Definition 14 (Decisional Diffie-Hellman (DDH) Assumption). *In a group $\mathbb{G}$ of prime order p, it states that for any random elements $[\![x]\!]$ and $[\![y]\!]$, it is computationally hard to distinguish between $[\![x \cdot y]\!]$ and a random element $[\![z]\!]$.*

Pairing Groups. Let PGGen be a pairing group generator, a PPT algorithm that on input the security parameter 1^λ returns a description $\mathcal{PG} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, P_1, P_2, e)$ of asymmetric pairing groups where $\mathbb{G}_1$, $\mathbb{G}_2$, $\mathbb{G}_T$ are additive cyclic groups of order p for a 2λ-bit prime p, P_1 and P_2 are generators of $\mathbb{G}_1$ and $\mathbb{G}_2$, respectively, and $e : \mathbb{G}_1 \times \mathbb{G}_2 \to \mathbb{G}_T$ is an efficiently computable (non-degenerate) bilinear map. For $s \in \{1, 2, T\}$ and $a \in \mathbb{Z}_p$, we define $[\![a]\!]_s = aP_s \in \mathbb{G}_s$ as the implicit representation of a in $\mathbb{G}_s$. Given $[\![a]\!]_1$, $[\![b]\!]_2$, one can efficiently compute $[\![ab]\!]_T$ using the pairing e. The DDH assumption can be extended to $\mathcal{PG}$ as follows.

Definition 15 (Symmetric eXternal Diffie-Hellman (SXDH) Assumption). *In a pairing group $\mathcal{PG} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, P_1, P_2, e)$ of prime order p, it states that the DDH assumption holds in both $\mathbb{G}_1$ and $\mathbb{G}_2$.*

A.3 Public-Key Encryption

We recall the notion of public-key encryption, along with relevant properties.

Definition 16 (PKE). *A public-key encryption over a group $\mathcal{G} = (\mathbb{G}, p)$ consists of four algorithms:*

- $\mathsf{SetUp}(1^\lambda, \rho)$*: On input a security parameter* 1^λ *and a dimension* ρ*, it outputs a public key* pk *and a secret key* sk*.*
- $\mathsf{Enc}(\mathsf{pk}, [\![\boldsymbol{m}]\!])$*: On input a public key* pk *and a message vector* $[\![\boldsymbol{m}]\!] \in \mathbb{G}^\rho$ *, it outputs a ciphertext* ct*.*
- $\mathsf{Dec}(\mathsf{sk}, \mathsf{ct})$*: On input a secret key* sk *and a ciphertext* ct*, it outputs an element in* $\mathbb{G}^\rho$*.*

Correctness For all parameters $\lambda, \rho \in \mathbb{N}$, and for key pairs $(\mathsf{sk}, \mathsf{pk})$ issued by $\mathsf{SetUp}(1^\lambda, \rho)$, and for any message $[\![\boldsymbol{m}]\!] \in \mathbb{G}^\rho$, if $\mathsf{ct} = \mathsf{Enc}(\mathsf{pk}, [\![\boldsymbol{m}]\!])$, then the decryption satisfies $\mathsf{Dec}(\mathsf{sk}, \mathsf{ct}) = [\![\boldsymbol{m}]\!]$ with overwhelming probability, where the probability is taken over the randomness of all algorithms.

Definition 17 (Security). *Let* PKE *be a public-key encryption scheme, we define the experiment* $\mathrm{IND}^{\mathsf{PKE}}$ *in Figure 12, where the oracles are defined as:*

$\underline{\mathrm{IND}^{\mathsf{PKE}}(\lambda, \mathcal{A})}$:
$b \xleftarrow{\$} \{0,1\}$, $(\mathsf{pk}, \mathsf{sk}) \leftarrow \mathsf{SetUp}(1^\lambda, \rho)$
$\alpha \leftarrow \mathcal{A}^{\mathsf{QEnc}_b(\cdot,\cdot)}(\mathsf{pk})$
Output: $\beta = 1$ if $\alpha = b$, and $\beta = 0$ otherwise.

Fig. 12. Security experiment for PKE

- ***Encryption oracle*** $\mathsf{QEnc}_b([\![m^0]\!], [\![m^1]\!])$*: it returns* $\mathsf{ct} \leftarrow \mathsf{Enc}(\mathsf{pk}, [\![m^b]\!])$*.*

The PKE *scheme is said to be* IND-CPA*-secure if given any parameters* $\lambda, \rho \in \mathbb{N}$*, for every PPT adversary* $\mathcal{A}$*, the advantage of* $\mathcal{A}$*, defined as* $\mathsf{Adv}^{\mathrm{IND}^{\mathsf{PKE}}}(\mathcal{A}) := \left|\Pr[\beta = 1] - \frac{1}{2}\right|$ *is negligible in* λ*.*

Definition 18 (Homomorphic Ciphertext). *A* PKE *scheme supports homomorphic ciphertexts if, given any two encryptions* $\mathsf{ct}_1 = \mathsf{Enc}(\mathsf{pk}, [\![\boldsymbol{m}_1]\!]; r_1)$ *and* $\mathsf{ct}_2 = \mathsf{Enc}(\mathsf{pk}, [\![\boldsymbol{m}_2]\!]; r_2)$ *(in the form of vectors in* $\mathbb{G}$*) under randomness* r_1 *and* r_2 *(in the form of elements in* $\mathbb{Z}_p$*) respectively, for any scalars* $\alpha_1, \alpha_2 \in \mathbb{Z}_p$*, the following holds:*

$$\alpha_1 \cdot \mathsf{ct}_1 + \alpha_2 \cdot \mathsf{ct}_2 = \mathsf{Enc}(\mathsf{pk}, [\![\alpha_1 \cdot \boldsymbol{m}_1 + \alpha_2 \cdot \boldsymbol{m}_2]\!]; \alpha_1 \cdot r_1 + \alpha_2 \cdot r_2)$$

A.4 Non-Interactive Zero-Knowledge Proofs

Let $R(\cdot;\cdot)$ be a relation that is computable in polynomial time with respect to the size of its first input. Associated with R, we define the NP language $L_R = \{x \mid \exists w \text{ such that } R(x;w) = \mathsf{true}\}$. A value w is called a *witness* for the statement x if $R(x;w) = \mathsf{true}$, which we denote as $(x;w) \in R$.

Definition 19 (NIZK). *A non-interactive proof system for a relation* R *with setup consists of four algorithms:*

- $\mathsf{SetUp}(1^\lambda)$*: On input security parameter* λ*, it outputs a common reference string* σ*;*
- $\mathsf{Prove}(\sigma, x; w)$*: On input a common reference string* σ *and a pair of statement and witness* $(x; w)$*, it outputs a proof* π*;*
- $\mathsf{Verify}(\sigma, x, \pi)$*: On input a common reference string* σ *and a pair of statement and proof* (x, π)*, it outputs* 1 *for accepting and* 0 *for rejecting;*

These algorithms satisfy the following properties:

- *Completeness. For all* $\sigma \leftarrow \mathsf{SetUp}(1^\lambda)$ *and* $(x; w) \in R$*, then the probability that* $\mathsf{Verify}(\sigma, x, \pi) = 1$ *for* $\pi \leftarrow \mathsf{Prove}(\sigma, x; w)$ *is* 1*.*

- *Soundness. For all PPT adversary $\mathcal{A}$, and for $\sigma \leftarrow \mathsf{SetUp}(1^\lambda)$, the probability that $\mathcal{A}(\sigma)$ outputs (x, π) such that $x \notin L$ but $\mathsf{Verify}(\sigma, x, \pi) = 1$, is negligible.*
- *Zero-Knowledge. There exists a polynomial-time simulator algorithm $S = (S_1, S_2)$ such that $S_1(1^\lambda)$ outputs (σ_{sim}, τ_s) and $S_2(\sigma_{sim}, \tau_s, x)$ outputs a value π_s such that for all $(x; w) \in R$ and PPT adversaries $\mathcal{A}$, the following two interactions are indistinguishable: in the first, we compute $\sigma \leftarrow \mathsf{SetUp}(1^\lambda)$ and give σ to $\mathcal{A}$ and oracle access to $\mathsf{Prove}(\sigma, \cdot; \cdot)$ (where Prove outputs $\perp$ on input $(x; w)$ such that $(x; w) \notin R$); in the second, we compute (σ_{sim}, τ_s) and give σ_{sim} to $\mathcal{A}$ and oracle access to $S(\sigma_{sim}, \tau_s, \cdot, \cdot)$ where, on input $(x; w)$, S outputs $S_2(\sigma_{sim}, \tau_s, x)$ if $(x; w) \in R$ and $\perp$ otherwise. Perfect zero-knowledge is achieved if for all $(x; w) \in R$, these interactions are distributed identically.*

When the common reference string and the statement are clear in the context, we assume that they are contained in the proof.

(Non-interactive) Σ-protocols. A Σ-protocol is a special type of 3-move proof system. Given a common reference string σ containing a relation R, a statement x and a witness w such that $R(x; w) = \mathsf{true}$. The protocol processes as follows: The prover sends an initial message a, receives a challenge e and produces an answer z. The verifier can now evaluate (σ, x, a, e, z) and decide whether to accept or reject the proof.

Given access oracle $\mathcal{RO}$ we can transform a Σ-protocol into a non-interactive proof system. To get the challenge e we form the initial message a, query with $\mathcal{RO}$ with (x, a, aux)[10] to get the challenge e and then compute the answer z. The proof is then $\pi = (a, z, \mathsf{aux})$. To verify such a proof, one queries $\mathcal{RO}$ with (x, a, aux) to get e and then run the verifier from the Σ-protocol.

The random oracle model is an idealization of the Fiat-Shamir heuristic [7]. In practice, a prover uses a cryptographic hash-function $\mathcal{H}$ to produce challenge as $e = \mathcal{H}(x, a, \mathsf{aux})$.

B More on Voting Definitions

B.1 Other Voting Definitions

In practice, voting systems usually involve multiple authorities that are responsible for various tasks. We specify the following entities:

1. *Election administrator*, denoted by $\mathcal{E}$, sets up the election by publishing the universe $\mathcal{U}$ of voters, the list of candidates and the result function F of the election.
2. *(Public) Registrar*, denoted by $\mathcal{R}$, issues voting eligibility to voters upon their public keys. Everybody should be able to check the validity of each public key. The registrar publishes and updates the list of eligible voters from the universe $\mathcal{U}$, along with their public keys.
3. *Trustee*, denoted by $\mathcal{T}$, generates master secret/public keys for the election (in possibly a thresholdized manner), processes the public keys of eligible voters to generate a *helper public key*. In our context, trustees include servers that execute mixing and decrypting.
4. A set of *Voters*, identified by id in a universe $\mathcal{U}$, who each generate and send their public key to $\mathcal{R}$ for the registration of voting eligibility. The private encryption key is generated and stored by each voter itself during the computation of the public key.
5. *(Public) Ballot-box Manager*, denoted by $\mathcal{B}$, processes and stores valid ballots in the public ballot box BB. By handling ballot submission, it also manages the list of dropout voters.
6. *(Public) Tallier*, denoted by $\mathcal{D}$, computes the final result based on BB and possibly auxiliary public components such as correcting keys.

The correctness of a self-tallying election is stated as follows.

Definition 20 (Correctness). *A voting scheme $\mathcal{V}ot$ is said to be correct if for all parameters λ and all integers n, the following properties hold with an overwhelming probability in λ: given $(\mathsf{mpk}, \mathsf{msk})$ issued by $\mathsf{SetUp}(1^\lambda)$, $(\mathsf{ek}_i, \mathsf{pk}_i)$ issued by $\mathsf{KeyGen}(\mathsf{id}_i, \mathsf{msk}, \mathsf{mpk})$ for $i \in [n]$, $\mathsf{L}_i := (\mathsf{pk}_1, ..., \mathsf{pk}_i)$ for all $i \in [n]$ and $L_0 = \emptyset$, hpk issued by $\mathsf{PKProcess}(\mathsf{L}_n, \mathsf{msk})$, $(v_i)_{i \in [n]} \in \mathbb{V}^n$ be votes from voters in $(\mathsf{id}_i)_{i \in [n]}$, b_i issued by $\mathsf{Vote}(\mathsf{ek}_i, v_i, \mathsf{hpk})$ and $\mathsf{BB}_i := (b_1, ..., b_i)$ for all $i \in [n]$ and $\mathsf{BB}_0 = \emptyset$, then*

[10] As in [29], the auxiliary information aux will contain the identity of the prover (the voter in this work) to prevent somebody else duplicating the proof and claiming to have made it.

– *public keys are registered successfully, that is,* $\mathsf{Register}(\mathsf{L}_{i-1}, \mathsf{pk}_i) = (\mathsf{L}_i, 1)$ *for all* $i \in [n]$*;*
– *ballots are appended successfully, that is,* $\mathsf{Append}(\mathsf{BB}_{i-1}, b_i, \mathsf{L}_n, \mathsf{hpk}) = (\mathsf{BB}_i, 1)$ *for all* $i \in [n]$*;*
– *the tally is correct and valid, that is, given* $\mathsf{res} = \mathsf{SelfTally}(\mathsf{BB}_n, \mathsf{hpk})$ *then*

$$\mathsf{res} = F((\mathsf{id}_i, v_i)_{i\in[n]}) \text{ and } \mathsf{Verify}(\mathsf{BB}_n, \mathsf{res}, \mathsf{L}_n, \mathsf{hpk}) = 1.$$

The universal verifiability is stated as follows.

Definition 21 (Universal Verifiability). *A voting scheme* $\mathcal{V}ot$ *for a universe of voters* $\mathcal{U}$ *and a result function* $F : (\mathcal{U} \times \mathbb{V})^* \to \{0,1\}^*$ *satisfies universal verifiability if, for any PPT adversary* $\mathcal{A}$*, the following experiment is sound with overwhelming probability:*

– $\mathcal{A}$ *outputs a master public key* mpk*, and selects the honest-voter set* $\mathcal{HS}$*.*
– *The challenger computes* $(\mathsf{pk}_{\mathsf{id}}, \mathsf{ek}_{\mathsf{id}}) \leftarrow \mathsf{KeyGen}(\mathsf{id}, \mathsf{msk}, \mathsf{mpk})$ *(possibly with* $\mathcal{A}$ *when a master secret key* msk *is needed as input) for* $\mathsf{id} \in \mathcal{HS}$*, posts* $\{\mathsf{pk}_{\mathsf{id}}\}_{\mathsf{id}\in\mathcal{HS}}$ *to a list of eligible voters* L *and stores the private encryption keys* $\{\mathsf{ek}_{\mathsf{id}}\}_{\mathsf{id}\in\mathcal{HS}}$*.*
– *On input* $\{\mathsf{pk}_{\mathsf{id}}\}_{\mathsf{id}\in\mathcal{HS}}$*,* $\mathcal{A}$ *completes the list* L *and outputs a helper public key* hpk*.*
– *On input valid votes* $v_{\mathsf{id}} \in \mathbb{V}$ *for* $\mathsf{id} \in \mathcal{HS}$ *chosen by* $\mathcal{A}$*, the challenger generates ballots* $b_{\mathsf{id}} \leftarrow \mathsf{Vote}(\mathsf{ek}_{\mathsf{id}}, v_{\mathsf{id}}, \mathsf{hpk})$ *and posts them to a ballot box* BB*.*
– *On input* $(b_{\mathsf{id}})_{\mathsf{id}\in\mathcal{HS}}$*,* $\mathcal{A}$ *completes the box* BB*.*
– *On input a complete box* BB*, the challenger generates correcting keys* $\mathsf{ck}_{\mathsf{id}} \leftarrow \mathsf{FaultCrt}(\mathsf{FaultAgg}(\mathsf{BB}, \mathsf{hpk}, \mathsf{id}), \mathsf{ek}_{\mathsf{id}})$ *for* $\mathsf{id} \in \mathcal{HS}$*.*
– *On input* $\{\mathsf{ck}_{\mathsf{id}}\}_{\mathsf{id}\in\mathcal{HS}}$*,* $\mathcal{A}$ *completes the list of correcting keys* $\{\mathsf{ck}_{\mathsf{id}}\}_{\mathsf{id}}$*.*
– $\mathcal{A}$ *finalises the game with a result* res*.*

Given the ballot box BB *containing* $\{b_{\mathsf{id}}\}_{\mathsf{id}\in\mathcal{HS}}$ *and* $\{b_{\mathsf{id}}\}_{\mathsf{id}\in\mathcal{CS}}$ *for some malicious-voter set* $\mathcal{CS}$*, if*

$$\mathsf{Verify}(\mathsf{BB}, \{\mathsf{ck}_{\mathsf{id}}\}_{\mathsf{id}\in\mathcal{HS}}, \{\mathsf{ck}_{\mathsf{id}}\}_{\mathsf{id}\in\mathcal{CS}}, \mathsf{res}, \mathsf{L}, \mathsf{hpk}) = 1,$$

then there exists a subset $\mathcal{CS}' \subset \mathcal{CS}$ *and* $v_{\mathsf{id}} \in \mathbb{V}$ *for* $\mathsf{id} \in \mathcal{CS}'$ *such that*

$$\mathsf{res} = F((\mathsf{id}, v_{\mathsf{id}})_{\mathsf{id}\in\mathcal{HS}}, (\mathsf{id}, v_{\mathsf{id}})_{\mathsf{id}\in\mathcal{CS}'}).$$

This notion is strong in the sense that if the verification succeeds, intended votes from honest voters are always included in the final result, along with only valid votes from malicious voters. In our voting protocols, this is particularly important because maliciously generated ballots and public keys may alter the intended votes of honest voters. Therefore, a verifiability notion that ensures only the correctness of tallying process is generally insufficient to prevent this kind of attack. For example, if the tallying process does not include ballot validation, even if the tallying is executed correctly, a malicious ballot can distort other votes in the final result.

B.2 Other Notions

We consider the following desirable properties, which are stated informally as in [28, 38] to avoid unnecessary formal overhead.

– **Fairness**: Nobody has access to a partial tally before the deadline. In the setting of self-tallying, we will interpret this demand in a relaxed way such that it is guaranteed by the ballot-box manager that plays as a voter to casts a dummy vote at the end of the voting phase. Through non-interactive zero-knowledge proofs, this action can be publicly verified.
– **Dispute-freeness**: This notion extends universal verifiability. A scheme is dispute-free if everybody can check whether voters act according to the protocol or not. In particular, this means that the result is publicly verifiable.

These properties are straightforward to inspect in our main voting protocol (described in Figure 8 and Figure 7) and do not require intricate analysis.

C More on Voting with Offline Mix-net

C.1 Correctness, Corrective Fault Tolerance, and Client Scalability

Correctness. On n registered public keys $\{\mathsf{pk}_i\}_{i\in[n]} = \{\mathsf{id}_i, \mathsf{ct}_i, \mathsf{ip.dk}^i_{(0,1)}, \mathsf{ip.dk}^i_{(0,0)}, \pi^{\mathsf{dk}}_i, \pi^{\mathsf{FS}}_i, [\![t_i]\!]_2\}_{i\in[n]}$, the $\mathsf{PKProcess}$ algorithm outputs MBox such that[11]

$$\mathsf{MBox}(j,i) = \mathsf{ip.dk}^i_j = \begin{cases} \mathsf{PKE.Enc}(\mathsf{PKE.pk}, \mathsf{ip.dk}^i_{(1,0;\gamma_i^{\mathsf{ip.dk}})}; \gamma_{\mathsf{PKE},j,i}) & \text{if } j = \Pi(i), \\ \mathsf{PKE.Enc}(\mathsf{PKE.pk}, \mathsf{ip.dk}^i_{(0,0;\gamma_i'^{\mathsf{ip.dk}})}; \gamma_{\mathsf{PKE},j,i}) & \text{if } j \neq \Pi(i). \end{cases}$$

and OBox such that

$$\mathsf{OBox}(j,i) = \mathsf{PKE.Enc}(\mathsf{PKE.pk}, \mathsf{ip.dk}^i_{(0,z_j;\gamma_{\mathsf{IPE},j,i})}; \gamma^{\mathsf{OBox}}_{j,i}) \ \forall i,j \in [n]$$

thanks to the structure-preserving homomorphic property of PKE ciphertexts and FE keys. Furthermore, one has

$$\begin{aligned} &\mathsf{MBox}(j,i) + \mathsf{OBox}(j,i) \\ &= \begin{cases} \mathsf{PKE.Enc}(\mathsf{PKE.pk}, \mathsf{ip.dk}^i_{(1,z_j;\gamma_i^{\mathsf{ip.dk}}+\gamma_{\mathsf{IPE},j,i})}; \gamma_{\mathsf{PKE},j,i} + \gamma^{\mathsf{OBox}}_{j,i}) & \text{if } j = \Pi(i), \\ \mathsf{PKE.Enc}(\mathsf{PKE.pk}, \mathsf{ip.dk}^i_{(0,z_j;\gamma_i'^{\mathsf{ip.dk}}+\gamma_{\mathsf{IPE},j,i})}; \gamma_{\mathsf{PKE},j,i} + \gamma^{\mathsf{OBox}}_{j,i}) & \text{if } j \neq \Pi(i). \end{cases} \end{aligned}$$

Therefore, the functional decryption keys output from the $\mathsf{PKProcess}$ algorithm will be in the same form as in the proof-of-concept voting protocol, i.e.

$$\mathsf{ip.dk}^i_j = \mathsf{ip.DKGen}(\mathsf{ip.sk}_i, [\![p^i_j, z_j, \mathbf{0}^{\rho(\lambda)}]\!]_1)$$

where $\mathbf{0}^{\rho(\lambda)}$ is zero padding for security and $\boldsymbol{p}^i$ is a vector that represents $\Pi(i)$ such that $p^i_j = 1$ if $j = \Pi(i)$ and $p^i_j = 0$ otherwise. Furthermore, as the vote encryption $(\mathsf{ip.ct}^i)_{i\in[n]}$ are the same as in the proof-of-concept voting protocol, then the decryption by $\sum_{i\in[n]} \mathsf{ip.Dec}(\mathsf{ip.dk}^i_j, \mathsf{ip.ct}^i)$ $\forall j \in [n]$ is also correct. The correctness of the protocol is complete as all proofs $\{\pi^{\mathsf{ct}}_i, \pi^{\mathsf{dk}}_i, \pi^{\mathsf{FS}}_i\}_{i\in[n]}, \pi^{\mathsf{Mix}}, \pi^{\mathsf{otp}}, \pi^{\mathsf{Dec}}$, and $\{\pi^i_{\mathsf{ck}}\}_{i\in[n]}$ are valid due to the completeness of the respective proof systems.

Corrective Fault Tolerance. The corrective fault tolerance is achieved as in the proof-of-concept protocol and can be verified by inspection.

Client Scalability. The protocol guarantees that both the per-voter communication and computation costs remain constant with respect to the total number of voters. Since the number of operations in each FE algorithm is $O_\lambda(1)$, the corresponding proofs for correct ciphertexts and functional keys also have complexities in $O_\lambda(1)$. Specifically, we have:

- In $\mathsf{KeyGen}(\mathsf{id}, \mathsf{mpk})$, each voter generates one element in $\mathbb{G}_2$, one FE secret key, three functional keys in $\mathbb{G}_1$, one public-key encryption in $\mathbb{G}_1$, and 2 accompanying correctness proofs for the functional keys and the public-key encryption.
- In $\mathsf{Vote}(\mathsf{ek}_{\mathsf{id}}, v, \mathsf{hpk})$, each voter generates one FE ciphertext in $\mathbb{G}_2$ and a correctness proof for its generation.
- In $\mathsf{FaultCrt}(\mathsf{fa}_{\mathsf{id}}, \mathsf{ek}_{\mathsf{id}})$, each voter generates one element in $\mathbb{G}_2$ and a proof of known discrete logarithm.

Hence, the protocol satisfies client scalability as defined in Definition 4.

C.2 Universal Verifiability

Theorem 4 (Universal Verifiability). *The voting protocol described in Figure 7 and Figure 8 achieves universal verifiability as per Definition 21, assuming the soundness property of the NIZK proof systems for the relations $\mathcal{R}_{\mathsf{dk}}$, $\mathcal{R}_{\mathsf{ct}}$, $\mathcal{R}_{\mathsf{Mix}}$, $\mathcal{R}_{\mathsf{otp}}$, $\mathcal{R}_{\mathsf{Dec}}$ and $\mathcal{R}_{\mathsf{Agg}}$.*

[11] We denote by $\gamma_i^{\mathsf{ip.dk}}$, $\gamma_i'^{\mathsf{ip.dk}}$, and $\gamma_{\mathsf{IPE},j,i}$ the randomness that generate the FE keys.

Proof. Assume that at the end of the soundness game described in Definition 21, one has $\mathsf{Verify}(\mathsf{BB} := \{b_{\mathsf{id}}\}_{\mathsf{id}\in\mathcal{HS}\cup\mathcal{CS}}, \{\mathsf{ck}_{\mathsf{id}}\}_{\mathsf{id}\in\mathcal{HS}\cup\mathcal{CS}}, \mathsf{res}, \mathsf{L} := \{\mathsf{pk}_i\}_{i\in[n]}, \mathsf{hpk}) = 1$ where $\mathcal{HS}$ is the set of honest voters and $\mathcal{CS}$ is the set of corrupted voters. Applied to the protocol, the valid verification implies that $(\mathsf{pk}_{\mathsf{id}})_{\mathsf{id}\in\mathcal{HS}\cup\mathcal{CS}}$ are contained in the list $\mathsf{L} := \{\mathsf{pk}_i\}_{i\in[n]}$ of some n voters. The following verifications are also valid:

- Registering: the validity of π_i^{dk} for all $i \in [n]$ implies that the basis keys $\{\mathsf{ct}_i, \mathsf{ip.dk}^i_{(0,1)}, \mathsf{ip.dk}^i_{(0,0)}, [\![t_i]\!]_2\}_{i\in[n]}$ are generated as in the KeyGen algorithm.
- Public-key processing: the validity of π^{Mix} implies the mixing process is valid, i.e. the MBOX is generated from the basis functional keys of all voters in L; the validity of π^{otp} implies the process of generating one-time pads and key randomness is valid, i.e. the terms in OBox and $(\mathsf{ct}_j^{\mathsf{PAD}})_{j\in[n]}$ are consistently and correctly generated. Lastly, one has the validity of π^{Dec} implies $(\mathsf{ct}_j^{\mathsf{PAD}})_{j\in[n]}$ and the sum between MBOX and OBox are decrypted correctly, giving correct $[\![\boldsymbol{z}]\!]_1$ and decryption keys $(\mathsf{ip.dk}^i_j)_{j,i\in[n]}$, respectively, as in the $\mathsf{PKProcess}$ algorithm.
- Ballot appending: the validity of π_i^{ct} for all $i \in \mathcal{CS}$ implies that the ciphertexts $(\mathsf{ip.ct}^i)_{i\in\mathcal{CS}}$ are generated correctly with respect to the FE secret keys $(\mathsf{ip.sk}_i)_{i\in\mathcal{CS}}$ used in the basis functional keys $\{\mathsf{ct}_i, \mathsf{ip.dk}^i_{(0,1)}, \mathsf{ip.dk}^i_{(0,0)}\}_{i\in\mathcal{CS}}$ in L.
- Fault correcting: the validity of π^i_{ck} for all $i \in \mathcal{CS}$ implies that the correcting keys ck_i are generated correctly as in the $\mathsf{FaultCrt}$ algorithm, with respect to the registered keys $[\![t_i]\!]_2$ and the fault aggregates fa_i for the set $n \setminus (\mathcal{HS} \cup \mathcal{CS})$.
- Tallying: the validity of res implies that it is equal to the output of the public algorithm $\mathsf{SelfTally}(\mathsf{BB}, \{\mathsf{ck}_i\}_{i\in\mathcal{CS}\cup\mathcal{HS}}, \mathsf{hpk})$, which by the corrective-fault-tolerance property returns a permutation of $\{v_i\}_{i\in\mathcal{HS}\cup\mathcal{CS}'}$ where $\{v_i\}_{i\in\mathcal{HS}}$ are valid votes chosen by the adversary and $\{v_i\}_{i\in\mathcal{CS}'}$ are valid votes that can be recovered from the set of corrupted voters' ballots.

The protocol therefore satisfies the universal verifiability. □

C.3 Ballot Privacy

We first provide in Figure 13 a specification of the non-interactive Σ-protocol, denoted as $\mathsf{NIZK}_{\mathcal{R}_{\mathsf{FS}}}$, that is used to avoid copy-ballot attacks in the voting protocol with offline mix-net (see Figure 7 and Figure 8). Both soundness and zero-knowledge property of $\mathsf{NIZK}_{\mathcal{R}_{\mathsf{FS}}}$ rely on the random oracle for rewinding and programming, respectively.

- *Knowledge Soundness*: Given two transcripts $(R_{\mathsf{FS}}, \boldsymbol{R}_{\mathsf{mess}}, e, \boldsymbol{\rho}_{\mathsf{mess}}, \rho_{\mathsf{FS}})$ and $(R_{\mathsf{FS}}, \boldsymbol{R}_{\mathsf{mess}}, e', \boldsymbol{\rho}'_{\mathsf{mess}}, \rho'_{\mathsf{FS}})$ as input, the extractor will output $\boldsymbol{m} := (\boldsymbol{\rho}_{\mathsf{mess}} - \boldsymbol{\rho}'_{\mathsf{mess}})/(e - e')$ and $r := (\rho_{\mathsf{FS}} - \rho'_{\mathsf{FS}})/(e - e')$ as witness.
- *Zero-Knowledge Property*: Given a statement $(\mathsf{id}, \mathsf{pk} := \boldsymbol{P}, \mathsf{ct} := (\mathsf{ct}_1, \mathsf{ct}_2))$ as input, the zero-knowledge simulator will output a proof $\pi_{\mathsf{FS}} = (\mathsf{id}, \mathsf{pk}, \mathsf{ct}, R_{\mathsf{FS}} := [\![\rho_{\mathsf{FS}}]\!] - e \cdot \mathsf{ct}_2, \boldsymbol{R}_{\mathsf{mess}} := \rho_{\mathsf{FS}} \cdot \boldsymbol{P} + [\![\boldsymbol{\rho}_{\mathsf{mess}}]\!] - e \cdot \mathsf{ct}_1, \boldsymbol{\rho}_{\mathsf{mess}}, \rho_{\mathsf{FS}})$ where $\boldsymbol{\rho}_{\mathsf{mess}}, \rho_{\mathsf{FS}}$ and e are selected at random and set $\mathcal{H}(\mathsf{id}, \mathsf{pk}, \mathsf{ct}, R_{\mathsf{FS}}, \boldsymbol{R}_{\mathsf{mess}}) := e$.

Theorem 5 (Ballot Privacy). *The voting protocol described in Figure 7 and Figure 8 achieves ballot privacy in the selective and symmetric-key setting (as per Definition 5), assuming the zero-knowledge property of the NIZK proof systems for the relations in $\{\mathcal{R}_{\mathsf{dk}}, \mathcal{R}_{\mathsf{ct}}, \mathcal{R}_{\mathsf{Mix}}, \mathcal{R}_{\mathsf{otp}}, \mathcal{R}_{\mathsf{Dec}}, \mathcal{R}_{\mathsf{ck}}\}$, the random oracle in the $\mathsf{NIZK}_{\mathcal{R}_{\mathsf{FS}}}$ proof system (described in Figure 13), the IND-CPA security of the PKE scheme, the SXDH assumption in $\mathcal{PG}$ and the IND-CPA security of the FH-IPFE scheme.*

Proof. At the end of the game, let L be the list of eligible voters, $\mathcal{CS}$ be the set of voters that are corrupted via QCor, and $\mathcal{HS}$ be the subset of honest voters (whose public keys are generated via $\mathsf{QNewHon}$ but not corrupted via QCor) that are queried and recorded in QVote_β. In the selective setting, $\mathcal{HS}, \mathcal{CS}$, and $\{(\mathsf{id}, v^0, v^1)\}_{\mathsf{id}\in\mathcal{HS}}$ are specified beforehand from the $\mathsf{QNewHon}$, QCor and QVote_β queries. In the symmetric-key setting, any (id, v^0, v^1) vote queries for $\mathsf{id} \in \mathcal{CS}$ must satisfy $v^0 = v^1$.

We also note that in L, there is also a subset of dropout honest voters (honest-but-not recorded by QVote_β queries), that we denote by $\mathcal{DH}$; and a subset of voters that are registered by the adversary via $\mathsf{QRegCast}$ queries, that we denote by $\mathcal{RC}$. The four sets $\mathcal{HS}$, $\mathcal{CS}$, $\mathcal{DH}$, $\mathcal{RC}$ are disjoint sets that make up L.

$\mathsf{NIZK}_{\mathcal{R}_{\mathsf{FS}}}.\mathsf{SetUp}(1^\lambda)$: Outputs a group $(\mathbb{G}, p)$ and a hash function $\mathcal{H} : \{0,1\}^* \to \mathbb{Z}_p$

$\mathsf{NIZK}_{\mathcal{R}_{\mathsf{FS}}}.\mathsf{Prove}(\mathsf{id}, \mathsf{pk} := \boldsymbol{P}, \mathsf{ct} := (r \cdot \boldsymbol{P} + [\![\boldsymbol{m}]\!], [\![r]\!]); (\boldsymbol{m}, r))$:

1. Commit randomness $r_{\mathsf{FS}} \xleftarrow{\$} \mathbb{Z}_p$ as $R_{\mathsf{FS}} = [\![r_{\mathsf{FS}}]\!]$ and $\boldsymbol{r}_{\mathsf{mess}} \xleftarrow{\$} \mathbb{Z}_p^{|\boldsymbol{m}|}$ as $\boldsymbol{R}_{\mathsf{mess}} = r_{\mathsf{FS}} \cdot \boldsymbol{P} + [\![\boldsymbol{r}_{\mathsf{mess}}]\!]$;
2. Compute a challenge $e \leftarrow \mathcal{H}(\mathsf{id}, \mathsf{pk}, \mathsf{ct}, R_{\mathsf{FS}}, \boldsymbol{R}_{\mathsf{mess}})$;
3. Compute a response $(\boldsymbol{\rho}_{\mathsf{mess}} = \boldsymbol{r}_{\mathsf{mess}} + e \cdot \boldsymbol{m}, \rho_{\mathsf{FS}} = r_{\mathsf{FS}} + e \cdot r)$ and outputs $\pi_{\mathsf{FS}} = (\mathsf{id}, \mathsf{pk}, \mathsf{ct}, R_{\mathsf{FS}}, \boldsymbol{R}_{\mathsf{mess}}, \boldsymbol{\rho}_{\mathsf{mess}}, \rho_{\mathsf{FS}})$;

$\mathsf{NIZK}_{\mathcal{R}_{\mathsf{FS}}}.\mathsf{Verify}(\pi_{\mathsf{FS}} := (\mathsf{id}, \mathsf{pk}, \mathsf{ct}, R_{\mathsf{FS}}, \boldsymbol{R}_{\mathsf{mess}}, \boldsymbol{\rho}_{\mathsf{mess}}, \rho_{\mathsf{FS}}))$:

1. Compute the challenge $e \leftarrow \mathcal{H}(\mathsf{id}, \mathsf{pk}, \mathsf{ct}, R_{\mathsf{FS}}, \boldsymbol{R}_{\mathsf{mess}})$;
2. Parse $\mathsf{ct} = (\mathsf{ct}_1, \mathsf{ct}_2)$;
3. Verify if $e \cdot \mathsf{ct}_1 + \boldsymbol{R}_{\mathsf{mess}} = \rho_{\mathsf{FS}} \cdot \boldsymbol{P} + [\![\boldsymbol{\rho}_{\mathsf{mess}}]\!]$ and $e \cdot \mathsf{ct}_2 + R_{\mathsf{FS}} = [\![\rho_{\mathsf{FS}}]\!]$;
4. If yes, output 1, and 0 otherwise.

Fig. 13. The $\mathsf{NIZK}_{\mathcal{R}_{\mathsf{FS}}}$ protocol for ElGamal encryption used in Section 4

To align with permutations, we assume $n = |\mathsf{L}|$ and index the voters in L using integers from $[n]$. From this point forward, we denote by $\mathcal{HI}$, $\mathcal{CI}$, $\mathcal{DI}$, $\mathcal{RI}$ as the sets of indices in $[n]$ corresponding to the voters in $\mathcal{HS}$, $\mathcal{CS}$, $\mathcal{DH}$, $\mathcal{RC}$ respectively.

Compared to the proof-of-concept scheme in Figure 6, the protocol in Figure 7 exposes more information about secrets of voters in $\mathcal{HI}$, whose ballots contain the challenge bit. Besides zero-knowledge proofs, the adversary also gains additional access to:

- the functional keys $\mathsf{ip.dk}^i_{(1,0)}$ for decrypting the vote slot, which are encrypted under PKE in ct_i of pk_i;
- the functional keys embedding $\{\Pi(i)\}_{i \in \mathcal{HI}}$ that are encrypted under PKE as columns $\{i\}_{i \in \mathcal{HI}}$ in MBox of hpk;
- the functional keys embedding the one-time pads $\{z_j\}_{j \in [n]}$ and the functional-key randomness $\{\gamma_{\mathsf{IPE},j,i}\}_{j \in [n], i \in \mathcal{HI}}$, encrypted under PKE as columns $\{i\}_{i \in \mathcal{HI}}$ in OBox of hpk.

To eliminate this additional leakage, we modify the protocol so that all the above PKE ciphertexts instead encrypt a null value $\mathbf{0}$. More formally, we introduce the following hybrid games to transition from the real game $\mathbf{G}_0$ to the game $\mathbf{G}_3$, where the adversary's view contains no more information on secrets of voters in $\mathcal{HI}$ than in the proof-of-concept scheme.

Game $\mathbf{G}_1$: In this game, via a hybrid, we switch all the proof systems that ensure universal verifiability, namely for the relations in $\{\mathcal{R}_{\mathsf{dk}}, \mathcal{R}_{\mathsf{ct}}, \mathcal{R}_{\mathsf{Mix}}, \mathcal{R}_{\mathsf{otp}}, \mathcal{R}_{\mathsf{Dec}}, \mathcal{R}_{\mathsf{ck}}\}$ to the zero-knowledge setting, by using simulator to generate proofs[12]. Through a hybrid argument, one can use the adversary $\mathcal{A}$ to construct a distinguisher between two settings for one of the proof systems.

Game $\mathbf{G}_2$: In this game, by relying on the programmability of the random oracle $\mathcal{RO}$, the challenger simulate Σ-protocol proofs π_i^{FS} for the knowledge of plaintexts under ciphertexts ct_i that are generated via $\mathsf{QNewHonest}$ queries. Let $(\mathsf{PKE.pk}, \mathsf{ct}_i)$ be the statement and id_i be the auxiliary information, the challenger chooses z as a response, e as a random challenge and computes the initial message a. Provided that $(\mathsf{id}_i, (\mathsf{PKE.pk}, \mathsf{ct}_i), a)$ has not been queried before by the adversary, the challenger assigns the value e to be the answer to query $\mathcal{RO}(\mathsf{id}_i, (\mathsf{PKE.pk}, \mathsf{ct}_i), a)$. The indistinguishability between $\mathbf{G}_1$ and $\mathbf{G}_2$ are statistical, as the probability that the adversary correctly guess the randomness in ct_i, which is generated honestly in $\mathsf{QNewHonest}$ oracle, to be queried to $\mathcal{RO}$ is negligible.

Game $\mathbf{G}_3$: In this game, the challenger replaces PKE ciphertexts by the following strategy: it first replaces the ciphertexts ct_i that are queried via $\mathsf{QNewHonest}$ by encryptions of $\mathbf{0}$. It then runs the $\mathsf{PKProcess}$ algorithm on the replaced ciphertexts ct_i (in pk_i) to answer the $\mathsf{QPKProcess}$ query. Before outputting, it replaces the ciphertext columns indexed by $\{i\}_{i \in \mathcal{HI}}$ in the resulting MBox and OBox by encryptions of $\mathbf{0}$. Additionally, the challenger outputs the functional keys $(\mathsf{ip.dk}^i_j)_{j,i \in [n]}$ as if they were generated without any replacements. The indistinguishability between $\mathbf{G}_2$ and $\mathbf{G}_3$ follows from the IND-CPA security of the PKE scheme, as shown in Lemma 1.

[12] If the NIZK systems are $\mathcal{RO}$-based, we add a specific counter in $\mathcal{RO}$ queries of each system to avoid collisions across different proofs during simulation.

In game $\mathbf{G}_3$, the adversary's view on secrets of voters in $\mathcal{HI}$ is identical to that in the proof-of-concept scheme, while the challenger retains the ability to program the random permutation, the one-time pads, and the FH-IPFE keys/ciphertexts that correspond to these voters. Let the challenge votes be $(v_i^\beta)_{i\in\mathcal{HI}}$, where $\beta \in \{0,1\}$. There exists a random permutation Π^β over n such that $\Pi^\beta(i) = i$ for $i \in [n]\setminus\mathcal{HI}$ and for any random permutation Π, applying $\Pi\circ\Pi^\beta$ to $(v_i^\beta)_{i\in[n]}$ is equivalent to applying Π to $(v_i^0)_{i\in[n]}$, assuming $v_i^\beta = v_i^0 = v_i^1$ for $i \in [n] \setminus \mathcal{HI}$.

Denote the columns forming the matrices corresponding to $\Pi \circ \Pi^\beta$ and Π as $(\boldsymbol{p}^{\beta,i})_{i\in[n]}$ and $(\boldsymbol{p}^{0,i})_{i\in[n]}$, respectively, one has

$$\sum_{i\in\mathcal{HI}} p_j^{\beta,i} \cdot v_i^\beta = \sum_{i\in\mathcal{HI}} p_j^{0,i} \cdot v_i^0 \ \forall j \in [n]$$

by Remark 3. Using a similar argument as in the proof of Theorem 1, we transition from $\mathbf{G}_3$ to the following game, where there is no information on the challenge bit.

Game $\mathbf{G}_4$: In this game, instead of using $(v_i^\beta)_{i\in\mathcal{HI}}$ and $(\boldsymbol{p}^{\beta,i})_{i\in\mathcal{HI}}$, the challenger uses $(v_i^0)_{i\in\mathcal{HI}}$ and $(\boldsymbol{p}^{0,i})_{i\in\mathcal{HI}}$ to respond to QVote_β and QPKProcess queries, respectively. The indistinguishability between $\mathbf{G}_3$ and $\mathbf{G}_4$ is implied by the SXDH assumption in $\mathcal{PG}$ and the IND-CPA security of the FH-IPFE scheme.

We finish the proof by proving the below lemma in Section C.

Lemma 1. *The transition between $\mathbf{G}_2$ and $\mathbf{G}_3$ is indistinguishable, under the IND-CPA security of the* PKE *scheme.*

Proof. Given an adversary $\mathcal{B}$ that plays against the IND-CPA security and receives the public key $\mathsf{PKE.pk}$, then $\mathcal{B}$ simulates the challenger of the voting protocol as follows:

- To answer $\mathsf{QNewHonest(id)}$: $\mathcal{B}$ generates basis functional keys and stores the FH-IPFE secret key as in the KeyGen algorithm. To generate the ciphertext $\mathsf{ct_{id}}$, it sends $(m_0 = \mathsf{ip.dk}^{\mathsf{id}}_{(1,0)}, m_1 = \mathbf{0})$ as challenge messages to the PKE encryption oracle. If m_0 is chosen, we are in game $\mathbf{G}_2$; if m_1 is chosen, we are in game $\mathbf{G}_3$. To complete, $\mathcal{B}$ simulates the proof $\pi^{\mathsf{FS}}_{\mathsf{id}}$ for $\mathsf{ct_{id}}$ by programming $\mathcal{RO}$ as described in game $\mathbf{G}_2$. If any simulation fails, $\mathcal{B}$ aborts with an error called "Simulation Error".
  Additionally, $\mathcal{B}$ will need to run a "no-replacement" version, it generates a ciphertext $\mathsf{ct'_{id}}$ that encrypts $\mathsf{ip.dk}^{\mathsf{id}}_{(1,0)}$ and stores the pair $(\mathsf{ip.dk}^{\mathsf{id}}_{(1,0)}, \mathsf{ct'_{id}})$.
- To answer $\mathsf{QCor(id)}$: $\mathcal{B}$ returns the FH-IPFE secret key to the adversary if $\mathsf{ip.sk_{id}}$ is stored and ignore otherwise.
- To answer QPKProcess: Given the list $\mathsf{L} = \{\mathsf{pk}_i\}_{i\in[n]}$, $\mathcal{B}$ chooses a random permutation Π, one-time pads $\{z_j\}_{j\in[n]}$ and key randomness $\{\gamma_{\mathsf{IPE},j,i}\}_{j,i\in[n]}$. It generates $(\mathsf{pz}_i)_{i\in[n]}$ and $(\mathsf{ct}_j^{\mathsf{PAD}})_{j\in[n]}$ and processes them as in the real game. The following modifications are applied:
  - It first generates OBox as in the PKProcess algorithm. For each entry on the columns indexed by $\mathcal{HI}$, $\mathcal{B}$ sends $(m_0 = z_j \cdot \mathsf{ip.dk}^i_{(0,1)} + \gamma_{\mathsf{IPE},j,i} \cdot \mathsf{ip.dk}^i_{(0,0)}, m_1 = \mathbf{0})$ to the PKE encryption oracle, receives back a ciphertext $\mathsf{ct}^{\mathsf{OBox}}_{j,i}$, and replaces the entry $\mathsf{OBox}(j,i)$ by $\mathsf{ct}^{\mathsf{OBox}}_{j,i}$. If m_0 is encrypted, we have the resulting box is generated as in game $\mathbf{G}_2$ (replacement and no-replacement are equivalent in this case); and if m_1 is encrypted, we have the resulting box is generated as in game $\mathbf{G}_3$.
  - To generate MBox, it starts with MBox_I that is patched with replaced ciphertexts ct_i where

$$\mathsf{MBox}_I(j,i) = \begin{cases} \mathsf{ct}_i & \text{if } j = i, \\ \mathsf{PKE.Enc}(\mathsf{PKE.pk}, \mathsf{ip.dk}^i_{(0,0)}; 0) & \text{if } j \neq i. \end{cases}$$

  and then permutes and re-randomizes MBox_I by Π. On the other hand, it runs a no-replacement version by starting with MBox'_I that is instead patched with $\mathsf{ct'_{id}}$, resulting in MBox'.

$$\mathsf{MBox}'_I(j,i) = \begin{cases} \mathsf{ct}_i & \text{if } j = i \text{ and } i \in \mathcal{RI}, \\ \mathsf{ct}'_i & \text{if } j = i \text{ and } i \in [n] \setminus \mathcal{RI}, \\ \mathsf{PKE.Enc}(\mathsf{PKE.pk}, \mathsf{ip.dk}^i_{(0,0)}; 0) & \text{if } j \neq i. \end{cases}$$

As the plaintext of ct'_i is associated with $\mathsf{ip.dk}^i_{(1,0)}$, $\mathcal{B}$ knows the plaintext $\boldsymbol{m}^{\mathsf{MBox}'}_{j,i}$ under $\mathsf{MBox}'(j,i)$ for all voters $i \in [n] \setminus \mathcal{RI}$ (those who are registered via QNewHonest queries). For each entry on the columns indexed by $\mathcal{HI}$, it sends $(m_0 = \boldsymbol{m}^{\mathsf{MBox}'}_{j,i}, m_1 = \mathbf{0})$ to the PKE encryption oracle, receives back a ciphertext $\mathsf{ct}^{\mathsf{MBox}}_{j,i}$, and replaces the entry $\mathsf{MBox}(j,i)$ by $\mathsf{ct}^{\mathsf{MBox}}_{j,i}$. If m_0 is encrypted, we have the resulting box is generated as in game $\mathbf{G}_2$ (box-entry replacement and no-replacement are equivalent in this case); and if m_1 is encrypted, we have the resulting box is generated as in game $\mathbf{G}_3$.

- To simulate the decryption of MBox+OBox, $\mathcal{B}$ needs to extract $\mathsf{ip.dk}^i_{(1,0)}$ from ct_i of each voter $i \in \mathcal{RI}$ who is registered by the adversary via QReqCast queries. The extraction is done via the proof π^{FS}_i submitted by $\mathcal{A}$ in the form of (identity, statement, initial message, reponse), or $(\mathsf{aux} := \mathsf{id}_i, x, a, z)$ when using the notation in Σ-protocol. To do that, $\mathcal{B}$ rewinds the adversary $\mathcal{A}$ up to this decryption time. If $\mathcal{A}$ queries $\mathcal{RO}$ on input (aux, x, a) to compute the challenge, then in each rewinding time, $\mathcal{B}$ assigns the output with a different value e'. Once a valid proof (x, a, z') is submitted by $\mathcal{A}$ in this rewinding, $\mathcal{B}$ can extract the plaintext of ct_i. This extraction fails if $\mathcal{A}$ does not need to query $\mathcal{RO}$ to compute the challenge, or if the assignment of (aux, x, a) with an output for the oracle has already been made by $\mathcal{B}$ for the zero-knowledge simulation; in both cases, $\mathcal{B}$ aborts with an error called "Extraction Error". Once all $\mathsf{ip.dk}^i_{(1,0)}$ are extracted for all $i \in \mathcal{RI}$, $\mathcal{B}$ achieves the basis functional keys of all registered voters in plain, then performs a shuffling by Π and adds randomness $(z_j, \gamma_{\mathsf{IPE},j,i})_{j,i\in[n]}$ as if they were encrypted under PKE and processed homomorphically in the PKProcess algorithm.
- $\mathsf{QReqCast}(\mathsf{id})$, $\mathsf{QVote}_\beta(\mathsf{id}, v_0, v_1)$, $\mathsf{QVotCast}(\mathsf{id}, b_{\mathsf{id}})$, $\mathsf{QFaultCrt}()$: $\mathcal{B}$ operates the same as the challenger does in these oracles. This is because there is no requirement for querying PKE encryption oracle or PKE.sk-involved operations in answering these queries.

At the end, $\mathcal{B}$ returns the output bit of the ballot privacy game as its guess for the IND-CPA game. As in [10], we claim the probability that $\mathcal{B}$ aborts in the above simulation is negligible:

- Simulation Error happens when $\mathcal{A}$ has already queried $(\mathsf{id}_i, (\mathsf{PKE.pk}, \mathsf{ct}_i), a)$ for some voter i registered via QNewHonest. To do this, $\mathcal{A}$ has to guess on the randomness used in ct_i, which is generated honestly. The probability is negligible.
- Extraction Error: the first case is when $\mathcal{A}$ does not need to query $\mathcal{RO}$ to compute the challenge, then it must have guessed correctly the output of $\mathcal{RO}$ on input $(i, (\mathsf{PKE.pk}, \mathsf{ct}_i), a)$ for which it generates the proof (e.g. $i \in \mathcal{RI}$). This guess is before $\mathcal{B}$ makes an $\mathcal{RO}$ assignment for the verification. As the output to be assigned will be random, the first case happens with negligible probability. The second case happens when the input $(\mathsf{id}_i, (\mathsf{PKE.pk}, \mathsf{ct}_i), a)$ needs to be assigned with some output for the zero-knowledge simulation. As we hardwire id_i in each proof for ct_i and simulate proofs for only i queried via QNewHon oracle (e.g. $i \in [n] \setminus \mathcal{RI}$), such case never happens. □

The ballot-privacy proof completes as the above lemma holds. □

D More on Linearly-Homomorphic Signature

D.1 Additional Assumptions

In this work, security notions of linearly-homomorphic signature will be proved in the generic bilinear group model, under the following assumptions.

Definition 22 (Square Discrete Logarithm (SDL) Assumption). *In a group $\mathbb{G}$ of prime order p, it states that for any generator P and a random $x \xleftarrow{\$} \mathbb{Z}_p$, given $(\mathbb{G}, p, P)$ and $(Y = x \cdot P, Z = x^2 \cdot P)$, it is computationally hard to recover x.*

Definition 23 (Extractability Assumption). *In a group $\mathbb{G}$ of prime order p, the extractability assumption states that given vectors $(\boldsymbol{M}_j = (M_{j,i})_i)_j$ from elements in $\mathbb{G}$, for any adversary $\mathcal{A}$, there exists an extractor $\mathsf{Ext}^{\mathcal{A}}$ such that whenever $\mathcal{A}$ produces a new vector $\boldsymbol{M} = \sum_j \alpha_j \cdot \boldsymbol{M}_j$, $\mathsf{Ext}^{\mathcal{A}}$ outputs $(\alpha_j)_j$.*

We will rely on the following theorem, which is proved in [35], to restrict combinations of vectors.

Theorem 6 (DL-Based Restricted Combinations of Vectors). *In a group $\mathbb{G}$ of prime order p, given n valid Square Diffie-Hellman tuples $(G_i, A_i = w_i \cdot G_i, B_i = w_i \cdot A_i)_{i\in[n]}$ for random $G_i \xleftarrow{\$} \mathbb{G}^*$ and $w_i \xleftarrow{\$} \mathbb{Z}_p^*$, outputting $(\alpha_i)_{i\in[n]}$ such that $(G = \sum_i \alpha_i \cdot G_i, A = \sum_i \alpha_i \cdot A_i, B = \sum_i \alpha_i \cdot B_i)$ is a valid Square Diffie-Hellman tuple, with at least two non-zero coefficients α_i, is computationally hard under the DL assumption.*

D.2 Tag-Binding Unforgeability

The tag-binding unforgeability ensures that if the forged signatures are valid under some tag, their corresponding messages must be forged from those that are signed under the same original tag.

We show that if a LHS scheme has homomorphic verification, which is defined below, then its unforgeability implies its tag-binding unforgeability.

Definition 24 (Homomorphic Verification). *A* LHS *scheme supports homomorphic verification if, given any two signatures $(\boldsymbol{M}_1, \sigma_1)$ and $(\boldsymbol{M}_2, \sigma_2)$ that are valid (but possibly malformed) with respect to a tag τ and a verification key* vk*, i.e.* $\mathsf{Verif}(\mathsf{vk}, \tau, \boldsymbol{M}_i, \sigma_i) = 1$ *for $i \in \{1, 2\}$, for any scalars $\alpha_1, \alpha_2 \in \mathbb{Z}_p$, the signature σ derived from $\alpha_1\sigma_1 + \alpha_2\sigma_2$ is also valid with respect to the message $(\alpha_1\boldsymbol{M}_1 + \alpha_2\boldsymbol{M}_2)$ and the tuple (τ, vk).*

We have the following main theorem.

Theorem 7 (Tag-Binding Unforgeability from Homomorphic Verification). *Given a* LHS *scheme, if it supports homomorphic verification, it is also tag-binding unforgeable (as defined in Definition 11).*

Proof. We consider an adversary $\mathcal{A}$ in the tag-binding unforgeability game of the scheme. We then construct an adversary $\mathcal{B}$ that runs $\mathcal{A}$ to break the unforgeability.

Upon receiving the NewTag and Sign queries from $\mathcal{A}$, the adversary $\mathcal{B}$ simply forwards these queries to the oracles of its game to simulate answers. We then denote by $(\boldsymbol{M}_i)_i$ the set of queried messages under tags $(\tau_i)_i$ (possibly equal across i) of which both are chosen by $\mathcal{A}$. We also denote by V_{τ_i} the sub-vector space generated by messages queried to Sign under τ_i.

In the end when $\mathcal{A}$ outputs the set of forgeries $(\mathsf{vk}, \tau, \boldsymbol{M}'_j, \sigma_j)_{j\in[\ell]}$, the adversary $\mathcal{B}$ then generates randomly $(\alpha_j)_{j\in[\ell]} \xleftarrow{\$} \mathbb{Z}_p^\ell$ and outputs its forgery as

$$(\mathsf{vk}, \tau, \boldsymbol{M}^\star = \sum_{j\in[\ell]} \alpha_j \cdot \boldsymbol{M}'_j, \sigma^\star = \sum_{j\in[\ell]} \alpha_j \cdot \sigma_j)_{j\in[\ell]}.$$

By the homomorphic verification, if $\mathcal{A}$'s forgeries are valid, then $\mathcal{B}$'s forgery is also valid. It then suffices to prove that if there does not exist a queried tag $\tau' \in (\tau_i)_i$ such that $\boldsymbol{M}'_j \in V_{\tau'}$ for all $j \in [\ell]$, then with overwhelming probability, there does not exist a queried tag $\tau' \in (\tau_i)_i$ such that $\boldsymbol{M}^\star \in V_{\tau'}$. We assume that the former case already happened.

By the assumption, for each $\tau' \in (\tau_i)_i$, we have a non-empty index subset $\bar{\mathcal{I}}'_\tau \subset [\ell]$ such that $\boldsymbol{M}'_k \notin V_{\tau'}$ if $k \in \bar{\mathcal{I}}'_\tau$. Let $U_{\tau'}$ represent the vector space generated by $V_{\tau'}$ and vectors $(\boldsymbol{M}'_k)_{k\in\bar{\mathcal{I}}_{\tau'}}$. By construction, $U_{\tau'}$ strictly contains $V_{\tau'}$. WLOG, we assume that $\{\boldsymbol{b}_t\}_{t\in N}$ is a subset of a basis of $U_{\tau'}$ but does not lie in $V_{\tau'}$, for some N subject to $1 \le N \le |\mathcal{I}'_\tau|$. By the assumption, the coefficient tuple $\boldsymbol{c}_k$ of each message in $(\boldsymbol{M}'_k)_{k\in[\ell],k\notin\bar{\mathcal{I}}'_\tau}$ with respect to basis vectors $\{\boldsymbol{b}_t\}_{t\in N}$ is $(0, .., 0)$ (trivial) as these messages lie in $V_{\tau'}$, while that of each message in $(\boldsymbol{M}'_k)_{k\in\bar{\mathcal{I}}'_\tau}$ is non-trivial.

Therefore, the forged message $\boldsymbol{M}^\star$ lies in $V_{\tau'}$ if and only if the equality $\sum_{k\in\bar{\mathcal{I}}'_\tau} \alpha_k \boldsymbol{c}_k = \boldsymbol{0}$ holds. Since each $\boldsymbol{c}_k$ for $k \in \bar{\mathcal{I}}'_\tau$ is non-trivial, the number of solutions for $(\alpha_k)_{k\in\bar{\mathcal{I}}'_\tau}$ is upper-bounded by $p^{|\mathcal{I}'_\tau|-1}$. Therefore, the probability that a random $(\alpha_k)_{k\in\bar{\mathcal{I}}'_\tau}$ becomes a solution is upper-bounded by $1/p$. As $p = \mathcal{O}(2^\lambda)$, the probability that $\boldsymbol{M}^\star$ lies in $V_{\tau'}$ is negligible.

As the number of queried tags $(\tau_i)_i$ is polynomial in λ, then the probability that $\boldsymbol{M}^\star$ lies in V_{τ_i} for some i is also negligible. This completes the proof. □

Remark 4. The LHS scheme, originally constructed for signing projective equivalence classes in [26] and subsequently shown to support the signing of linear spaces in [35], is tag-binding unforgeable.

We recall the verification algorithm of the scheme as follows:

– the parameter is $\mathsf{pp} = (\mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, p, e, \mathcal{T} = \mathbb{G}_1 \times \mathbb{G}_2, \rho)$;
– the verification key is $\mathsf{vk} = ([s_i]_2)_{i\in[\rho]}$ where $(s_i)_{i\in[\rho]}$ is the signing key;
– a vector-message is $\boldsymbol{M} = (M_i)_{i\in[\rho]} \in \mathbb{G}_1^\rho$;
– a tag is $\tau = (\tau_1 \in \mathbb{G}_1, \tau_2 \in \mathbb{G}_2)$;
– a signature σ is in $\mathbb{G}_1$.

The verification then takes as input a tuple $(\mathsf{pp}, \mathsf{vk}, \tau, \boldsymbol{M}, \sigma)$ and checks if the equalities $e(\sigma, \tau_2) = \sum_{i\in[\rho]} e(M_i, [s_i]_2)$ and $e(\tau_1, [1]_2) = e([1]_1, \tau_2)$. Therefore, given two tuples $(\mathsf{vk}, \tau, \boldsymbol{M}_1, \sigma_1)$ and $(\mathsf{vk}, \tau, \boldsymbol{M}_2, \sigma_2)$ that are verified as valid, then one can forge a signature under (vk, τ) for any message $\boldsymbol{M} = \alpha_1 \cdot \boldsymbol{M}_1 + \alpha_2 \cdot \boldsymbol{M}_2$ by computing $\sigma = \alpha_1 \cdot \sigma_1 + \alpha_2 \cdot \sigma_2$. The tuple $(\mathsf{vk}, \tau, \boldsymbol{M})$ is valid by the following equalities:

$$\begin{aligned} e(\sigma, \tau_2) &= e(\alpha_1 \cdot \sigma_1 + \alpha_2 \cdot \sigma_2, \tau_2) \\ &= \alpha_1 \cdot e(\sigma_1, \tau_2) + \alpha_2 \cdot e(\sigma_2, \tau_2) \\ &= \alpha_1 \sum_{i\in[\rho]} e(M_{1,i}, [s_i]_2) + \alpha_2 \sum_{i\in[\rho]} e(M_{2,i}, [s_i]_2) \\ &= \sum_{i\in[\rho]} e(\alpha_1 \cdot M_{1,i} + \alpha_2 \cdot M_{2,i}, [s_i]_2) \end{aligned}$$

D.3 Two-Dimensional Tags

We give in the body a definition that describes the notion of two-dimensional tags, which intuitively ensures that only messages signed under tags that are associated to the same position of a matrix can be combined together. Furthermore, the public randomisability and the tag-binding property still hold with respect to the row (sub-)tags.

The below unlinkability of the tag in $\mathcal{T}_1$ will ensure that a re-randomized row tag does not reveal any information about its original one, eventually allowing an unlinkable shuffle on the set of rows.

Definition 25 ($\mathcal{T}_1$-Tag Unlinkability). *Given any valid tuple $(\mathsf{vk}, \tau = (\tau_1, \tau_2), \boldsymbol{M}, \sigma)$, then the tuple $(\mathsf{vk}, \tau' = (\tau_1', \tau_2), \boldsymbol{M}, \sigma')$ generated by $\mathsf{RandTag}(\mathsf{vk}, \tau, \boldsymbol{M}, \sigma)$ has a tag τ_1' unlinkable to τ_1.*

From now on, we will refer to LHS schemes with two-dimensional tags as 2-Tag-LHS, to differentiate them from one-dimensional-tag LHS schemes.

Definition 26 (Tag-Binding Unforgeability for 2-Tag-LHS). *Given a 2-tag-LHS scheme, for any adversary $\mathcal{A}$ that, given tags and signatures on messages $(\boldsymbol{M}_i)_i$ under two-dimensional tags $(\tau_{1,i}, \tau_{2,i})_i$, both of its choice (for Chosen-Message Attacks), outputs a set of valid tuples $(\mathsf{vk}, \tau_j = (\tau_1, \tau_{2,j}), \boldsymbol{M}_j', \sigma_j)_j$ with $\tau_1 \in \mathcal{T}_1$ and each $\tau_{2,j} \in (\tau_{2,i})_i$, there must exist some tag τ_1' such that each $(\tau_1', \tau_{2,j}) \in (\tau_{1,i}, \tau_{2,i})_i$, and coefficients $(\omega_{i,j})_{i\in I_{\tau_1',\tau_{2,j}},j}$, where each $I_{\tau_1',\tau_{2,j}}$ is the set of messages already signed under $(\tau_1', \tau_{2,j})$, such that*

$$\boldsymbol{M}_j' = \sum_{i\in I_{\tau_1',\tau_{2,j}}} \omega_{i,j} \cdot \boldsymbol{M}_i$$

with overwhelming probability.

D.4 Construction of 2-Tag-LHS

We apply the generic technique of imposing a tag on the one-time LHS scheme in [35] to impose a tag on any existing LHS with a randomisable tag set, denoted as $\mathsf{LHS}_1 = (\mathsf{SetUp}_1, \mathsf{KeyGen}_1, \mathsf{NewTag}_1, \mathsf{VerifTag}_1, \mathsf{Sign}_1, \mathsf{DerivSign}_1, \mathsf{Verify}_1, \mathsf{RandTag}_1)$, as in the following.

Theorem 8 (Tag-Binding Unforgeability for 2-tag-LHS). *If LHS_1 is a tag-binding unforgeable LHS, then 2-tag-LHS, as constructed in Figure 14, is also tag-binding unforgeable under the DL assumption in the generic group model.*

Proof. We consider an adversary $\mathcal{A}$ in the tag-binding unforgeability game of 2-tag-LHS. We then construct an adversary $\mathcal{B}$ that uses $\mathcal{A}$ to either break the tag-binding unforgeability of LHS_1 or the DL assumption in the generic group model as follows:

$\underline{\mathsf{SetUp}(1^\lambda)}$: It runs $\mathsf{SetUp}_1(1^\lambda)$ to obtain pp_1, which includes the tag space $\mathcal{T}_1$ and adds the tag space $\mathcal{T}_2 = (\mathbb{Z}_p^* \times \mathbb{G}_1^*)$. It outputs $\mathsf{pp} = (\mathsf{pp}_1, \mathcal{T}_2)$.

$\underline{\mathsf{KeyGen}(\mathsf{pp}, \rho)}$: It runs $(\mathsf{sk}_1, \mathsf{vk}_1) \leftarrow \mathsf{KeyGen}_1(\mathsf{pp}_1, \rho + 3)$ and outputs $\mathsf{sk} = \mathsf{sk}_1$ and $\mathsf{vk} = \mathsf{vk}_1$.

$\underline{\mathsf{NewTag}_{\mathcal{T}_1}(\mathsf{sk})}$: It outputs a tag in $\mathcal{T}_1$ as $(\tau_1, \tilde{\tau}_1) \leftarrow \mathsf{NewTag}_1(\mathsf{sk}_1)$.

$\underline{\mathsf{NewTag}_{\mathcal{T}_2}(\mathsf{sk})}$: It chooses a scalar $w \xleftarrow{\$} \mathbb{Z}_p^*$ and an element $H \xleftarrow{\$} \mathbb{G}$, and outputs $\tilde{\tau}_2 = \tau_2 = (w, H)$.

$\underline{\mathsf{VerifTag}_{\mathcal{T}_1}(\mathsf{vk}, \tau_1)}$: It checks whether $\mathsf{VerifTag}(\mathsf{vk}, \tau_1) = 1$ or not.

$\underline{\mathsf{VerifTag}_{\mathcal{T}_2}(\mathsf{vk}, \tau_2)}$: It checks whether $\tau_2 = (w, H) \in \mathbb{Z}_p^* \times \mathbb{G}$ or not.

$\underline{\mathsf{Sign}(\mathsf{sk}, \tilde{\tau}_1, \tilde{\tau}_2 = (w, H), \boldsymbol{M})}$: It uses $\tilde{\tau}_2$ to extend $\boldsymbol{M} \in \mathbb{G}_1^\rho$ into $\boldsymbol{M}' = (\boldsymbol{M}, H, w \cdot H, w^2 \cdot H) \in \mathbb{G}_1^{\rho+3}$, and outputs $\sigma \leftarrow \mathsf{Sign}_1(\mathsf{sk}_1, \tilde{\tau}_1, \boldsymbol{M}')$.

$\underline{\mathsf{DerivSign}(\mathsf{vk}, \tau_1, \tau_2 = (w, H), \{\omega_i, \boldsymbol{M}_i, \sigma_i\}_{i \in [\ell]})}$: It extends each $\boldsymbol{M}_i$ into the signed message $\boldsymbol{M}'_i = (\boldsymbol{M}_i, H, w \cdot H, w^2 \cdot H)$ in σ_i and computes $\sigma \leftarrow \mathsf{DerivSign}_1(\mathsf{vk}_1, \tau_1, \{\omega_i, \boldsymbol{M}'_i, \sigma_i\}_{i \in [\ell]})$ and $\tau'_2 = (w, H' = \sum_i \omega_i \cdot H)$. It outputs $(\sigma, \tau_1, \tau'_2)$.

$\underline{\mathsf{Verify}(\mathsf{vk}, \tau_1, \tau_2 = (w, H), \boldsymbol{M}, \sigma)}$: It extends $\boldsymbol{M}$ into $\boldsymbol{M}' = (\boldsymbol{M}, H, w \cdot H, w^2 \cdot H)$ and checks whether $\mathsf{VerifTag}_{\mathcal{T}_1}(\mathsf{vk}, \tau_1) = 1$ and $\mathsf{VerifTag}_{\mathcal{T}_2}(\mathsf{vk}, \tau_2) = 1$ and $\mathsf{Verify}_1(\mathsf{vk}_1, \tau_1, \boldsymbol{M}', \sigma) = 1$ or not.

$\underline{\mathsf{RandTag}(\mathsf{vk}, \tau_1, \tau_2 = (w, H), \boldsymbol{M}, \sigma)}$: It extends $\boldsymbol{M}$ into $\boldsymbol{M}' = (\boldsymbol{M}, H, w \cdot H, w^2 \cdot H)$ and randomises τ_1 by running $(\tau'_1, \sigma') \leftarrow \mathsf{RandTag}_1(\mathsf{vk}_1, \tau_1, \boldsymbol{M}', \sigma)$. It outputs (τ'_1, τ_2) and σ'.

Fig. 14. Construction of LHS with two-dimensional tags.

- since the $\mathcal{T}_2$-tags are fully public, any $\mathsf{NewTag}_{\mathcal{T}_2}$-query is answered by a random pair (w, H);
- any $\mathsf{NewTag}_{\mathcal{T}_1}$-query is answered by forwarding a query to NewTag_1 of the LHS_1 security game;
- any Sign-query on $(\tau_1, \tau_2 = (w, H), \boldsymbol{M})$ is answered by first extending $\boldsymbol{M}$ into $\boldsymbol{M}' = (\boldsymbol{M}, w, w \cdot H, w^2 \cdot H)$ and forwarding $(\tau_1, \boldsymbol{M}')$ to the Sign_1-query of the LHS_1 security game.

We then denote by $(\boldsymbol{M}_i)_i$ the set of queried messages under two-dimensional tags $(\tau_{1,i}, \tau_{2,i} = (w_i, H_i))_i$, of which both are chosen by $\mathcal{A}$ and tags of either the first component or the second component are possibly equal across i when the same tag is used several times. Furthermore, $(\boldsymbol{M}'_i)_i$ are the extended messages of $(\boldsymbol{M}_i)_i$ with their corresponding tags.

Upon receiving the valid 2-tag-LHS forgeries $(\mathsf{vk}, \tau_j = (\tau_1, \tau_{2,j}), \boldsymbol{M}^\star_j, \sigma_j)_j$ from $\mathcal{A}$ where $\tau_1 \in \mathcal{T}_1$ and each $\tau_{2,j}$ is already queried, $\mathcal{B}$ extends $\boldsymbol{M}^\star_j$ into $\boldsymbol{M}^{\star\prime}_j$ by using $\tau_{2,j} = (w^\star_j, H^\star_j)$ and outputs valid LHS_1 forgeries $(\mathsf{vk}, \tau_1, \boldsymbol{M}^{\star\prime}_j, \sigma_j)_j$. Unless the tag-binding unforgeability of LHS_1 is broken, there must exist a tag τ'_1 queried to Sign_1 and coefficients $(\omega_{i,j})_{i \in I_{\tau'_1}, j}$ where $I_{\tau'_1}$ is the index set of extended messages already signed under τ'_1, such that

$$\boldsymbol{M}^{\star\prime}_j = \sum_{i \in I_{\tau'_1}} \omega_{i,j} \cdot \boldsymbol{M}'_i \tag{2}$$

with overwhelming probability. It thus suffices to restrict the index set in the last equality from $I_{\tau'_1}$ to $I_{\tau'_1, \tau_{2,j}}$, where the latter is the index set of messages already signed under $(\tau'_1, \tau_{2,j})$.

The last three components of the LHS vector and the RHS vector in Equality (2) give

$$(H^\star_j, w^\star_j \cdot H^\star_j, w^{\star 2}_j \cdot H^\star_j) = \left(\sum_{i \in I_{\tau'_1}} \omega_{i,j} \cdot H_i, \sum_{i \in I_{\tau'_1}} \omega_{i,j} \cdot (w_i \cdot H_i), \sum_{i \in I_{\tau'_1}} \omega_{i,j} \cdot (w_i^2 \cdot H_i) \right).$$

This means that the LHS_1-forgery on j (the RHS triple) forms a square Diffie-Hellman triple. We assume that the $\mathcal{T}_2$-tags that are queried with τ_1 are contained by the following set of *different* items $(\tau_{2,k} = (w_k, H_k))_k$. By combining the identical $\mathcal{T}_2$-tags in the RHS together, and so by summing in

each β_k the coefficients $(\omega_{i,j})_i$ that correspond to the same triple $(H_k, w_k \cdot H_k, w_k^2 \cdot H_k)$, we then have that the RHS is equal to

$$\left(\sum_k \beta_k \cdot H_k, \sum_k \beta_k \cdot (w_k \cdot H_k), \sum_k \beta_k \cdot (w_k^2 \cdot H_k)\right).$$

From Theorem 6, unless the DL assumption is broken, at most one coefficient is non-zero: either none or β_K, and so at most one $\mathcal{T}_2$-tag is represented: none or (w_K, H_K). Hence $\boldsymbol{M}_j^\star$ is either $(1, \ldots, 1)$ or $\sum_{i \in I_{\tau_1', \tau_{2,K}}} \omega_{i,j} \cdot \boldsymbol{M}_i$. If the latter case happens, then one has

$$(H_j^\star, w_j^\star \cdot H_j^\star, w_j^{\star 2} \cdot H_j^\star) = \big(\beta_K \cdot H_K, \beta_K \cdot (w_K \cdot H_K), \beta_K \cdot (w_K^2 \cdot H_K)\big)$$

which implies $w_K = w_j^\star$. This concludes $\boldsymbol{M}_j^\star = \sum_{i \in I_{\tau_1', \tau_{2,j}}} \omega_{i,j} \cdot \boldsymbol{M}_i$. □

E More on Offline Protocols for Voting

E.1 Security Notions for Offline Protocols

For the security of OMIX and OPAD in the multi-server setting, we will introduce two notions, namely soundness and unlinkability, which are adapted for the context of mixing eligible voters' public keys, from those in [35]. Intuitively, soundness ensures that for any execution of the protocol which verifies as correct, the output matrix of ciphertexts is equivalent to the input one (e.g up to an authorized transformation). Privacy, on the other hand, ensures that the output matrix is unlinkable from the transformation that is applied to the input matrix.

Definition 27 (Soundness). *An offline protocol $\mathcal{P}_{OFF}$ between multiple servers for a family of authorized transformation sets $\{\mathcal{T}_{\lambda,n}\}_{\lambda,n}$ is said to be sound in the certified key setting, if any PPT adversary $\mathcal{A}$ has a negligible success probability in the following experiment:*

1. *On input a security parameter λ and a number of users n, the challenger generates public parameters of $\mathcal{P}_{OFF}$ as pp and master keys as $(\mathsf{msk}, \mathsf{mpk})$. It provides $(\mathsf{pp}, \mathsf{mpk})$ to $\mathcal{A}$.*
2. *Upon receiving $(\mathsf{pp}, \mathsf{mpk})$, the adversary $\mathcal{A}$ does as follows:*
   - *It determines the set of corrupted users $\mathcal{CS} \subset [n]$ and generates itself their public keys $(\mathsf{pk}_i)_{i \in \mathcal{CS}}$.*
   - *For $i \in \mathcal{CS}$, it proves knowledge of the secret keys sk_i for pk_i to obtain certifications $\mathcal{CA}.\Sigma_i$ (which is generated by using msk) from the challenger.*
3. *The challenger generates the keys of the honest users $(\mathsf{sk}_i, \mathsf{pk}_i)_{i \in [n] \setminus \mathcal{CS}}$ and certifies their public keys by generating $(\mathcal{CA}.\Sigma_i)_{i \in [n] \setminus \mathcal{CS}}$.*
4. *The initial mix box is thus defined by $\mathsf{MBox} = (\mathcal{CA}.\Sigma_i)_{i \in [n]}$.*
5. *The adversary mixes MBox in a provable way, resulting in $(\mathsf{MBox}', \mathsf{proof})$.*

The adversary wins if $\mathsf{MixVerif}\big(\mathsf{MBox}, \mathsf{MBox}', \mathsf{proof}\big) = 1$, but there does not exist an authorized transformation $T \in \mathcal{T}_{\lambda,n}$ such that

$$\mathsf{MBox}' = T(\mathsf{MBox})$$

Based on the definition above, our offline mix-net protocol assumes that all user inputs are encapsulated in their public keys and certified by the authority before being added to the mix box. Inputs in the voting context are functional keys, which are mixed and eventually decrypted into functional keys for permutation. We emphasize that this certification only occurs in the offline phase of voting, and voters are not required to interact with the authority each time they vote.

The protocol's soundness will be determined by a family of authorized transformation sets, specified by the protocol itself to define which operations the mix servers are allowed to perform from the certified inputs in the initial mix box.

Besides soundness, we introduce the following privacy game, where the adversary is allowed to generate two transformation sets of its choice. The challenger then picks a transformation in one of these sets and challenges the adversary to guess from which set the transformation was applied.

Definition 28 (Unlinkability). *An offline $\mathcal{P}_{OFF}$ for a family of authorized transformation sets $\{\mathcal{T}_{\lambda,n}\}_{\lambda,n}$ is said to provide unlinkability in the certified key setting, if any PPT adversary $\mathcal{A}$ has no more than a negligible advantage in guessing b in the following security game:*

1. *On input a security parameter λ, a number of users n, a number of mix-servers N, the challenger generates public parameters of $\mathcal{P}_{OFF}$ as pp and master keys $(\mathsf{msk}, \mathsf{mpk})$. It provides $(\mathsf{pp}, \mathsf{mpk})$ to $\mathcal{A}$.*
2. *Upon receiving $(\mathsf{pp}, \mathsf{mpk})$, the adversary $\mathcal{A}$ does as follows:*
   - *It determines the set of corrupted users $\mathcal{CS} \subset [n]$ and generates itself their public keys $(\mathsf{pk}_i)_{i\in\mathcal{CS}}$.*
   - *For $i \in \mathcal{CS}$, it proves knowledge of the secret keys sk_i for pk_i to obtain certifications $\mathcal{CA}.\Sigma_i$ (which is generated by using msk) from the challenger.*
   - *It determines the set of corrupted mix-servers $\mathcal{J} \subset [N]$ and generates itself their keys $(\mathsf{VK}_s)_{s\in\mathcal{J}}$.*
3. *The challenger randomly chooses a bit $b \xleftarrow{\$} \{0,1\}$. It generates the keys of the honest users $(\mathsf{sk}_i, \mathsf{pk}_i)_{i\in[n]\setminus\mathcal{CS}}$ and the keys of the honest mix-servers $(\mathsf{SK}_s, \mathsf{VK}_s)_{s\in[N]\setminus\mathcal{J}}$. It certifies their public keys and generates $(\mathcal{CA}.\Sigma_i)_{i\in[n]\setminus\mathcal{CS}}$. The initial mix box is thus defined by $\mathsf{MBox}^{(0)} = (\mathcal{CA}.\Sigma_i)_{i\in[n]}$. The challenger enters into a loop for $s \in [N]$ with $\mathcal{A}$:*
   - *if $s \in \mathcal{J}$, $\mathcal{A}$ builds itself the new box $\mathsf{MBox}^{(s)}$ with the proof $\mathsf{proof}^{(s)}$;*
   - *if $s \notin \mathcal{J}$, $\mathcal{A}$ provides two transformation sets $\mathcal{T}_{s,0}$ and $\mathcal{T}_{s,1}$ of its choice (typically defined by a polynomial-sized string). Then the challenger applies a transformation T from $\mathcal{T}_{s,b}$ to get $\mathsf{MBox}^{(s)} = T(\mathsf{MBox}^{(s-1)})$, and provides the proof $\mathsf{proof}^{(s)}$.*

   In the end, the adversary outputs its guess b' for b. The experiment outputs 1 if $b' = b$ and 0 otherwise.

E.2 Deferred Proofs for OMIX

We provide proofs for the soundness and the unlinkability below.

Theorem 9 (OMIX Soundness). *Our OMIX protocol is designed for the family of authorized tranformation sets $\mathcal{T}_{\lambda,n}$ such that each transformation $T \in \mathcal{T}_{\lambda,n}$ is specified by a permutation on the set of rows $\Pi : [n] \to [n]$ and randomness $(\gamma_{\mathsf{PKE},j,i})_{j,i\in[n]}$ such that if $\mathsf{MBox}' = T(\mathsf{MBox})$ then*

$$C'_{\Pi(j),i} = C_{j,i} + \gamma_{\mathsf{PKE},j,i} \cdot C_{\mathsf{PKE}}$$

for all $i, j \in [n]$ where $(C_{j,i})_{j,i\in[n]}$ and C_{PKE} are in MBox and $(C'_{j,i})_{j,i\in[n]}$ are in MBox'. The protocol has soundness in the random oracle model, under the tag-binding unforgeability property of the 2-tag-LHS scheme, the soundness of the $\mathsf{NIZK}_{\mathsf{DH}}$ system and the discrete logarithm assumption in the group $\mathbb{G}_1$.

Proof. As the verification is valid, one can further parse

$$\mathsf{MBox} = \{(C_{j,i}, \sigma_{j,i}, \Sigma_{\mathsf{PKE},j,i})_{j,i\in[n]}, (\tau_j, G_{1,j}, \Sigma_{\boldsymbol{G}_j})_{j\in[n]}, (\theta_i)_{i\in[n+1]}, G_0, C_{\mathsf{PKE}}\};$$
$$\mathsf{MBox}' = \{(C'_{j,i}, \sigma'_{j,i}, \Sigma'_{\mathsf{PKE},j,i})_{j,i\in[n]}, (\tau'_j, G'_{1,j}, \Sigma'_{\boldsymbol{G}_j})_{j\in[n]}, (\theta_i)_{i\in[n+1]}, G'_0, C_{\mathsf{PKE}}\};$$

along with the signed proof $(\mathsf{proof}, \mathsf{sig}_{\mathsf{mix}})$. When the hash function H is modeled as a random oracle $\mathcal{RO}$, with an overwhelming probability, the challenger in the soundness game can answer $G_0 \xleftarrow{\$} \mathbb{G}_1$ to the query $\mathcal{RO}(0)$, and $G_{1,j} := \alpha_j \cdot G_0$ to the query $\mathcal{RO}(j)$ where $(\alpha_j)_{j\in[n]}$ are pairwise distinct randomness. The soundness holds by the following verifications:

1. $\mathsf{Sig.Verify}(\mathsf{VK}^{\mathsf{mix}}, \mathsf{proof}, \mathsf{sig}_{\mathsf{mix}}) = 1$ guarantees that the mixing process is executed by the shuffling server that owns $\mathsf{SK}^{\mathsf{mix}}$[13];
2. For each $(j, i) \in [n] \times [n]$, one has

   $$\text{2-tag-LHS.Verify}\left(\mathsf{LHS.vk}, (\tau'_j, \theta_i), ([\![1]\!]_1, C'_{j,i}), \sigma'_{j,i}\right) = 1.$$

   By the tag-binding unforgeability of the 2-tag-LHS scheme, there exists a tag τ_{k_j} in the initial MBox such that

   $$([\![1]\!]_1, C'_{j,i}) = \phi_{k_j,i} \cdot ([\![1]\!]_1, C_{k_j,i}) + \gamma_{\mathsf{PKE},k_j,i} \cdot ([\![0]\!]_1, C_{\mathsf{PKE}})$$

   which implies that $\phi_{k_j,i} = 1$ and then $C'_{j,i} = C_{k_j,i} + \gamma_{\mathsf{PKE},k_j,i} \cdot C_{\mathsf{PKE}}$. It remains to prove that $\{k_j\}_{j\in[n]} = \{j\}_{j\in[n]}$.

[13] In case the adversary corrupts one of multiple servers, the mix is identified.

3. For each $j \in [n]$, one has

$$\text{2-tag-LHS.Verify}\left(\mathsf{LHS.vk}, (\tau'_j, \theta_{n+1}), (G'_0, G'_{1,j}), \Sigma'_{\boldsymbol{G}_j}\right) = 1.$$

   Also by the tag-binding unforgeability, this implies that $(G'_0, G'_{1,j}) = (\beta \cdot G_0, \beta \cdot G_{1,k_j})$ for some β.
4. The same β is ensured by a valid proof and further satisfies that

$$G'_0 = \beta \cdot G_0; \qquad \sum_{j\in[n]} G'_{1,j} = \beta \cdot \sum_{j\in[n]} G_{1,j}$$

   Then one has

$$\sum_{j\in[n]} G_{1,k_j} = \sum_{j\in[n]} G_{1,j}.$$

   If $\{k_j\}_{j\in[n]} \neq \{j\}_{j\in[n]}$, one can use this game to solve a DLOG instance that is embedded by guessing as $G_{1,j^\star}$, in which a correct guess means that $G_{1,j^\star}$ does not appear in $\{G_{1,k_j}\}_{j\in[n]}$.

Therefore, with an overwhelming probability, one has $\{k_j\}_{j\in[n]} = \{j\}_{j\in[n]}$, which implies that there exists a permutation $\Pi : [n] \to [n]$ such that $C'_{\Pi(j),i} = C_{j,i} + \gamma_{\mathsf{PKE},j,i} \cdot C_{\mathsf{PKE}}$. □

In the following remarks, we demonstrate two methods by which any party acting as a challenger in the OMIX protocol can extract the permutation applied by an adversary (playing the role of a corrupted mix-server). These methods can be interleaved during the simulation in the unlinkability proof, ensuring that permutations applied by honest servers remain private, while those applied by the corrupted servers are always extractable. This strong security property is essential for proving ballot privacy in an OMIX-based voting system with multiple (possibly corrupted) mix-servers.

Remark 5 (Permutation Extractor with $\mathcal{RO}$). In the soundness proof, using the knowledge of $(\alpha_j)_{j\in[n]}$ that are assigned with the $\mathcal{RO}$ queries on input $(j)_{j\in[n]}$, the challenger can ensure the knowledge soundness of the mix by extracting the permutation applied by the adversary. Furthermore, this knowledge soundness does not rely on the $\mathsf{NIZK}_{\mathsf{DH}}$ proof system. The extraction proceeds as follows:

- Under the soundness of the verifications (with 2-tag-LHS and standard signature) at points 1,2, and 3, the challenger has
$$(G'_0, G'_{1,j}) = (\beta \cdot G_0, \beta \cdot G_{1,k_j})$$
for some β and all $j \in [n]$.
- For each j, it searches α_{κ_j} with $\kappa_j \in [n]$ such that $\alpha_{\kappa_j} \cdot G'_0 = G'_{1,j}$. If yes, then $k_j = \kappa_j$, the challenger stores (j, κ_j). Otherwise, the mix is not correct.
- It tests if $\{\kappa_j\}_{j\in[n]} = \{j\}_{j\in[n]}$. If no, the mix is not correct. If yes, this implies $\{k_j\}_{j\in[n]} = \{j\}_{j\in[n]}$ and then the challenger knows the permutation $\Pi : k_j \to j$ that was applied by the adversary.

To differentiate with the row (verification) keys that are hash outputs in $\mathbb{G}_1$, we will call the exponents $(\alpha_j)_{j\in[n]}$ as the row secret keys.

Remark 6 (Permutation Extractor with Decryption Key). In the soundness proof, using the decryption key $\mathsf{PKE.sk}$ of the PKE scheme and the extracted secret keys sk_i of the corrupted voters, the challenger can ensure the knowledge soundness of the mix by extracting the permutation applied by the adversary. Furthermore, this knowledge soundness does not rely on the $\mathsf{NIZK}_{\mathsf{DH}}$ proof system. The extraction proceeds as follows:

- Given the encrypted functional keys $(\mathsf{sk}_i = \mathsf{ip.dk}^i_{(1,0)}, \mathsf{ip.dk}^i_{(0,0)})$[14] submitted as mix inputs from the voters. Under the soundness of the verifications (with 2-tag-LHS and standard signature) at points 1,2, and 3, the challenger has
$$C'_{j,i} = C_{k_j,i} + \gamma_{\mathsf{PKE},k_j,i} \cdot C_{\mathsf{PKE}}.$$

[14] Note that $\mathsf{ip.dk}^i_{(1,0)}$ and $\mathsf{ip.dk}^i_{(0,0)}$ need not be valid functional keys but must differ; this can be checked during certification. This condition is usually ensured in our voting protocol by a proof of correct generation for functional keys.

– The challenger decrypts each $C'_{j,i}$ into $m'_{j,i}$. Then one has

$$m'_{j,i} := \mathsf{PKE.Dec}(\mathsf{PKE.sk}, C_{k_j,i}),$$

which means $m'_{j,i} = \mathsf{ip.dk}^i_{(1,0)}$ if $k_j = i$; and $m'_{j,i} = \mathsf{ip.dk}^i_{(0,0)}$ if $k_j \neq i$.
– By decrypting $\{C'_{j,i}\}_{j,i\in[n]}$ into $\{m'_{j,i}\}_{j,i\in[n]}$, the challenger can test if $\boldsymbol{M} = \{m'_{j,i}\}_{j,i\in[n]}$ is a matrix resulted by applying some permutation $\Pi : k_j \to j$ on the rows of the identity matrix that the functional keys are associated with during the certification phase. If no, the mix is not correct. If yes, this implies Π was applied by the adversary.

Theorem 10 (OMIX Unlinkability). *Our OMIX protocol is designed for the family of authorized transformation sets $\mathcal{T}_{\lambda,n}$ such that each $T \in \mathcal{T}_{\lambda,n}$ is specified by a permutation on the set of rows $\Pi : [n] \to [n]$ and scalars $\gamma_{\mathsf{PKE},j,i} \in \mathbb{Z}_p$ such that if $\mathsf{MBox}' = T(\mathsf{MBox})$ then*

$$C'_{\Pi(j),i} = C_{j,i} + \gamma_{\mathsf{PKE},j,i} \cdot C_{\mathsf{PKE}}$$

for all $i, j \in [n]$ where $C_{j,i}$ and C_{PKE} are in MBox and $C'_{\Pi(j),i}$ are in MBox'. The challenge pairs of transformation sets are $\mathcal{T}_{s,0}$ and $\mathcal{T}_{s,1}$ that correspond to transformations specified by permutations $\Pi_{s,0}$ and $\Pi_{s,1}$ respectively. The protocol then has unlinkability in the random oracle model under the DDH assumption in the group $\mathbb{G}_1$, the tag randomizability of the 2-tag-LHS scheme and the zero-knowledge property of the $\mathsf{NIZK}_{\mathsf{DH}}$ system.

Proof. We will prove this theorem using the following series of games. The changes between games occur in the OMIX.Shuffle algorithm and are described in Figure 15.

Game $\mathbf{G}_0$: This is the real game, as in Definition 28. The security game is specified by the protocol as follows:
– The challenger runs the OMIX.SetUp algorithm to obtain public parameters pp and generates the signing key $(\mathsf{LHS.sk}, \mathsf{LHS.vk})$ in OMIX.KeyGen. It stores $\mathsf{msk} = \mathsf{LHS.sk}$, and passes pp and $\mathsf{mpk} = \mathsf{LHS.vk}$ to $\mathcal{A}$. In the random oracle model, with an overwhelming probability, we note that challenger can answer $G_0 \xleftarrow{\$} \mathbb{G}_1$ to the query $\mathcal{RO}(0)$, and $G_{1,j} := \alpha_j \cdot G_0$ to the query $\mathcal{RO}(j)$ where $(\alpha_j)_{j\in[n]}$ are pairwise distinct randomness.
– The adversary chooses a set of corrupted users $\mathcal{CS} \subset [n]$ and a set of corrupted mix-servers $\mathcal{J} \subset [N]$. Under the proof of knowledge for certification, the challenger receives the following keys

$$\mathsf{sk}_i := \mathsf{ip.dk}^i_{(1,0)};\ \ \mathsf{pk}_i := (\mathsf{id}_i, \mathsf{ct}_i, \mathsf{ip.dk}^i_{(0,1)}, \mathsf{ip.dk}^i_{(0,0)}, \pi_i^{\mathsf{pk}})$$

and keys for the corrupted mix-servers $(\mathsf{SK}^{\mathsf{mix}}_s)_{s\in\mathcal{J}}$.
– The challenger chooses a bit $b \xleftarrow{\$} \{0,1\}$. It generates keys for honest users $(\mathsf{pk}_i, \mathsf{sk}_i)_{i\in[n]\setminus\mathcal{CS}}$ and keys for honest mix-servers $(\mathsf{SK}^{\mathsf{mix}}_s, \mathsf{VK}^{\mathsf{mix}}_s)_{s\in[N]\setminus\mathcal{J}}$. It outputs the initial mix box $\mathsf{MBox}^{(0)}$ as

$$\{(C^{(0)}_{j,i}, \sigma^{(0)}_{j,i}, \Sigma^{(0)}_{\mathsf{PKE},j,i})_{j,i\in[n]}, (\tau^{(0)}_j, G^{(0)}_{1,j}, \Sigma^{(0)}_{\boldsymbol{G}_j})_{j\in[n]}, (\theta_i)_{i\in[n+1]}, G^{(0)}_0, C_{\mathsf{PKE}}\}$$

by running $\mathsf{OMIX.Certify}(\mathsf{msk}, (\mathsf{pk}_i)_{i\in[n]}, (\mathsf{VK}^{\mathsf{mix}}_s)_{s\in[N]})$.
– The challenger enters into a loop for $s \in [N]$ with $\mathcal{A}$:
  • if $s \in \mathcal{J}$, $\mathcal{A}$ provably builds itself the new box $\mathsf{MBox}^{(s)}$ with $\mathsf{proof}^{(s)}, \mathsf{sig}^{(s)}$;
  • if $s \notin \mathcal{J}$, when $\mathcal{A}$ provides two permutations over the set of rows $\Pi_{s,0}$ and $\Pi_{s,1}$, the challenger transforms the ballot $\mathsf{MBox}^{(s-1)}$ into $\mathsf{MBox}^{(s)}$ using $\Pi_{s,b}$ as in the

$$\mathsf{OMIX.Shuffle}(\mathsf{SK}^{\mathsf{mix}}_s, \mathsf{MBox}^{(s-1)}, (\mathsf{proof}^{(k)}, \mathsf{sig}^{(k)}_{\mathsf{mix}})_{k\in[s-1]})$$

algorithm.

Game $\mathbf{G}_1$: In this game, we switch the $\mathsf{NIZK}_{\mathsf{DH}}$ proof system for

$$(G_0, G^{(s)}_0, \sum_{j\in[n]} G_{1,j}, \sum_{j\in[n]} G^{(s)}_{1,j})$$

to the zero-knowledge setting (proving without knowing β): the permutation over the set of rows can be tested and extracted using either the row secret keys or the PKE decryption key, as presented in Remark 5 and Remark 6. Any adversary that can distinguish between $\mathbf{G}_0$ and $\mathbf{G}_1$ is a distinguisher between the settings of the zero-knowledge proofs. The input box to the considered honest server in Lemma 2 is thus correctly transformed from the initial box with overwhelming probability.

Game $\mathbf{G}_2$: In this game, all the honest mix-servers generate random row tags $(\tau_j^\star)_{j\in[n]}$, random encryptions $(C'_{j,i})_{j,i\in[n]}$ of $\mathbf{0}$, for all the users, random row verification keys $(G'_0, (G'_{1,j})_{j\in[n]})$ and generate the signatures using the signing key $\mathsf{LHS.sk}$. Then, they apply the permutation $\Pi_{s,b}$ on the set of rows. In this last game, $\Pi_{s,b}$ is applied on the rows of random encryptions, random row verification keys, and random row tags. The resulting box is then identical to the one from applying $\Pi_{s,1-b}$, which implies that the advantage of any adversary in this game is 0.

We continue to prove the theorem by having the following lemma.

Lemma 2. *Under the DDH Assumption in $\mathbb{G}_1$, the adversarial views in $\mathbf{G}_1$ and $\mathbf{G}_2$ are computationally indistinguishable.*

In the next hybrid games, from $\mathbf{G}_{1.0}(s_l)$ to $\mathbf{G}_{1.3}(s_l)$, for $l = N - |J|, \ldots, 1$ and each mix-server s_l is honest, we have the input ballots in $\mathsf{MBox}^{(s_l-1)}$ contain proper permutations of randomized ciphertexts from the initial box, as nothing is modified before the game $\mathbf{G}_1$ except the zero-knowledge setting. In the following series of hybrid games, for index s_l, the honest mix-servers up to the $s_l - 1$-th round play as in $\mathbf{G}_1$ and from the $s_l + 1$-th round, they play as in $\mathbf{G}_2$. Only the behaviour of the s_l-th mix-server is modified, starting from an honest behavior. Hence, $\mathbf{G}_{1.0}(s_{N-|J|}) = \mathbf{G}_1$ and $\mathbf{G}_{1.3}(s_1) = \mathbf{G}_2$.

Game $\mathbf{G}_{1.0}(s_l)$: In this game, we assume that the initial ballot-box has been correctly generated, and mixing steps up to $\mathsf{MBox}^{(s_l)}$ have been honestly generated (except the zero-knowledge proofs that have been simulated). The next rounds are generated as in $\mathbf{G}_2$.

Game $\mathbf{G}_{1.1}(s_l)$: We modify the signature adaptation and the randomization of the row tags by the s_l-th mix-server. The mixing step in $\mathbf{G}_{1.0}(l)$ consists in updating the ciphertexts, adapting their signatures accordingly, and then re-randomizing the row tags from $(\tau_j)_{j\in[n]}$ into $(\tau_j^\star)_{j\in[n]}$. In this game, the s_l-th honest mix-server uses $\mathsf{LHS.sk}$ to directly signs the updated ciphertexts under freshly generated row tags $(\tau_j^\star)_{j\in[n]}$. The indistinguishability between $\mathbf{G}_{1.0}(s_l)$ and $\mathbf{G}_{1.1}(s_l)$ is perfect, thanks to the $\mathcal{T}_1$-Tag Unlinkability in the 2-tag-LHS scheme.

Game $\mathbf{G}_{1.2}(s_l)$: We modify the randomization of the row keys $(G_0, (G_{1,j})_{j\in[n]})$ by the s_l-th mix-server. The mixing step in $\mathbf{G}_{1.1}(s_l)$ consists in updating $(G_0, (G_{1,j})_{j\in[n]})$ into $(G'_0 = \beta \cdot G_0, (G'_{1,j} = \beta \cdot G_{1,j})_{j\in[n]})$ for some random β. In this game, the s_l-th honest mix-server generates randomly $G'_0 \xleftarrow{\$} \mathbb{G}_1$ and $G'_{1,j} \xleftarrow{\$} \mathbb{G}_1$ for all $j \in [n]$. We note that the Diffie-Hellman proof system is already in the zero-knowledge setting in this game, therefore generating a valid proof without knowing the factor β is feasible.
The indistinguishability between $\mathbf{G}_{1.1}(s_l)$ and $\mathbf{G}_{1.2}(s_l)$ is implied by the Multi-DDH Assumption, where the adversary receiving an instance $(X, (Y_j, Z_j)_{j\in[n]})$ can play as the unlinkability challenger and embed the initial row keys in $\mathsf{MBox}^{(0)}$ as $(G_0, (G_{1,j} := Y_j)_{j\in[n]})$ and the output row keys by the s_l-th mix-server as $(G'_0 := X, (G'_{1,j} := Z_j)_{j\in[n]})$.
During the above simulation, the challenger uses the PKE decryption key to verify the corrupted mix-servers and extract their permutations until the mixing round of the next honest mix-server s_{l+1}. Starting from the round of s_{l+1}, the honest mix-servers will play as in $\mathbf{G}_2$, so extracting the corrupted permutations by using the freshly-generated row secret keys is feasible.

Game $\mathbf{G}_{1.3}(s_l)$: We modify the randomization of the encryptions $(C_{j,i})_{j,i\in[n]}$ by the s_l-th mix-server. The mixing step in $\mathbf{G}_{1.2}(s_l)$ consists in updating $(C_{j,i})_{j,i\in[n]}$ into $(C'_{j,i} := C_{j,i} + \gamma_{\mathsf{PKE},j,i} \cdot C_{\mathsf{PKE},i})_{j,i\in[n]}$ for some random factors $(\gamma_{\mathsf{PKE},j,i})_{j,i\in[n]}$. In this game, the s_l-th honest mix-server generates

$$C'_{j,i} = \mathsf{PKE.Enc}(\mathsf{PKE.pk}, \mathbf{0}; r'_{j,i})$$

for all $j, i \in [n]$. As long as the re-randomization is equivalent to a re-encryption, the indistinguishability between $\mathbf{G}_{1.2}(s_l)$ and $\mathbf{G}_{1.3}(s_l)$ is implied by the IND-CPA of the underlying PKE scheme. During this simulation, the challenger uses row secret keys to verify and extract corrupted permutations.

As the proof of the lemma is complete, the proof of the theorem is also complete. □

Corollary 1 (OMIX Strong Unlinkability). *The unlinkability of the OMIX protocol holds even if, prior to outputting its guess on the challenge bit (as defined in Definition 28), the adversary $\mathcal{A}$ is provided with an oracle $\mathsf{OCorPermute}$ that returns the permutations Π_s applied by the corrupted servers $s \in \mathcal{J}$ during the mixing loop with the challenger.*

Proof. The corollary follows directly from the proof of Theorem 10. Specifically, the challenger returns the corrupted permutations it has extracted by interleaving the extraction techniques described in Remark 5 and Remark 6. □

E.3 OPAD for Randomness Generating

We provide a construction for OPAD with multiple servers in Figure 16, and prove the soundness and unlinkability within the offline-protocol framework.

Theorem 11 (OPAD Soundness). *Our OPAD protocol is designed for the family of authorized transformation sets $\mathcal{T}_{\lambda,n}$ such that each $T \in \mathcal{T}_{\lambda,n}$ is specified by one-time pads $(z_j)_{j\in[n]}$ and key randomness $(\gamma_{\mathsf{IPE},j,i})_{j,i\in[n]}$, along with a set of implicit PKE randomness, such that if $\mathsf{OBox}' = T(\mathsf{OBox})$ then $\mathsf{OBox}' = \mathsf{OBox}||(C_{j,i})_{j,i\in[n]}, (\mathsf{ct}_j^{\mathsf{PAD}})_{j\in[n]}$ where*

$$C_{j,i} = \mathsf{PKE.Enc}(\mathsf{PKE.pk}, \gamma_{\mathsf{IPE},j,i} \cdot \mathsf{ip.dk}^i_{(0,0)} + z_j \cdot \mathsf{ip.dk}^i_{(0,1)})$$

and

$$\mathsf{ct}_j^{\mathsf{PAD}} = \mathsf{PKE.Enc}(\mathsf{PKE.pk}, [\![z_j]\!]_1) \ \forall j \in [n]$$

for all $i, j \in [n]$. In this protocol, $\mathsf{PKE.pk}$ is given as a public parameter, and $(\mathsf{ip.dk}^i_{(0,0)}, \mathsf{ip.dk}^i_{(0,1)})$ are given as part of the user public key. The protocol has soundness under that of the $\mathsf{NIZK}_{\mathsf{Gen}}$ system.

Proof. As the soundness of $\mathsf{NIZK}_{\mathsf{Gen}}$ ensures the correct generation of each $\mathsf{ct}_j^{\mathsf{PAD}}$ and each $C_{j,i}$, the proof is complete. □

Theorem 12 (OPAD Unlinkability). *Our OPAD protocol is designed for the family of authorized transformation sets $\mathcal{T}_{\lambda,n}$ such that each $T \in \mathcal{T}_{\lambda,n}$ is specified by one-time pads $(z_j)_{j\in[n]}$ and key randomness $(\gamma_{\mathsf{IPE},j,i})_{j,i\in[n]}$, along with a set of implicit PKE randomness, such that if $\mathsf{OBox}' = T(\mathsf{OBox})$ then $\mathsf{OBox}' = \mathsf{OBox}||(C_{j,i})_{j,i\in[n]}, (\mathsf{ct}_j^{\mathsf{PAD}})_{j\in[n]}$ where*

$$C_{j,i} = \mathsf{PKE.Enc}(\mathsf{PKE.pk}, \gamma_{\mathsf{IPE},j,i} \cdot \mathsf{ip.dk}^i_{(0,0)} + z_j \cdot \mathsf{ip.dk}^i_{(0,1)})$$

and

$$\mathsf{ct}_j^{\mathsf{PAD}} = \mathsf{PKE.Enc}(\mathsf{PKE.pk}, [\![z_j]\!]_1) \ \forall j \in [n]$$

for all $i, j \in [n]$. In this protocol, $\mathsf{PKE.pk}$ is given as a public parameter, and $(\mathsf{ip.dk}^i_{(0,0)}, \mathsf{ip.dk}^i_{(0,1)})$ are given as part of the user public key. The challenge pairs of transformation sets are $\mathcal{T}_{s,0}$ and $\mathcal{T}_{s,1}$ that correspond to transformations specified by $(z_j^0, \gamma^0_{\mathsf{IPE},j,i})_{j,i\in[n]}$ and $(z_j^1, \gamma^1_{\mathsf{IPE},j,i})_{j,i\in[n]}$ respectively. The protocol then has unlinkability under the IND-CPA of the PKE scheme, and the zero-knowledge properties of the $\mathsf{NIZK}_{\mathsf{Gen}}$ and $\mathsf{NIZK}_{\mathcal{R}_{\mathsf{FS}}}$ systems.

Proof. We will prove this theorem using the following series of games.

Game $\mathbf{G}_0$: This is the real game, with the challenge bit b, as defined in Definition 28.
Game $\mathbf{G}_1$: In this game, for all honest servers s, we switch the $\mathsf{NIZK}_{\mathsf{Gen}}$ proofs for

$$(\mathsf{PKE.pk}, \mathsf{ct}_j^{\mathsf{PAD},(s)}, C_{j,i}^{(s)}, \mathsf{ip.dk}^i_{(0,0)}, \mathsf{ip.dk}^i_{(0,1)})_{j,i\in[n]}$$

to the zero-knowledge setting (prove without knowing $\gamma^{(s,b)}_{\mathsf{IPE},j,i}$ and $z_j^{(s,b)}$). Any adversary that can distinguish between $\mathbf{G}_0$ and $\mathbf{G}_1$ is a distinguisher against the zero-knowledge property of $\mathsf{NIZK}_{\mathsf{Gen}}$.
Game $\mathbf{G}_2$: In this game, for all honest servers s, we switch the $\mathsf{NIZK}_{\mathcal{R}_{\mathsf{FS}}}$ proofs for

$$(\mathsf{VK}_s^{\mathsf{otp}}, \mathsf{PKE.pk}, C_{j,i}^{(s)})_{j,i\in[n]} \text{ and } (\mathsf{VK}_s^{\mathsf{otp}}, \mathsf{PKE.pk}, \mathsf{ct}_j^{\mathsf{PAD},(s)})_{j\in[n]}$$

to the zero-knowledge setting (prove without knowing $\gamma^{(s,b)}_{\mathsf{IPE},j,i} \cdot \mathsf{ip.dk}^i_{(0,0)} + z_j^{(s,b)} \cdot \mathsf{ip.dk}^i_{(0,1)}$ and $[\![z_j^{(s,b)}]\!]_1$). Any adversary that can distinguish between $\mathbf{G}_1$ and $\mathbf{G}_2$ is a distinguisher against the zero-knowledge property of $\mathsf{NIZK}_{\mathcal{R}_{\mathsf{FS}}}$.

$\mathbf{G}_0$, $\mathbf{G}_1$, $\mathbf{G}_{1.1}(s_l)$, $\mathbf{G}_{1.2}(s_l)$, $\mathbf{G}_{1.3}(s_l)$

$\mathsf{OMIX.Shuffle}(\mathsf{SK}^{\mathsf{mix}}_{s_l}, \mathsf{MBox}^{(s_l-1)}, (\mathsf{proof}^{(k)}, \mathsf{sig}^{(k)}_{\mathsf{mix}})_{k\in[s_l-1]})$[a]:

Each server $\mathcal{S}_s$ takes input $\mathsf{MBox}^{(s_l-1)}$ as

$$\{(C_{j,i}, \sigma_{j,i}, \Sigma_{\mathsf{PKE},j,i})_{j,i\in[n]}, (\tau_j, G_{1,j}, \Sigma_{\boldsymbol{G}_j})_{j\in[n]}, (C_{\mathsf{PKE},i})_{i\in[n]}, (\theta_i)_{i\in[n+1]}, G_0\}$$

1. It generates random $\gamma_{\mathsf{PKE},j,i}$ and randomises $C_{j,i}$ for each (j,i):
$$C'_{j,i} = C_{j,i} + \gamma_{\mathsf{PKE},j,i} \cdot C_{\mathsf{PKE},i} \qquad C'_{j,i} = \mathsf{PKE.Enc}(\mathsf{PKE.pk}, \mathbf{0}; r'_{j,i})$$
2. It adapts each signature $\sigma_{j,i}$ into $\sigma'_{j,i}$ for $\bar{C}'_{j,i} = ([\![1]\!]_1, C'_{j,i})$ accordingly
$$\sigma'_{j,i} \leftarrow \text{2-tag-}\mathsf{LHS.DerivSign}(\mathsf{LHS.vk}, (\tau_j, \theta_i), (\gamma_{\mathsf{PKE},j,i}, \bar{C}_{\mathsf{PKE},i}, \Sigma_{\mathsf{PKE},j,i}), (1, \bar{C}_{j,i}, \sigma_{j,i})).$$

   This step is omitted
3. It generates random β and randomises G_0 and $G_{1,j}$ for each j:
$$G'_0 = \beta \cdot G_0; \qquad G'_{1,j} = \beta \cdot G_{1,j} \qquad G'_0 \xleftarrow{\$} \mathbb{G}_1; \qquad G'_{1,j} \xleftarrow{\$} \mathbb{G}_1$$
4. It adapts each signature $\Sigma_{\boldsymbol{G}_j}$ into $\Sigma'_{\boldsymbol{G}_j}$ for $\boldsymbol{G}'_j = (G'_0, G'_{1,j})$:
$$\Sigma'_{\boldsymbol{G}_j} \leftarrow \text{2-tag-}\mathsf{LHS.DerivSign}(\mathsf{LHS.vk}, (\tau_j, \theta_{n+1}), (\beta, \boldsymbol{G}_j, \Sigma_{\boldsymbol{G}_j})).$$

   This step is omitted
5. It randomises each row tag τ_j into $\tau^\star_j$ and adapts the signatures $\sigma'_{j,i}, \Sigma_{\mathsf{PKE},j,i}, \Sigma'_{\boldsymbol{G}_j}$ accordingly
$$(\tau^\star_j, \bar{C}'_{j,i}, \sigma'^\star_{j,i}) \leftarrow \text{2-tag-}\mathsf{LHS.RandTag}(\mathsf{LHS.vk}, \tau_j, \bar{C}'_{j,i}, \sigma'_{j,i})$$
$$(\tau^\star_j, \bar{C}_{\mathsf{PKE},i}, \Sigma^\star_{\mathsf{PKE},j,i}) \leftarrow \text{2-tag-}\mathsf{LHS.RandTag}(\mathsf{LHS.vk}, \tau_j, \bar{C}_{\mathsf{PKE},i}, \Sigma_{\mathsf{PKE},j,i})$$
$$(\tau^\star_j, \boldsymbol{G}'_j, \Sigma^\star_{\boldsymbol{G}_j}) \leftarrow \text{2-tag-}\mathsf{LHS.RandTag}(\mathsf{LHS.vk}, \tau_j, \boldsymbol{G}'_j, \Sigma'_{\boldsymbol{G}_j})$$
$$(\tau^\star_j, \tilde{\tau}^\star_j) \leftarrow \text{2-tag-}\mathsf{LHS.NewTag}_{\mathcal{T}_1}(\mathsf{LHS.sk})$$
$$\sigma'^\star_{j,i} \leftarrow \text{2-tag-}\mathsf{LHS.Sign}(\mathsf{LHS.sk}, \tilde{\tau}^\star_j, \tilde{\theta}_i, \bar{C}'_{j,i})$$
$$\Sigma^\star_{\mathsf{PKE},j,i} \leftarrow \text{2-tag-}\mathsf{LHS.Sign}(\mathsf{LHS.sk}, \tilde{\tau}^\star_j, \tilde{\theta}_i, \bar{C}_{\mathsf{PKE},i})$$
$$\Sigma^\star_{\boldsymbol{G}_j} \leftarrow \text{2-tag-}\mathsf{LHS.Sign}(\mathsf{LHS.sk}, \tilde{\tau}^\star_j, \tilde{\theta}_{n+1}, \boldsymbol{G}'_j)$$
6. It arranges elements by the challenge permutation $\Pi_{s,b} : [n] \to [n]$
$$C^{(s)}_{\Pi(j),i} = C'_{j,i},\ \sigma^{(s)}_{\Pi(j),i} = \sigma'^\star_{j,i},\ \Sigma^{(s)}_{\mathsf{PKE},\Pi(j),i} = \Sigma^\star_{\mathsf{PKE},j,i},\ \tau^{(s)}_{\Pi(j)} = \tau^\star_j;$$
$$G^{(s)}_{1,\Pi(j)} = G'_{1,j};\ \Sigma^{(s)}_{\boldsymbol{G}_{\Pi(j)}} = \Sigma'_{\boldsymbol{G}_j};\ G^{(s)}_0 = G'_0$$
7. It computes $\mathsf{proof}^{(s)} \leftarrow \mathsf{NIZK}_{\mathsf{DH}}.\mathsf{Prove}\left(G_0, G^{(s)}_0, \sum_{j\in[n]} G_{1,j}, \sum_{j\in[n]} G^{(s)}_{1,j}; \beta\right)$

   $\mathsf{proof}^{(s)} \leftarrow \mathsf{NIZK}_{\mathsf{DH}}.\mathsf{Sim}\left(G_0, G^{(s)}_0, \sum_{j\in[n]} G_{1,j}, \sum_{j\in[n]} G^{(s)}_{1,j}\right)$
8. It signs the proofs as $\mathsf{sig}^{(s)}_{\mathsf{mix}} \leftarrow \mathsf{Sig.Sign}(\mathsf{SK}^{\mathsf{mix}}_s, \mathsf{proof}^{(s)})$.

The output $\mathsf{MBox}^{(s)} = \{(C^{(s)}_{j,i}, \sigma^{(s)}_{j,i}, \Sigma^{(s)}_{\mathsf{PKE},j,i})_{j,i\in[n]}, (\tau^{(s)}_j, G^{(s)}_{1,j}, \Sigma^{(s)}_{\boldsymbol{G}_j})_{j\in[n]}, (C_{\mathsf{PKE},i})_{i\in[n]}, (\theta_i)_{i\in[n+1]}, G^{(s)}_0\}$ and $\mathsf{proof}^{(s)}, \mathsf{sig}^{(s)}_{\mathsf{mix}}$.

[a] The input box $\mathsf{MBox}^{(s_l-1)}$ is verified and the game stops in case of invalidity.

Fig. 15. Modifications in the OMIX.Shuffle algorithm for the unlinkability game.

$\underline{\mathsf{OTP.Setup}(1^\lambda)}$:
$(\mathsf{PKE.sk}, \mathsf{PKE.pk}) \leftarrow \mathsf{PKE.SetUp}(1^\lambda)$; $\quad \mathsf{NIZK}_{\mathcal{R}_{\mathsf{FS}}}.\mathsf{pp} \leftarrow \mathsf{NIZK}_{\mathcal{R}_{\mathsf{FS}}}.\mathsf{SetUp}(1^\lambda)$
$\mathsf{NIZK}_{\mathcal{R}_{\mathsf{Gen}}}.\mathsf{pp} \leftarrow \mathsf{NIZK}_{\mathcal{R}_{\mathsf{Gen}}}.\mathsf{SetUp}(1^\lambda)$
Let $\mathsf{pp} = (\mathsf{PKE.pk}, \mathsf{NIZK}_{\mathcal{R}_{\mathsf{FS}}}.\mathsf{pp}, \mathsf{NIZK}_{\mathcal{R}_{\mathsf{Gen}}}.\mathsf{pp})$.

$\underline{\mathsf{OTP.KeyGen(pp)}}$: Each voter i for $i \in [n]$ prepares keys to be certified as

$$\mathsf{sk}_i := \perp;\ \mathsf{pk}_i := (\mathsf{id}_i, \mathsf{ip.dk}^i_{(0,1)}, \mathsf{ip.dk}^i_{(0,0)})$$

Each one-time pad server $\mathcal{S}^{\mathsf{otp}}_s$ for $s \in [N_{\mathsf{otp}}]$ generates Sig keys $(\mathsf{SK}^{\mathsf{otp}}_s, \mathsf{VK}^{\mathsf{otp}}_s) \leftarrow \mathsf{Sig.SetUp}(1^\lambda)$. The master public/secret key pair $(\mathsf{mpk}, \mathsf{msk})$ is set to be $(\perp, \perp)$[a].

$\underline{\mathsf{OTP.Certify}((\mathsf{pk}_i)_{i\in[n]}, (\mathsf{VK}^{\mathsf{otp}}_s)_{s\in[N_{\mathsf{otp}}]})}$: The authority simply registers servers' verification keys VK_s. The certification to a voter i is $\mathcal{CA}.\Sigma_i = \mathsf{pk}_i$. The initial box is $\mathsf{OBox}^{(0)} = \{\mathcal{CA}.\Sigma_i\}_{i\in[n]}$.

$\underline{\mathsf{OTP.Gen}(\mathsf{SK}^{\mathsf{otp}}_s, \mathsf{OBox}^{(s-1)})}$: Each server $\mathcal{S}^{\mathsf{otp}}_s$ processes as follows

1. It generates random $\gamma^{(s)}_{\mathsf{IPE},j,i}, z^{(s)}_j$ and ciphertexts
$$\mathsf{ct}^{\mathsf{PAD},(s)}_j \leftarrow \mathsf{PKE.Enc}(\mathsf{PKE.pk}, [\![z^{(s)}_j]\!]_1)\ \forall j \in [n]$$
$$C^{(s)}_{j,i} \leftarrow \mathsf{PKE.Enc}(\mathsf{PKE.pk}, \gamma^{(s)}_{\mathsf{IPE},j,i} \cdot \mathsf{ip.dk}^i_{(0,0)} + z^{(s)}_j \cdot \mathsf{ip.dk}^i_{(0,1)})\ \forall j, i \in [n]$$
2. It generates a proof for correct generation: $\forall j, i \in [n]$
$$\mathsf{proof}^{(s)}_{j,i} \leftarrow \mathsf{NIZK}_{\mathsf{Gen}}.\mathsf{Prove}(\mathsf{PKE.pk}, \mathsf{ct}^{\mathsf{PAD},(s)}_j, C^{(s)}_{j,i}, \mathsf{ip.dk}^i_{(0,0)}, \mathsf{ip.dk}^i_{(0,1)}; \gamma^{(s)}_{\mathsf{IPE},j,i}, z^{(s)}_j).$$
3. It generates Fiat-Shamir-based proofs of plaintext:
$$\pi^{\mathsf{FS},(s)}_{j,i} \leftarrow \mathsf{NIZK}_{\mathcal{R}_{\mathsf{FS}}}.\mathsf{Prove}(\mathsf{VK}^{\mathsf{otp}}_s, \mathsf{PKE.pk}, C^{(s)}_{j,i}; \gamma^{(s)}_{\mathsf{IPE},j,i} \cdot \mathsf{ip.dk}^i_{(0,0)} + z^{(s)}_j \cdot \mathsf{ip.dk}^i_{(0,1)}).$$
$$\pi'^{\mathsf{FS},(s)}_j \leftarrow \mathsf{NIZK}_{\mathcal{R}_{\mathsf{FS}}}.\mathsf{Prove}(\mathsf{VK}^{\mathsf{otp}}_s, \mathsf{PKE.pk}, \mathsf{ct}^{\mathsf{PAD},(s)}_j; [\![z^{(s)}_j]\!]_1).$$
4. It signs the proofs as $\mathsf{sig}^{(s)}_{\mathsf{otp}} \leftarrow \mathsf{Sig.Sign}(\mathsf{SK}^{\mathsf{otp}}_s, (\mathsf{proof}^{(s)}_{j,i}, \pi^{\mathsf{FS},(s)}_{j,i})_{j,i\in[n]}, (\pi'^{\mathsf{FS},(s)}_j)_{j\in[n]})$

The output box $\mathsf{OBox}^{(s)}$ is

$$\mathsf{OBox}^{(s-1)} || (C^{(s)}_{j,i}, \mathsf{proof}^{(s)}_{j,i}, \pi^{\mathsf{FS},(s)}_{j,i})_{j,i\in[n]}, (\mathsf{ct}^{\mathsf{PAD},(s)}_j, \pi'^{\mathsf{FS},(s)}_j)_{j\in[n]}, \mathsf{sig}^{(s)}_{\mathsf{otp}}.$$

$\underline{\mathsf{OTP.Verif}(\mathsf{OBox}^{(0)}, \mathsf{OBox}^{(N_{\mathsf{otp}})})$}: The verification continues unless $\mathsf{OBox}^{(0)}$ and $\mathsf{OBox}^{(N_{\mathsf{otp}})}$ can not be parsed as $\{\mathcal{CA}.\Sigma_i\}_{i\in[n]}$ and $\{\mathcal{CA}.\Sigma_i\}_{i\in[n]} || ((C^{(s)}_{j,i}, \mathsf{proof}^{(s)}_{j,i}, \pi^{\mathsf{FS},(s)}_{j,i})_{j,i\in[n]}, (\mathsf{ct}^{\mathsf{PAD},(s)}_j, \pi'^{\mathsf{FS},(s)}_j)_{j\in[n]}, \mathsf{sig}^{(s)}_{\mathsf{otp}})_{s\in[N_{\mathsf{otp}}]}$, respectively. The verification outputs 1 for validity if:

- $\mathsf{Sig.Verify}(\mathsf{VK}^{\mathsf{otp}}_s, (\mathsf{proof}^{(s)}_{j,i}, \pi^{\mathsf{FS},(s)}_{j,i})_{j,i\in[n]}, (\pi'^{\mathsf{FS},(s)}_j)_{j\in[n]}, \mathsf{sig}^{(s)}_{\mathsf{otp}}) = 1$ for $s \in [N_{\mathsf{otp}}]$;
- $\mathsf{NIZK}_{\mathsf{Gen}}.\mathsf{Verify}(\mathsf{proof}^{(s)}_{j,i}) = \mathsf{NIZK}_{\mathcal{R}_{\mathsf{FS}}}.\mathsf{Verify}(\pi^{\mathsf{FS},(s)}_{j,i}) = \mathsf{NIZK}_{\mathcal{R}_{\mathsf{FS}}}.\mathsf{Verify}(\pi'^{\mathsf{FS},(s)}_j) = 1$ for $j, i \in [n]$ and $s \in [N_{\mathsf{otp}}]$.

[a] No secret-key-involved action from the certificate authority is required.

Fig. 16. Setup, key generation, certification and one-time-pad generation.

Game $\mathbf{G}_2(s_l)$: The modification in this game is made by the honest server s_l (with l running in $l = N - |J|, \ldots, 1$). The server generates ciphertexts $\mathsf{ct}_j^{\mathsf{PAD},(s)}$ and $C_{j,i}^{(s)}$ that encrypt $\mathbf{0}$, instead of $[\![z_j^{(s,b)}]\!]_1$ and $\gamma_{\mathsf{IPE},j,i}^{(s,b)} \cdot \mathsf{ip.dk}_{(0,0)}^i + z_j^{(s,b)} \cdot \mathsf{ip.dk}_{(0,1)}^i$, respectively. Any adversary that can distinguish between $\mathbf{G}_2(s_{l-1})$ and $\mathbf{G}_2(s_l)$ is an attacker against the IND-CPA security of the PKE scheme. □

Corollary 2 (OPAD Strong Unlinkability). *The unlinkability of the OPAD protocol holds even if, prior to outputting its guess on the challenge bit (as defined in Definition 28), the adversary $\mathcal{A}$ is provided with an oracle* OCorPad *that decrypts the ciphertexts $\{\mathsf{ct}_j^{\mathsf{PAD},(s)}\}_{j\in[n]}$ and $\{C_{j,i}^{(s)}\}_{j,i\in[n]}$ that are applied by the corrupted servers $s \in \mathcal{J}$ during the box-appending loop with the challenger.*

Proof. For the transitions between games: $\mathbf{G}_0$ and $\mathbf{G}_1$, $\mathbf{G}_1$ and $\mathbf{G}_2$, which rely on the zero-knowledge property of the underlying proof systems, the challenger can simply use the PKE decryption key to decrypt corrupted servers' ciphertexts $\{\mathsf{ct}_j^{\mathsf{PAD},(s)}\}_{j\in[n],s\in\mathcal{J}}$ and $\{C_{j,i}^{(s)}\}_{j,i\in[n],s\in\mathcal{J}}$.

For the transition between $\mathbf{G}_2(s_{l-1})$ and $\mathbf{G}_2(s_l)$, which relies on the IND-CPA security of the PKE scheme, the challenger cannot have access to the decryption key anymore. Instead, it uses the random oracle in the $\mathsf{NIZK}_{\mathcal{R}_{\mathsf{FS}}}$ proof system to extract the plaintexts within ciphertexts of all corrupted servers via rewinding the adversary $\mathcal{A}$. On the other hand, the challenger uses the random oracle to simulate the proofs for the ciphertexts $\{\mathsf{ct}_j^{\mathsf{PAD},(s_l)}\}_{j\in[n]}$ and $\{C_{j,i}^{(s_l)}\}_{j,i\in[n]}$ generated by the honest server s_l. As we hardwire each server's identity $\mathsf{VK}_s^{\mathsf{otp}}$ in the $\mathcal{RO}$-query for proofs, an argument similar to that presented in the proof of Lemma 1 shows that there will be no collisions in assigning $\mathcal{RO}$ input/output for zero-knowledge simulation and plaintext extraction at the same time. Hence the proof is completed. □

E.4 Integrating Offline Protocols in Voting

With both offline protocols, OPAD and OMIX, available, we now state the following theorem, which establishes the ballot privacy of an OMIX-based voting scheme in the multi-authority setting.

Theorem 13. *The OMIX-based voting framework, as described in Section 4.1, can be instantiated using the OMIX protocol from Section 5.3 and the OPAD protocol from Section E.3. Furthermore, assuming that at least one OMIX server and one OPAD server are honest, the resulting voting protocol achieves ballot privacy in the selective and symmetric-key setting.*

Proof. The proof strategy is somewhat similar to that for the ballot privacy of the framework (see Theorem 5), except that we modify the behavior of honest servers when they are interleaved with corrupted ones in the OMIX and OPAD protocols. By switching any NIZK systems to their zero-knowledge setting, we first follow the games $\mathbf{G}_1$ and $\mathbf{G}_2$ as in the proof of Theorem 5. We present the modifications in the game $\mathbf{G}_3$ below:

Game $\mathbf{G}_3$: The goal is to remove information in the following PKE ciphertexts:

1. The ciphertexts encrypting the vote-slot decryption keys $(\mathsf{ip.dk}_{(1,0)}^i)_{i\in\mathcal{HI}}$.
2. The ciphertexts corresponding to the permutation images $(\Pi(i))_{i\in\mathcal{HI}}$ of honest mix-servers during OMIX.
3. The ciphertexts corresponding to the pads

$$([\![z_j]\!]_1)_{j\in[n]} \text{ and } \left(\gamma_{\mathsf{IPE},j,i} \cdot \mathsf{ip.dk}_{(0,0)}^i + z_j \cdot \mathsf{ip.dk}_{(0,1)}^i\right)_{j\in[n],\, i\in\mathcal{HI}}$$

of honest servers during OPAD.

The challenger implements the following modifications:

- For QNewHonest queries, the challenger replaces the ciphertexts ct_i with encryptions of $\mathbf{0}$.
- For a QPKProcess query, the challenger modifies the protocols as follows:
  - OMIX: Each honest mix-server s_M generates random row tags $(\tau_j^\star)_{j\in[n]}$, random encryptions $(C'_{j,i})_{j,i\in[n]}$ of $\mathbf{0}$, and random row verification keys $(G'_0, (G'_{1,j})_{j\in[n]})$, and produces signatures using the signing key LHS.sk. It then applies its permutation Π_{s_M} to the rows. Since the shuffled elements remain random encryptions, random row verification keys, and random row tags, no information about the applied permutation is leaked in this round. This is the last hybrid in the unlinkability proof of OMIX.

- OPAD: Each honest server s_P generates ciphertexts $(\mathsf{ct}_j^{\mathsf{PAD},(s_P)})_{j\in[n]}$ and $(C_{j,i}^{(s_P)})_{j,i\in[n]}$ that encrypt $\mathbf{0}$, instead of $([\![z_j^{(s_P)}]\!]_1)_{j\in[n]}$ and $\left(\gamma_{\mathsf{IPE},j,i}^{(s_P)} \cdot \mathsf{ip.dk}_{(0,0)}^i + z_j^{(s_P)} \cdot \mathsf{ip.dk}_{(0,1)}^i\right)_{j\in[n],\, i\in\mathcal{HI}}$, respectively, thereby eliminating information about the contributed one-time pads and key randomness. This is the last hybrid in the unlinkability proof of OPAD.

At the end of the process, the challenger outputs the functional keys $(\mathsf{ip.dk}_j^i)_{j,i\in[n]}$ and the one-time pads $[\![\boldsymbol{z}]\!]_1$ as if they were generated without any modifications by the honest servers. The indistinguishability between $\mathbf{G}_3$ and its preceding game follows from (i) a hybrid argument based on the IND-CPA security of the PKE scheme (to modify QNewHonest queries), (ii) the sequence of hybrids used to prove unlinkability in the OMIX protocol, and (iii) the sequence of hybrids used to prove the unlinkability in the OPAD protocol.

During the simulation of each hybrid, the challenger may need to extract corrupted inputs from the adversary to eventually output $(\mathsf{ip.dk}_j^i)_{j,i\in[n]}$ and $[\![\boldsymbol{z}]\!]_1$ as generated in the real game. The extractability of OMIX (for permutations), OPAD (for one-time pads and key randomness), and the Σ-protocol proof for ct_i (for secret $\mathsf{ip.dk}_{(1,0)}^i$ in the OMIX protocol) ensures that $(\mathsf{ip.dk}_j^i)_{j,i\in[n]}$ and $[\![\boldsymbol{z}]\!]_1$ can be simulated.

By extracting corrupted inputs for simulation, the challenger retains the programmability of jointly contributed components, including permutations, one-time pads, and key randomness. With this programmability, the proof follows a similar sequence of hybrids based on the IND-CPA security of the FH-IPFE scheme, as used in transitioning from $\mathbf{G}_3$ to $\mathbf{G}_4$ in the proof of Theorem 5. □

Besides ensuring ballot privacy, the OMIX-based voting scheme with multiple servers preserves the properties of self-tallying, client-scalability, and corrective fault tolerance. The universal verifiability is also achieved thanks to the soundness properties of both the OMIX and OPAD protocols.